Graduate scholarship application system



Name: Bi Qiaoyang

SCN:237160401

Class:21SD01

Project Proposal

Title	Beihang University undergraduate scholarship application management system
Aim	Students who apply for scholarships for Beihang University register, log in, enter personal information and personal data and other functions on the website, and upload it to the college database, compare the scholarship application conditions issued by the college, and then decide whether to issue scholarships to students.
Objectives	**1. Student Role:** 1) Personal information inquiry 2) Fill out the scholarship application form 3) Submit scholarship application documents 4) Modify the scholarship application information 5) Check the scholarship application results 6) Check the scholarship application process, time, and status **2. Mentor role** 1) Manage students' basic personal information 2) Review the student's scholarship application form and submit it to the department 3) Check the scholarship application materials of students and submit them to the department 4) Give suggestions to students, write them into the form and send them back 5) Preliminary screening of students who do not meet the requirements 6) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page, and explain why the application was accepted or rejected **Third, the role of the department** 1) Collect the preliminarily reviewed application materials collected by all tutors in the department 2) Conduct a secondary review to screen students who do not meet the requirements. 3) Submit to College Department for final review 4) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page, and explain why the application was accepted or rejected **Fourth, the role of the College department** 1) Provide scholarship application conditions

	2) Collect the selected student application forms and materials of each department 3) Make the final decision 4) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page, and explain why the application was accepted or rejected
Content	Detail project information, including: 1） In the current higher education environment, scholarship is an important means of motivating students to study hard. In order to manage the student scholarship application process more effectively, improve management efficiency and enhance fairness, I plan to develop a scholarship application management system for undergraduates of Beihang University. This is aimed at the students of the school, the class teacher and the judging department. The project is scheduled to begin on November 27, 2023 and be completed on April 10, 2024. 2） This project will use a laptop to write programs in flask in Pycharm2023.2.3, with jsp language from Eclipse2023-09 as an alternative. The database will use the mysql8.0.13 database to store data.
Methodology	1) At the beginning of the project, I will search the official website of Beihang University for the requirements for students to apply for scholarships, as well as the process of applying for scholarships. And refer to the scholarship application system of this school and other schools to study. 2) at the beginning of the project research, I plan on major video platform and https://tutorial.helloflask.com/, https://www.w3school.com.cn/, https://www.csdn.net/ and other large programming tutorial websites for flask and jsp self-study.
Proposal Submitted to	**Zhao Xingmei**
Date Submitted	**2023/12/3**
Proposal Result	**Pass/Fail**
Supervisor Signature:	

Content

1. Planning Part 3
1.1.1. Project task description 3
1.1.2. System design and architecture 3
1.1.2.1. Functional Requirements 3
1.1.2.2. Non-Functional Requirements 4
1.1.2.3. Use Case Diagram 4
1.1.3.1. Research similar projects 5
1.1.3.2. Questionnaire 6
1.1.3.3. Invitation Poster 14
2. Models 18
2.1 Identification of classes, methods and attribute
2.2 Top level Use Case Model 24
2.3 UML 27
2.4 Sequence Diagram 27
2.5 GUI design and HCI 30
3. Coding & Test Part 38
3.1. Business Model Code Listing 38
3.2. Viewing Model Code Listing 39
3.2.1. Login Page 39
3.2.2. User role 40
3.2.3. Admin role 51
3.2.4. Mentor role 59
3.2.5. Faculty role 63
3.2.6. College role 66
3.3. Use of unfamiliar construct 70
3.4. Error handling in coding 71
3.5. Internal Documentation 71
3.6. Testing 71

3.6.1. Login 72
3.6.2. User Role 73
3.6.3. Admin Role 75
3.6.4. Mentor role 76
3.6.5. Faculty role 77
3.6.6. College role 77
3.7. Debugging 78
3.7.1. Test No.22 78
3.7.2. Test No.26 79
3.7.3. Test No.28 80
3.7.4. Test No.31 82
3.7.5. Test No.34 84
3.8. User documentation 85
4. Evaluation 89
5. References 93
6. History of changes to Report 93

1. Planning Part

1.1. Interpretation of the project assignment brief.

1.1.1. Project task description

At the beginning of the project, I first looked for similar websites on the Internet for observation and learning, and determined the general framework. Write out the general database and the relationships between the databases, such as foreign key constraints.

In the middle of the project, first design the login and registration pages of each role, and write out routes for these pages. Then write out the steps for the user to apply for the scholarship, and how to pass this information to the database and use other roles for approval.

In the middle and later stages of the project, debugging tests are performed and errors found are corrected. Refine the code and put it together.

Finally, writing css files makes the entire page look better and easier to use.

1.1.2. System design and architecture

1.1.2.1. Functional Requirements

1. Student Role:

1) Personal information inquiry

2) Fill out the scholarship application form

3) Submit scholarship application documents

4) Modify the scholarship application information

5) Check the scholarship application results

6) Check the scholarship application process, time, and status

2. Mentor role

1) Manage students' basic personal information

2) Review the student's scholarship application form and submit it to the department

3) Check the scholarship application materials of students and submit them to the department

4) Give suggestions to students, write them into the form and send them back

5) Preliminary screening of students who do not meet the requirements

6) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page

Department role

1) Collect the preliminarily reviewed application materials collected by all tutors in the department

2) Conduct a secondary review to screen students who do not meet the requirements.

3) Submit to College Department for final review

4) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page

College role

1) Provide scholarship application conditions

2) Collect the selected student application forms and materials of each department

3) Make the final decision

4) Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page

1.1.2.2. Non-Functional Requirements

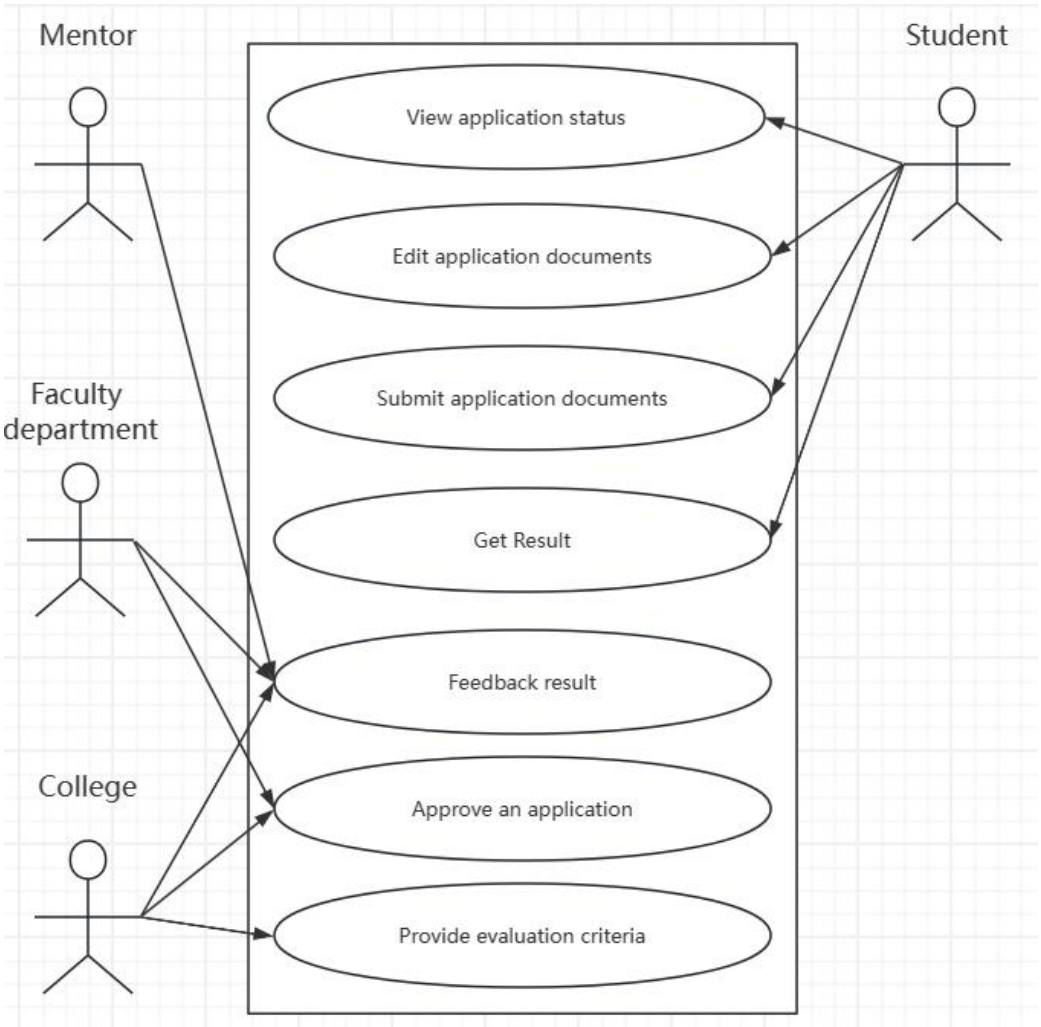
1. Prompt the user when incorrect data is inserted

2. Pages should be continuous and related

3. The value cannot be empty when inserting, if it is empty, the user should be prompted to fill in (except the award time and name).

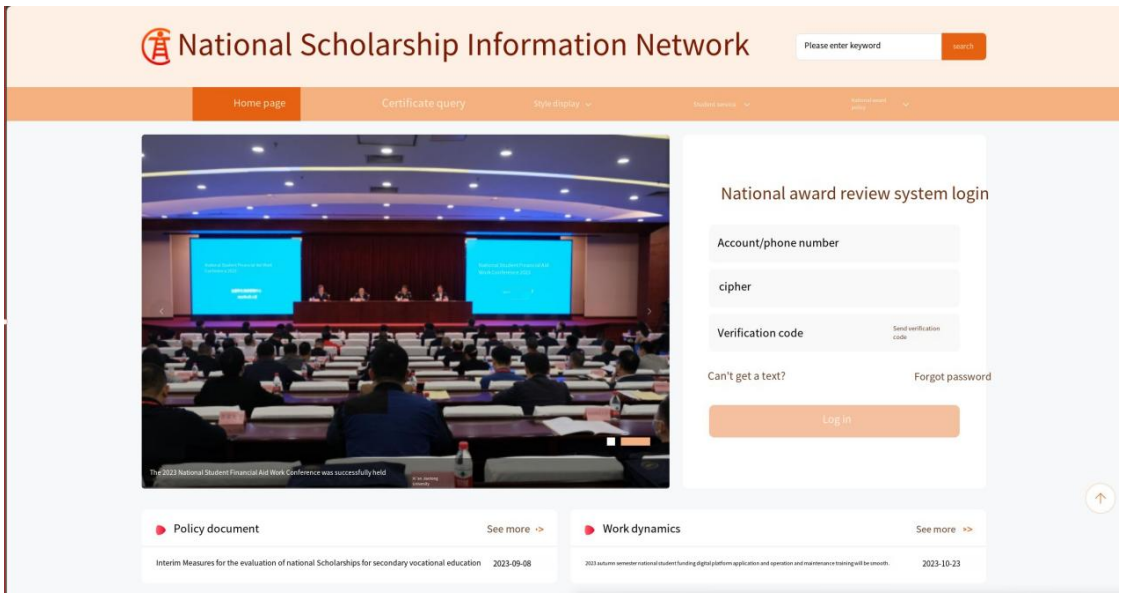
4. Prompt the user with each success message before proceeding to the next step

1.1.2.3. Use Case Diagram



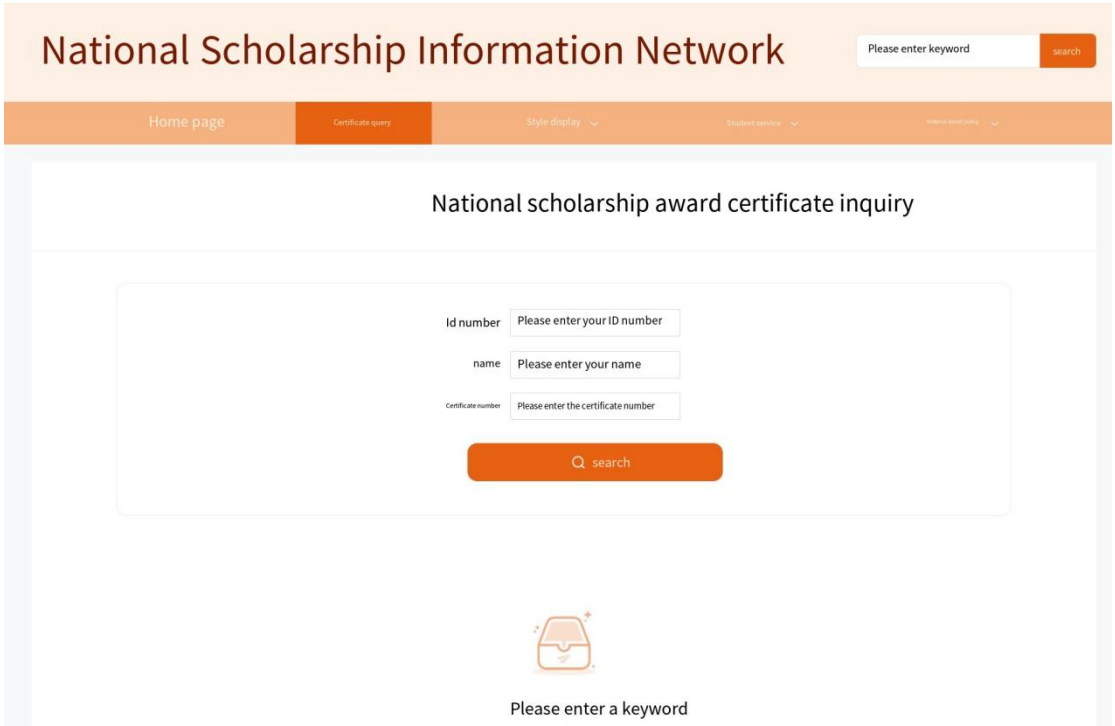
1.1.3. Findings and Background Research

1.1.3.1. Research similar projects

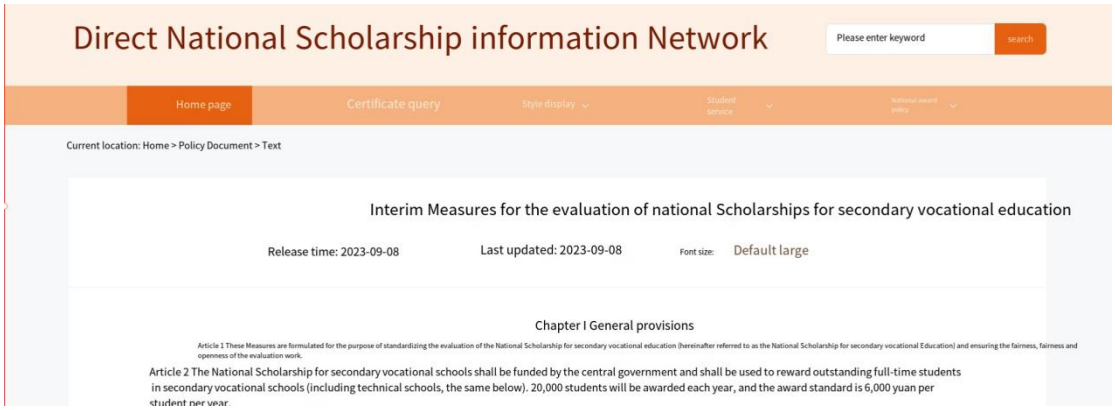


https://gjps.xszz.moe.edu.cn/#/index

In this viewing page, the page implements user login, user registration, and review guidelines. As well as providing enquiries on certificates and approval results, as well as updates on school work and policies. Putting this information on the home page can help users better apply for scholarships



In the part of the query website only the simplest database query is used to find whether the corresponding user role has scholarships or certificates, which is the most effective and easiest way. I can also use this method in my project.



For the criteria for application, the website has a separate page to display this information. This is to allow the user character to better prepare relevant materials and check in advance if they meet the scholarship application criteria.

Conclusion

This web page provides user login, user registration and review guide functions, and provides query services on certificates and approval results, as well as updates on school work and policies. Putting this information on the front page can help users better apply for scholarships. In the query page section, only the simplest database query is used to determine whether the corresponding user role has a scholarship or certificate, which is the most efficient and easiest way to use in the project. In terms of application criteria, the website has a separate page showing this information, which allows users to better prepare relevant materials and check in advance if they are eligible for the scholarship application.

1.1.3.2. Questionnaire

Beihang University Graduate Student Application Scholarship System

* 1. What is your occupation?

- ○ Graduate student
- ○ Learned scholar
- ○ Doctoral candidate
- ○ Teacher
- ○ Faculty Department staff
- ○ Faculty members

* 2. Have you ever applied for a scholarship before

- ○ Yes
- ○ No

* 3. What is your purpose of applying for the scholarship

- ○ Subsidized tuition
- ○ Family financial difficulties
- ○ Others

* 4. Do you think we should apply for scholarships through the website?

- ◯ Yes
- ◯ No

* 5. What features do you think you need

- ☐ View progress at any time
- ☐ Change information at any time
- ☐ Contact the teachers
- ☐ Others

* 6. What problems have you encountered during the application process so far

- ☐ Teachers cannot be reached often
- ☐ Not clear on the grading criteria
- ☐ Progress cannot be tracked at any time
- ☐ Application file lost
- ☐ Others

* 7. Where do you want the selection criteria to be

- ◯ Main Page
- ◯ A separate page is used to store standards

* 8. As a faculty member, how would you like to review student applications

- ◯ Approval on the computer
- ◯ Approval is done on paper and the result is entered into the computer

* 9. As a student, if your application is rejected, do you need to submit additional information or the opportunity to revise your information

- ◯ Yes
- ◯ No

* 10. As a student, if your application is rejected, do you need to know the reason for the rejection and show it on the website

- ◯ Yes
- ◯ No

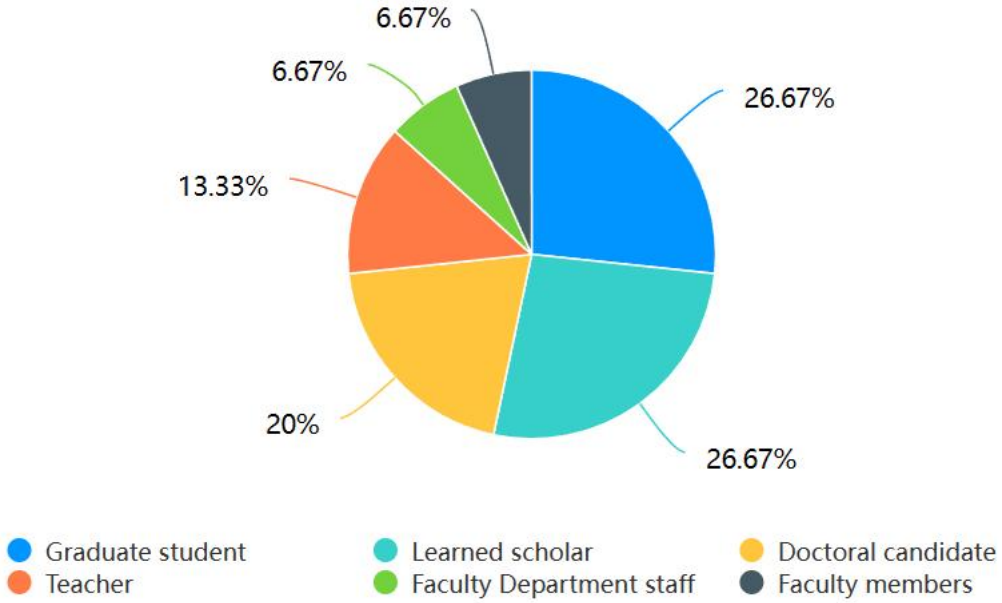
* 11. As a student, do you need a home page

- ◯ Yes
- ◯ No

* 12. Do you need update your application information after you submit?

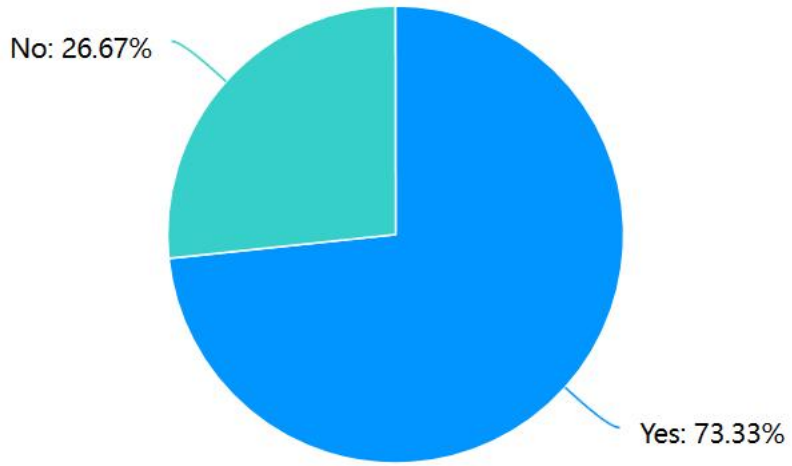
- ◯ Yes
- ◯ No

Question 1:What is your occupation?



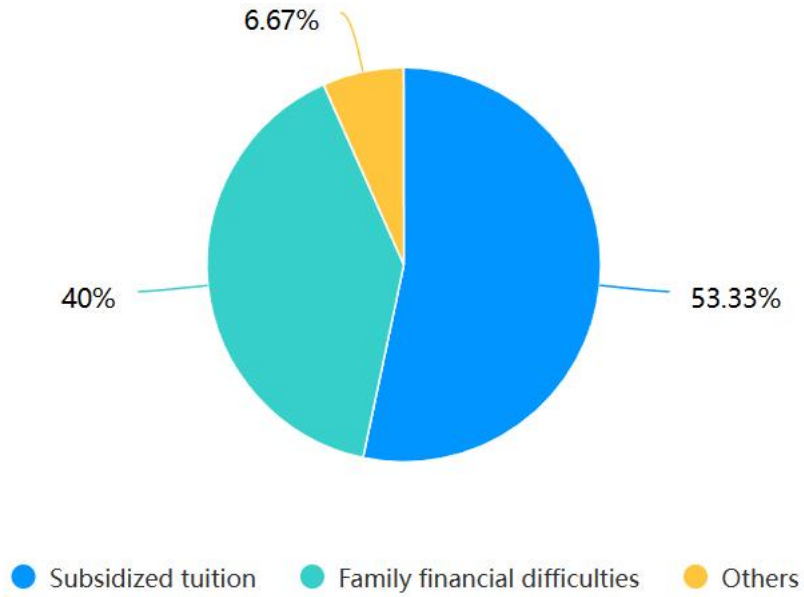
In this question, the proportion of graduate students and undergraduates is close, and the second place is teachers. This also means that these groups need a web-based application system. Since then, the main object of this time is the student user role.

Question 2:Have you ever applied for a scholarship before?



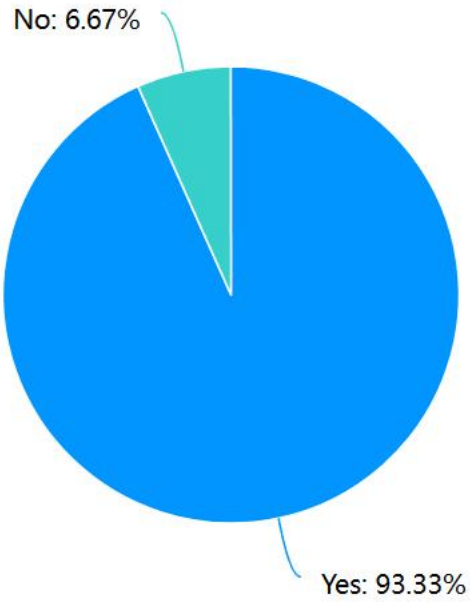
Most of them have applied for scholarships, accounting for 73.33%. But there are also a small number of people who do not apply for scholarships. Therefore, this system should focus on the efficiency, simplicity and ease of operation of the application process, rather than a variety of fresh styles.

Question 3: What is your purpose of applying for the scholarship?



More than half of the users apply for scholarships in order to subsidize tuition fees (53.33%), and a large part of the rest need scholarships because of family financial difficulties. Only a small percentage apply for scholarships for other reasons.

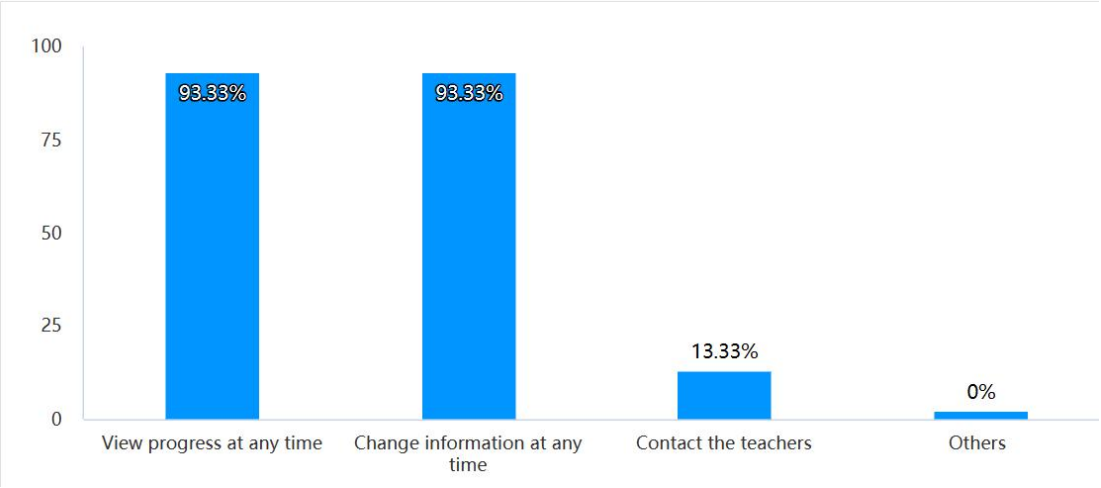
Question 4: Do you think we should apply for scholarships through the website?



In this questionnaire, most people have already applied for scholarships through the website, which means that they can skillfully handle every step of applying on the website. The purpose of this website will also make the way to operate more concise.

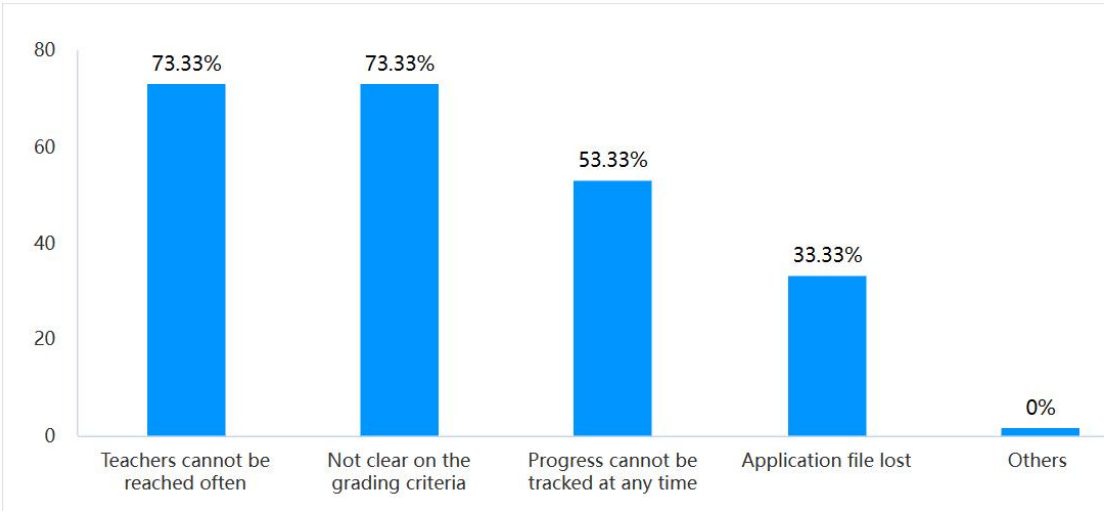
Question 5: What features do you think you need?

Question5: What feature do you think you need ?



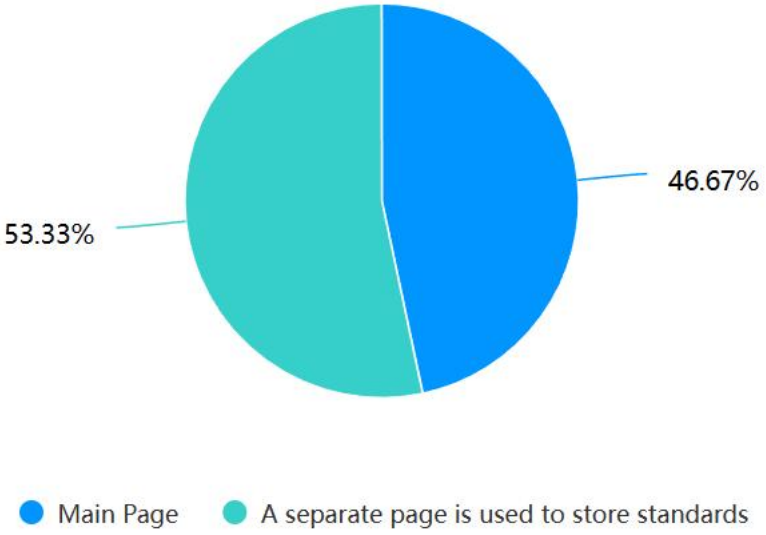
In this case, the most requested feature is the ability to check the status of the scholarship application at any time and to change the application information at any time. The development of this project will also focus on writing these two functions in the user role.

Question 6: What problems have you encountered during the application process so far?



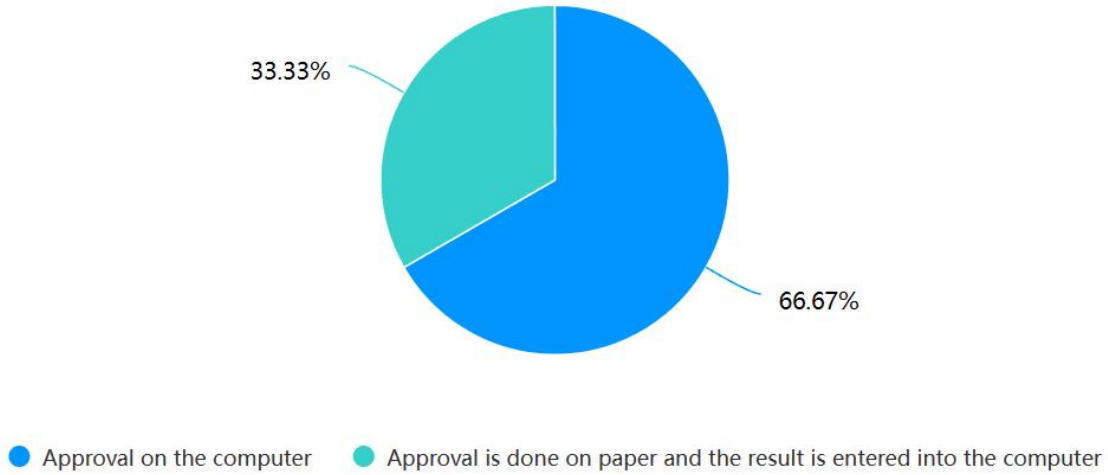
In this problem, most of the students were unable to communicate with the teacher and were not clear about the grading criteria (73.33%), which made it difficult for the role to submit relevant information and preliminarily assess whether they were qualified. More than half of people have also encountered the problem of "not being able to query the review process". In the process of audit, everyone is just waiting, and some people want to know what step is in progress, rather than waiting until the final announcement of the result to know whether they have successfully applied. At the same time, a small number of people have experienced the loss of application information (33.33%), and all application information on the website will be saved in the database and will not be lost.

Question 7: Where do you want the selection criteria to be



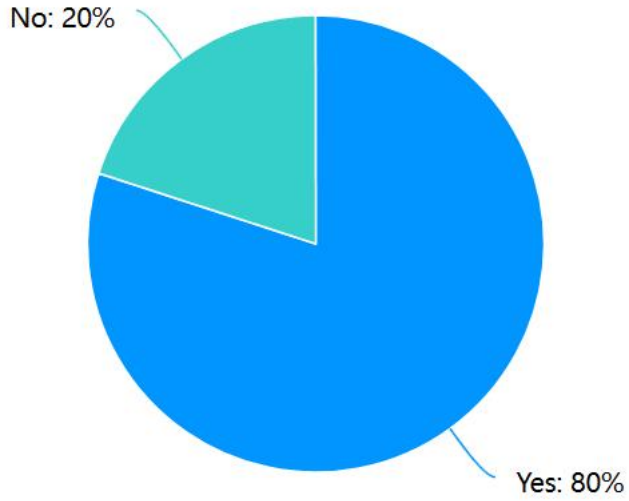
Regarding whether to put the application criteria on a separate page, the number of people on both sides is basically the same, but the number of people who need a separate page is superior (53.55%). The application criteria of this system will be displayed in a new page to avoid excessive text so that users can not see their personal information.

Question 8: As a faculty member, how would you like to review student applications?



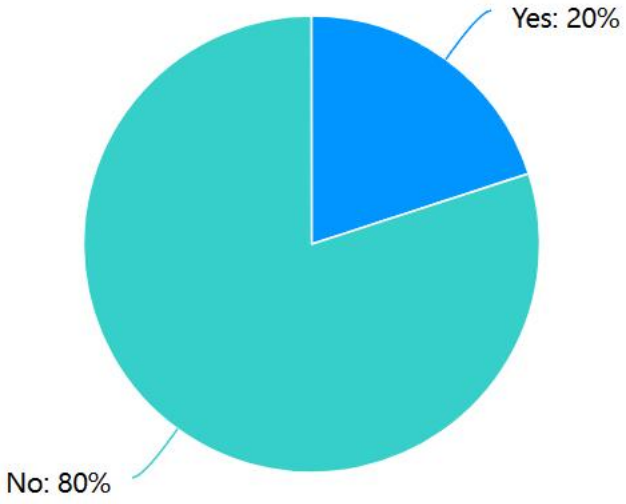
Nearly two-thirds of teachers want computers to look up and approve student applications, which is more efficient and less likely to lose student data.

Question 9: As a student, if your application is rejected, do you need to submit additional information or the opportunity to revise your information?



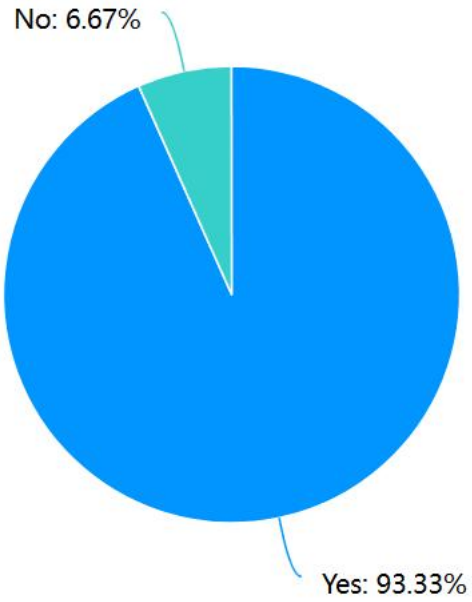
The majority of students want to submit an additional document to see if they are qualified after being rejected (80%), but the latest application criteria state that they cannot submit it again after being rejected and can only wait until the next scholarship application. This system will be made according to the latest application standards, and the function of submitting updated materials will be realized before the teacher's approval.

Question 10:As a student, if your application is rejected, do you need to know the reason for the rejection and show it on the website?



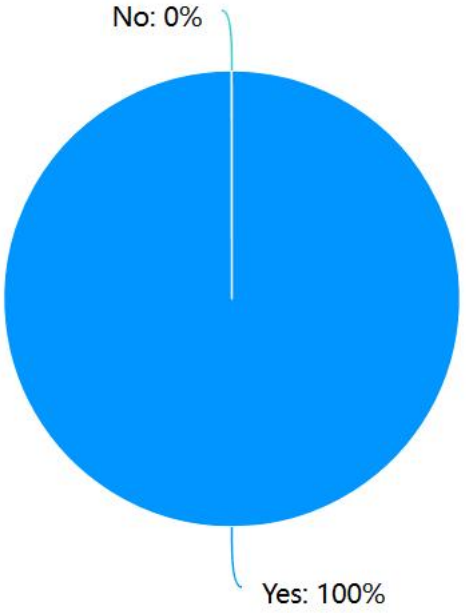
80% of student users indicated that they did not need to show the rejected question on the page, and the remaining 20% were students who wanted to show the reason for the rejection

Question 11: As a student, do you need a home page?



Almost all student users would like to have a user page (93.33%), and will consider how to design the user page during the development phase, rather than just filling out the application information.

Question 12: Do you need update your application information after you submit?



In this case, everyone wants to be able to resubmit the application document, which can also effectively prevent the student user from submitting the wrong information, or encounter other situations during the application and need to change the information.

1.1.3.3. Invitation Poster



1.1.4. Aims of the project assignment

The aim of this system website is to register, log in, enter personal information and personal data on the web page for students applying for scholarships at Beihang University, and upload it to the database of the college, compare whether the scholarship application conditions issued by the college are qualified, and then decide whether to issue scholarships to students.

1.1.5. Resources and Materials

Environment: Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32

Software: Pycharm2023.2.3 Professional Edition, MySQL workbench 8.0 CE

Programming Language: flask for writing route of websites, html is used to write the content of web pages.

Hardware tool: Legion Y9000P IAH7H, processor 12th Gen Intel(R) Core(TM) i9-12900H 2.50 GHz, Tape RAM 16.0 GB

1.1.6. Identification of information sources

Pycharm2023.2.3 Professional Edition: https://www.jetbrains.com/pycharm/

MySQL workbench: https://dev.mysql.com/downloads/workbench/

Python 3.10.2: https://www.python.org/

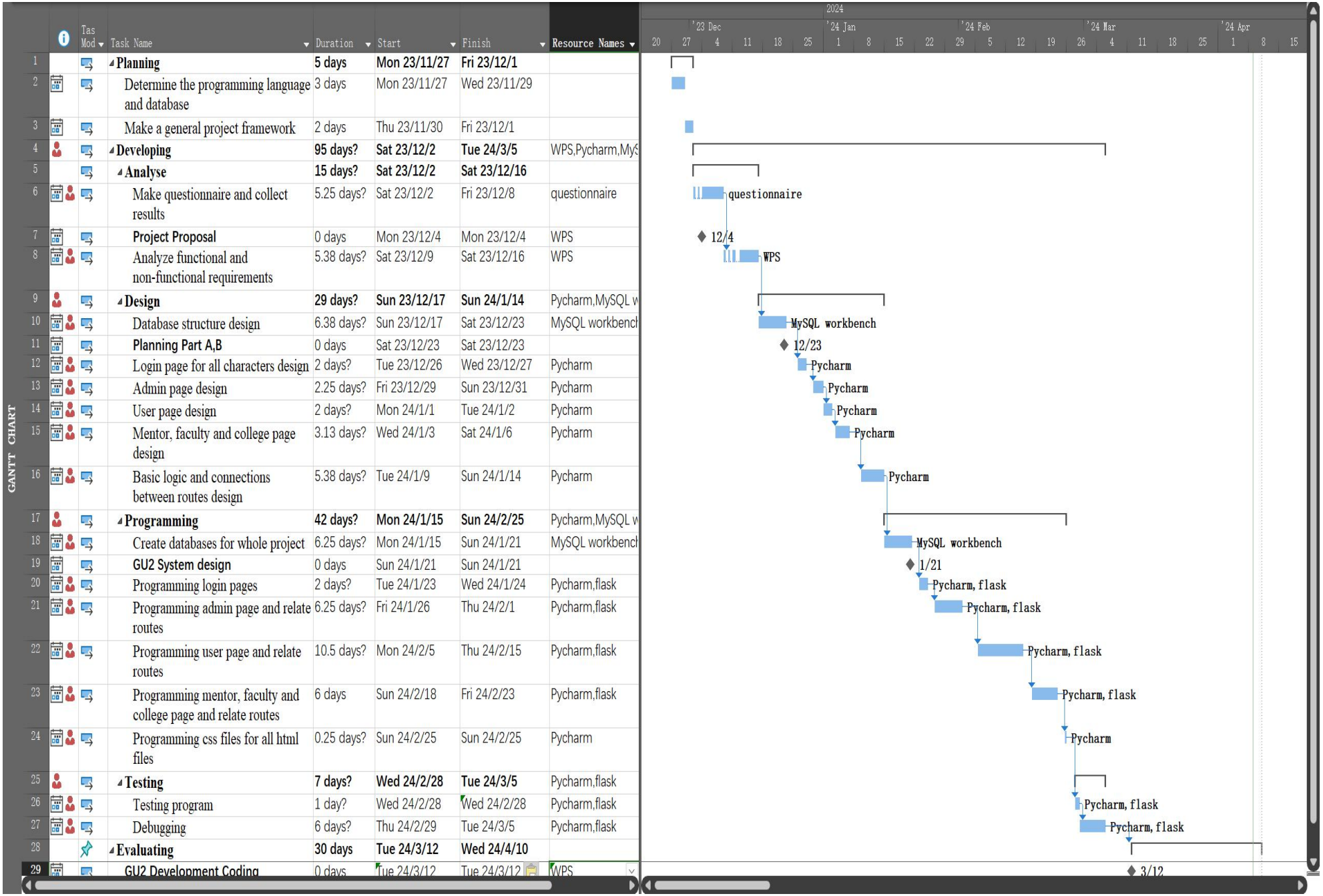
1.2. Project Plan

1.2.1. Stage Schedules

WBS

Task	**Time**
Planning Stage	**2023/11/27 ~ 2023/12/1**
Determine the programming language and database	2023/11/27 ~ 2023/11/29
Make a general project framework	2023/11/30 ~ 2023/12/1
Developing	**2023/12/2 ~ 2024/3/5**
Analyse	**2023/12/2 ~ 2023/12/16**
Make questionnaire and collect results	2023/12/2 ~ 2023/12/8
Analyze functional and non-functional requirements	2023/12/9 ~ 2023/12/16
Design	**2023/12/17 ~ 2024/1/14**
Database structure design	2023/12/17 ~ 2023/12/25
Login page for all characters design	2023/12/26 ~ 2023/12/27
Admin page design	2023/12/28 ~ 2023/12/31
User page design	2024/1/1 ~ 2024/1/2
Mentor, faculty and college page design	2024/1/3 ~ 2024/1/6
Basic logic and connections between routes design	2024/1/7 ~ 2024/1/14
Programming	**2024/1/15 ~ 2024/2/25**
Create databases for whole project	2024/1/15 ~ 2024/1/22
Programming login pages	2024/1/23 ~ 2024/1/24
Programming admin page and relate routes	2024/1/25 ~ 2024/2/1
Programming user page and relate routes	2024/2/2 ~ 2024/2/15
Programming mentor, faculty and college page and relate routes	2024/2/16 ~ 2024/2/23
Programming css files for all html files	2024/2/24 ~ 2024/2/25
Testing	**2024/2/26 ~ 2024/3/5**
Testing program	2024/2/26 ~ 2024/2/28
Debugging	2024/2/29 ~ 2024/3/5
Evaluating	**2024/3/6 ~ 2024/4/10**

Gantt Chart



	Tas Mod	Task Name	Duration	Start	Finish	Resource Names
1		**Planning**	**5 days**	**Mon 23/11/27**	**Fri 23/12/1**	
2		Determine the programming language and database	3 days	Mon 23/11/27	Wed 23/11/29	
3		Make a general project framework	2 days	Thu 23/11/30	Fri 23/12/1	
4		**Developing**	**95 days?**	**Sat 23/12/2**	**Tue 24/3/5**	WPS,Pycharm,MyS
5		**Analyse**	**15 days?**	**Sat 23/12/2**	**Sat 23/12/16**	
6		Make questionnaire and collect results	5.25 days?	Sat 23/12/2	Fri 23/12/8	questionnaire
7		**Project Proposal**	0 days	Mon 23/12/4	Mon 23/12/4	WPS
8		Analyze functional and non-functional requirements	5.38 days?	Sat 23/12/9	Sat 23/12/16	WPS
9		**Design**	**29 days?**	**Sun 23/12/17**	**Sun 24/1/14**	Pycharm,MySQL w
10		Database structure design	6.38 days?	Sun 23/12/17	Sat 23/12/23	MySQL workbench
11		**Planning Part A,B**	0 days	Sat 23/12/23	Sat 23/12/23	
12		Login page for all characters design	2 days?	Tue 23/12/26	Wed 23/12/27	Pycharm
13		Admin page design	2.25 days?	Fri 23/12/29	Sun 23/12/31	Pycharm
14		User page design	2 days?	Mon 24/1/1	Tue 24/1/2	Pycharm
15		Mentor, faculty and college page design	3.13 days?	Wed 24/1/3	Sat 24/1/6	Pycharm
16		Basic logic and connections between routes design	5.38 days?	Tue 24/1/9	Sun 24/1/14	Pycharm
17		**Programming**	**42 days?**	**Mon 24/1/15**	**Sun 24/2/25**	Pycharm,MySQL w
18		Create databases for whole project	6.25 days?	Mon 24/1/15	Sun 24/1/21	MySQL workbench
19		**GU2 System design**	0 days	Sun 24/1/21	Sun 24/1/21	
20		Programming login pages	2 days?	Tue 24/1/23	Wed 24/1/24	Pycharm,flask
21		Programming admin page and relate routes	6.25 days?	Fri 24/1/26	Thu 24/2/1	Pycharm,flask
22		Programming user page and relate routes	10.5 days?	Mon 24/2/5	Thu 24/2/15	Pycharm,flask
23		Programming mentor, faculty and college page and relate routes	6 days	Sun 24/2/18	Fri 24/2/23	Pycharm,flask
24		Programming css files for all html files	0.25 days?	Sun 24/2/25	Sun 24/2/25	Pycharm
25		**Testing**	**7 days?**	**Wed 24/2/28**	**Tue 24/3/5**	Pycharm,flask
26		Testing program	1 day?	Wed 24/2/28	Wed 24/2/28	Pycharm,flask
27		Debugging	6 days?	Thu 24/2/29	Tue 24/3/5	Pycharm,flask
28		**Evaluating**	**30 days**	**Tue 24/3/12**	**Wed 24/4/10**	
29		**GU2 Development Coding**	0 days	Tue 24/3/12	Tue 24/3/12	WPS

1.2.2. Milestones and Deliverables

Deliverables	**Time**
Project Proposal	2023/12/4
Planning Part A, B	2023/12/23
GU2 System design	2024/1/21
GU2 Development Coding	2024/3/12

1.2.3. Main Task

Task	**Time**
Planning Stage	2023/11/27 ~ 2023/12/1
Developing	2023/12/2 ~ 2024/3/5
Analyse	2023/12/2 ~ 2023/12/16
Design	2023/12/17 ~ 2024/1/14
Programming	2024/1/15 ~ 2024/2/25
Testing	2024/2/26 ~ 2024/3/5
Evaluating	2024/3/6 ~ 2024/4/10

1.2.4. Resources

WPS: writing proposal and reports
Questionnaire: make questionnaire and collect results
Pycharm: coding, including html files, and css files
Flask: writing routes
MySQL workbench: design, manage the structure and data of databases

2. Models

2.1. Identification of classes, methods and attribute

```
class Admin(db.Model):
    idadmin = db.Column(db.Integer, primary_key=True, autoincrement=True)
    admin_name = db.Column(db.String(45), unique=True, nullable=False)
    admin_password = db.Column(db.String(45), nullable=False)

class Majors(db.Model):
    idmajor = db.Column(db.Integer, primary_key=True, autoincrement=True)
    major = db.Column(db.String(45), primary_key=True)

class ApplyTable(db.Model):
    idapply = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(45), db.ForeignKey('user.username'),
nullable=False)
    name = db.Column(db.String(45), nullable=False)
    gender = db.Column(db.String(45), nullable=False)
    birth = db.Column(db.Date, nullable=False)
    IDnum = db.Column(db.String(12), nullable=False)
    studentNum = db.Column(db.String(45), nullable=False)
    faculty = db.Column(db.String(45), nullable=False)
    politicstatus = db.Column(db.String(45), nullable=False)
    nation = db.Column(db.String(45), nullable=False)
    timeofenrollment = db.Column(db.Date, nullable=False)
    scoreRanking = db.Column(db.Integer, nullable=False)
    NumberofCompulsoryCourses = db.Column(db.String(45), nullable=False)
    PassedNum = db.Column(db.String(45), nullable=False)
    AwardName = db.Column(db.String(45), nullable=False)
    ApplyDate = db.Column(db.Date, nullable=False)
    ApplyReason = db.Column(db.Text, nullable=False)

class Mentor(db.Model):
    idmentor = db.Column(db.Integer, primary_key=True)
    mentorname = db.Column(db.String(45))
    mpassword = db.Column(db.String(45))
    major = db.Column(db.String(45), db.ForeignKey('majors.major'))

class Faculty(db.Model):
    idfaculty = db.Column(db.Integer, primary_key=True, autoincrement=True)
    facultyname = db.Column(db.String(45))
    facultypassword = db.Column(db.String(45))

class College(db.Model):
    idcollege = db.Column(db.Integer, primary_key=True, autoincrement=True)
```

collegename = db.Column(db.String(45))
collegepassword = db.Column(db.String(45))

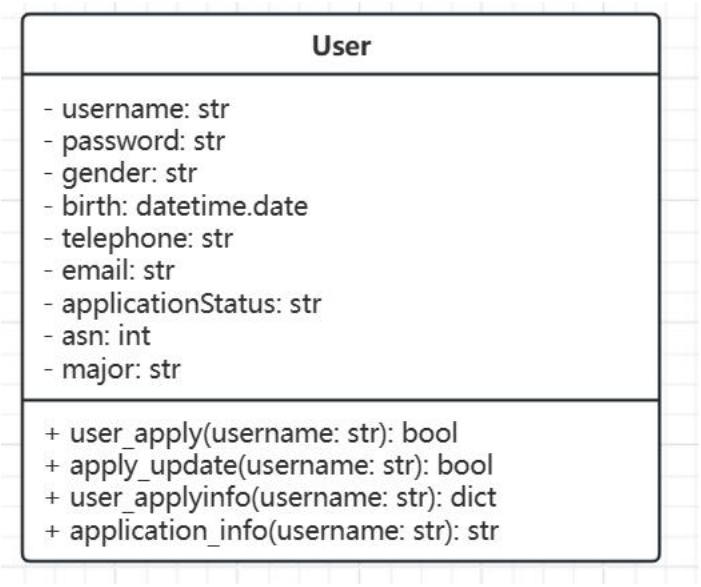
User Class:
Attributes:
username: strThe unique identifier for the user.
password: strThe password for the user's account.
gender: strThe gender of the user.
birth: datetime.dateThe date of birth of the user.
telephone: strThe telephone number of the user.
email: strThe email address of the user.
applicationStatus: strThe status of the user's scholarship application.
asn: intThe application status number indicating the stage of the application process.
major: strThe major of the user.

Methods (Routes):
user_apply(username): boolSubmits a scholarship application with the provided information.
Returns whether the application was successfully submitted.

apply_update(username): boolUpdates the user's scholarship application with new information.
Returns whether the application was successfully updated.

user_applyinfo(username): dictReturns a dictionary containing the user's scholarship application information.

application_info(username): strReturns the current status of the user's scholarship application.

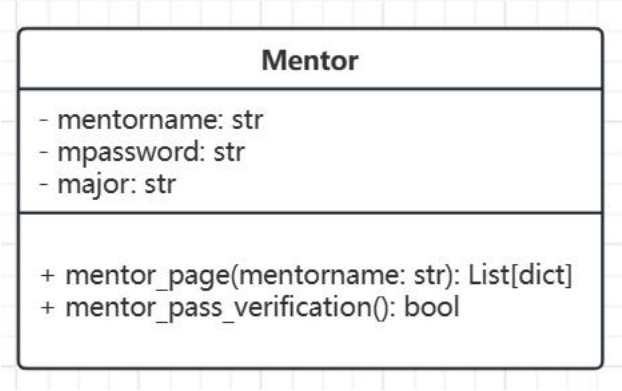


Use Case Name	User
Source	Personal information inquiry
Actor	User Database
Description	Allows a user to fill out the scholarship application form with their personal information.
Normal Course	1. User initiates the scholarship application process. 2. User fills out the scholarship application form.
Pre-Condition	User must be logged in.
Post-Condition	Scholarship application form is successfully filled out.

Mentor Class:
Attributes:
mentorname: strThe unique identifier for the mentor.
mpassword: strThe password for the mentor's account.
major: strThe major the mentor is associated with.

Methods (Routes):
mentor_page(mentorname): List[dict]Returns a list of dictionaries containing information about scholarship applications from students in the mentor's major.

mentor_pass_verification(): boolVerifies a scholarship application for a student by updating their application status.
Returns whether the application verification was successful.



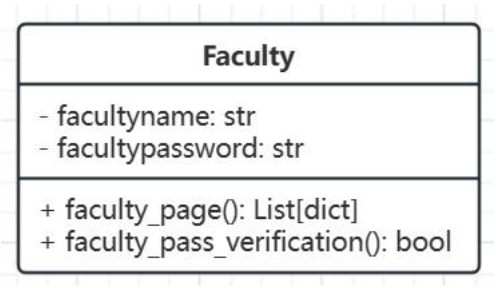
Use Case Name	Mentor
Source	Review the student's scholarship application form and submit it to the department
Actor	User Database
Description	Allows a mentor to review the scholarship application submitted by students in their major and submit it to the department.
Normal Course	1. Mentor accesses the mentor page. 2. Mentor reviews scholarship applications from students in their major.

	3. Mentor submits the applications to the department.
Pre-Condition	Mentor must be logged in.
Post-Condition	Applications are successfully submitted to the department.

Faculty Class:
Attributes:
facultyname: strThe unique identifier for the faculty member.
facultypassword: strThe password for the faculty member's account.

Methods (Routes):
faculty_page(): List[dict]Returns a list of dictionaries containing information about scholarship applications from students.

faculty_pass_verification(): boolVerifies a scholarship application for a student by updating their application status.
Returns whether the application verification was successful.



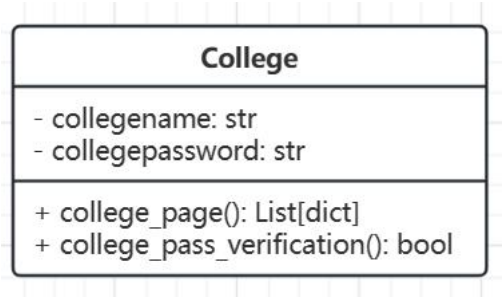
Use Case Name	Faculty
Source	Collect the preliminarily reviewed application materials collected by all tutors in the department
Actor	Faculty Database
Description	Allows a faculty member to conduct a secondary review of scholarship applications and screen students who do not meet the requirements.
Normal Course	1. Faculty member accesses the faculty page. 2. Faculty member conducts a secondary review of scholarship applications. 3. Faculty member screens students who do not meet the requirements.
Pre-Condition	Faculty must be logged in.
Post-Condition	Secondary review is successfully conducted, and students are screened accordingly.

College Class:

Attributes:
collegename: strThe unique identifier for the college.
collegepassword: strThe password for the college's account.

Methods (Routes):
college_page(): List[dict]Returns a list of dictionaries containing information about scholarship applications from students.

college_pass_verification(): boolVerifies a scholarship application for a student by updating their application status.
Returns whether the application verification was successful.



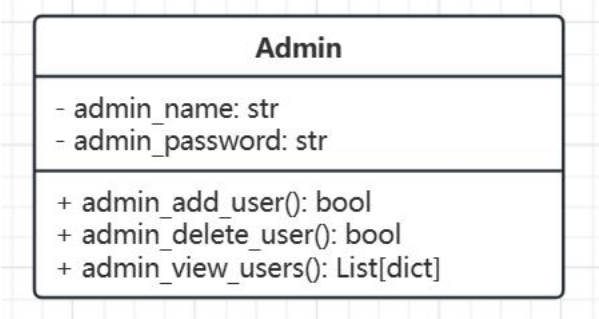
Use Case Name	College
Source	Make the final decision
Actor	Faculty Database
Description	Allows a College member to conduct a secondary review of scholarship applications and screen students who do not meet the requirements.
Normal Course	1. College member accesses the faculty page. 2. College member conducts a secondary review of scholarship applications. 3. College member screens students who do not meet the requirements.
Pre-Condition	College must be logged in.
Post-Condition	Final decisions are made on scholarship applications.

Admin Class:
Attributes:
admin_name: strThe unique identifier for the admin.
admin_password: strThe password for the admin's account.

Methods (Routes):
admin_add_user(): boolAdds a new user to the system with the provided information.
Returns whether the user was successfully added.

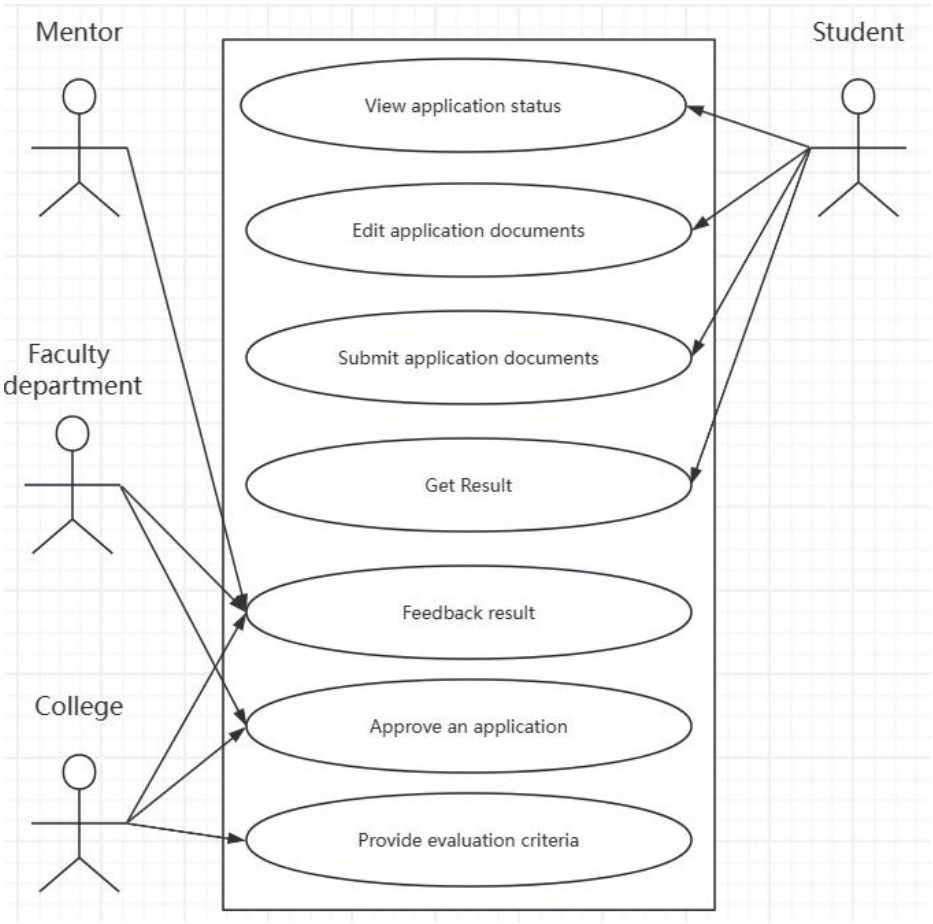
admin_delete_user(): boolDeletes a user from the system based on the username.
Returns whether the user was successfully deleted.

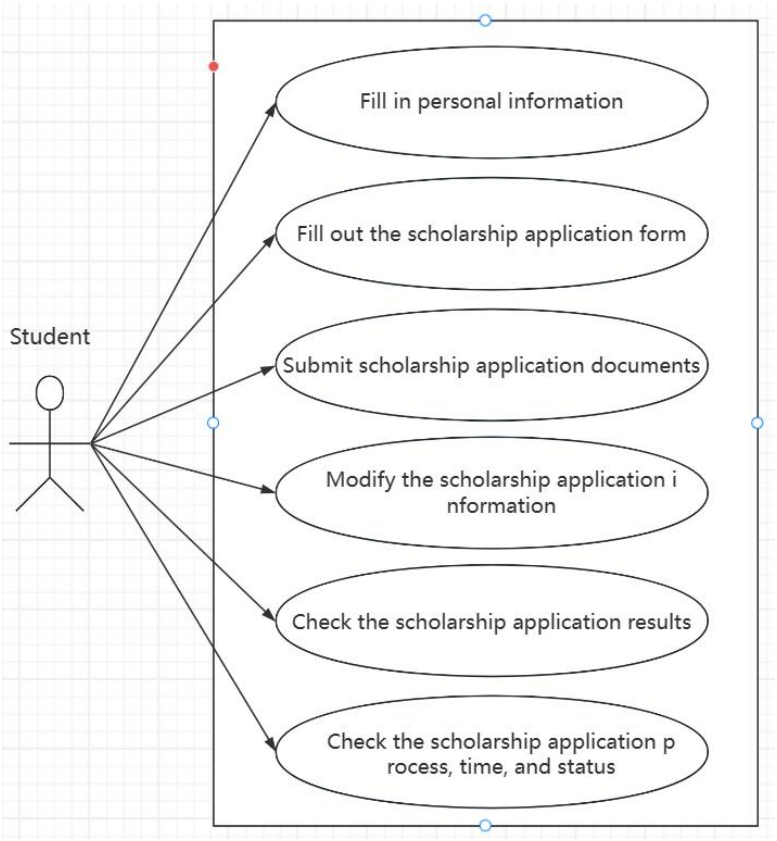
admin_view_users(): List[dict]Returns a list of dictionaries containing information about all users in the system.

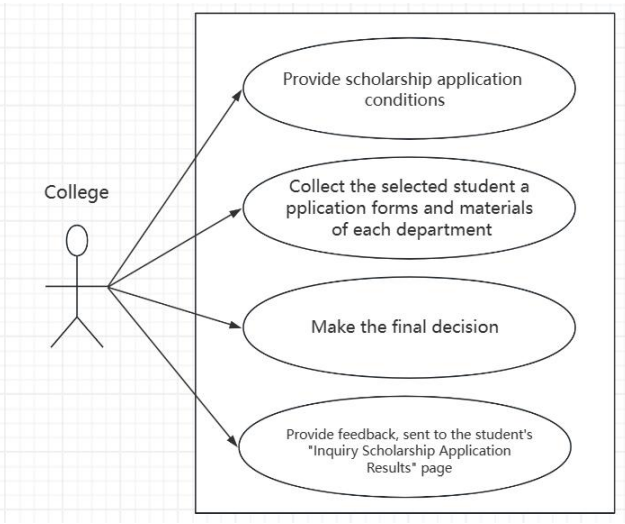


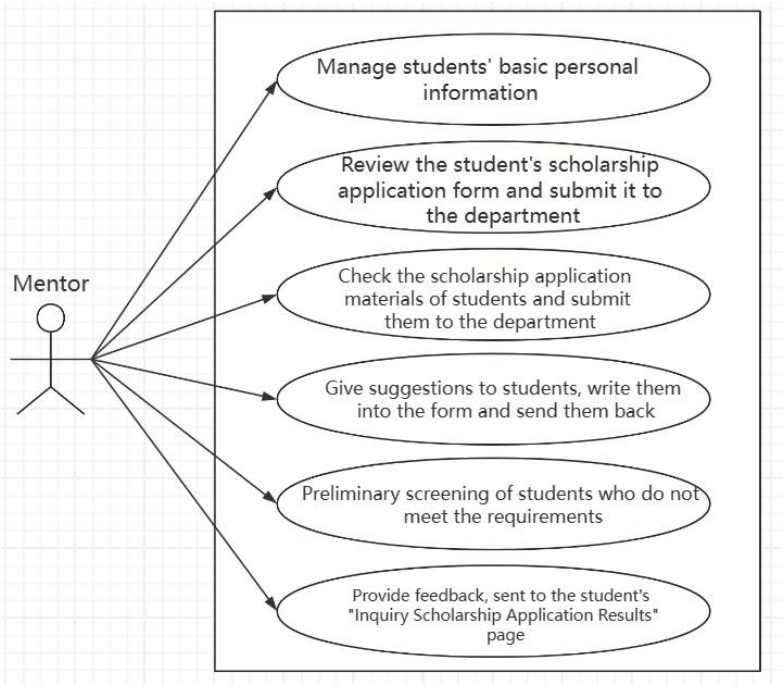
Use Case Name	Admin
Source	Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page
Actor	Admin Database
Description	Allows an admin to provide feedback to applicants regarding the status of their scholarship application.
Normal Course	1. Admin accesses the admin interface. 2. Admin provides feedback to applicants regarding the status of their application.
Pre-Condition	Admin must be logged in.
Post-Condition	Feedback is successfully provided to applicants.

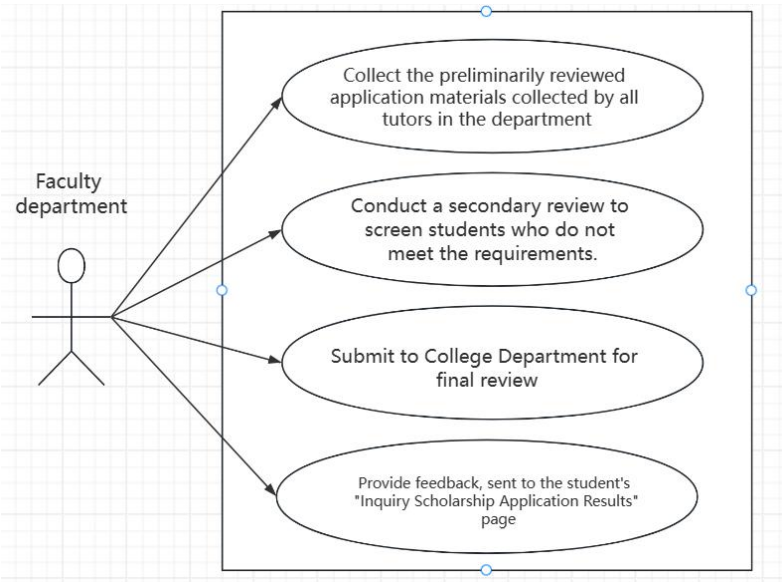
2.2. Top level Use Case Model



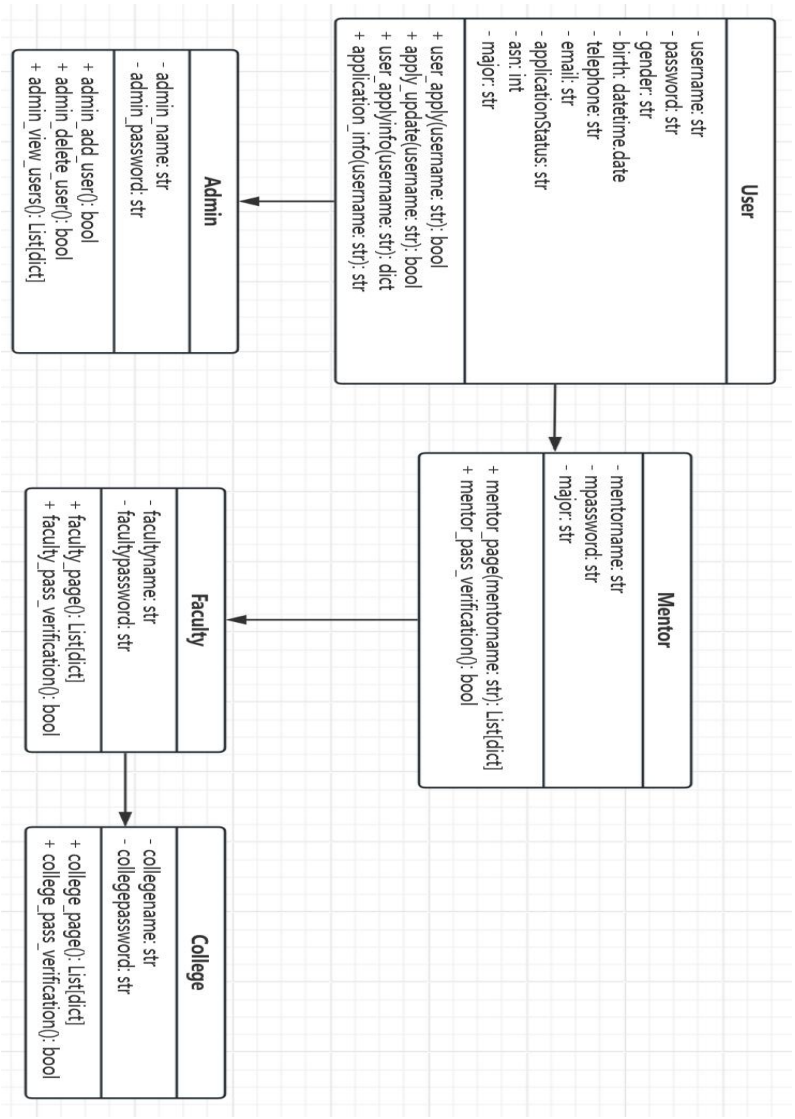

Student
Fill in personal information
Fill out the scholarship application form
Submit scholarship application documents
Modify the scholarship application i nformation
Check the scholarship application results
Check the scholarship application p rocess, time, and status


College
Provide scholarship application conditions
Collect the selected student a pplication forms and materials of each department
Make the final decision
Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page


Mentor
Manage students' basic personal information
Review the student's scholarship application form and submit it to the department
Check the scholarship application materials of students and submit them to the department
Give suggestions to students, write them into the form and send them back
Preliminary screening of students who do not meet the requirements
Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page

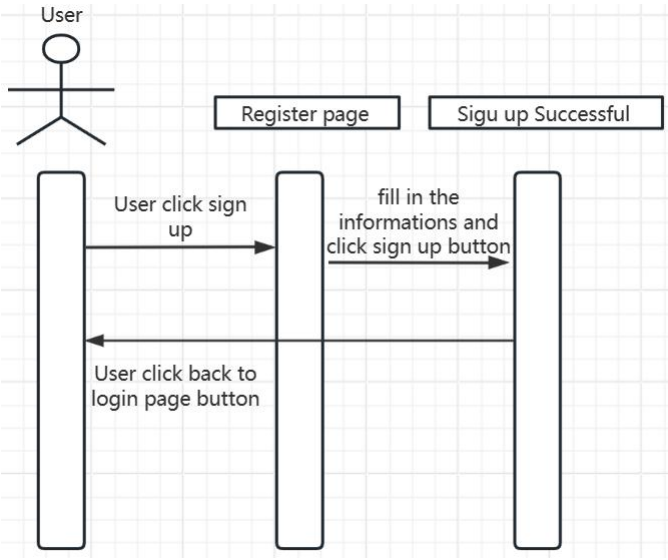

Faculty department
Collect the preliminarily reviewed application materials collected by all tutors in the department
Conduct a secondary review to screen students who do not meet the requirements.
Submit to College Department for final review
Provide feedback, sent to the student's "Inquiry Scholarship Application Results" page

2.3. UML

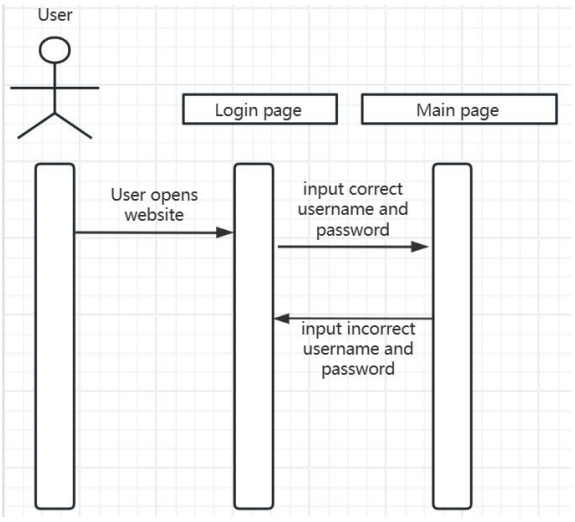


2.4. Sequence Diagram

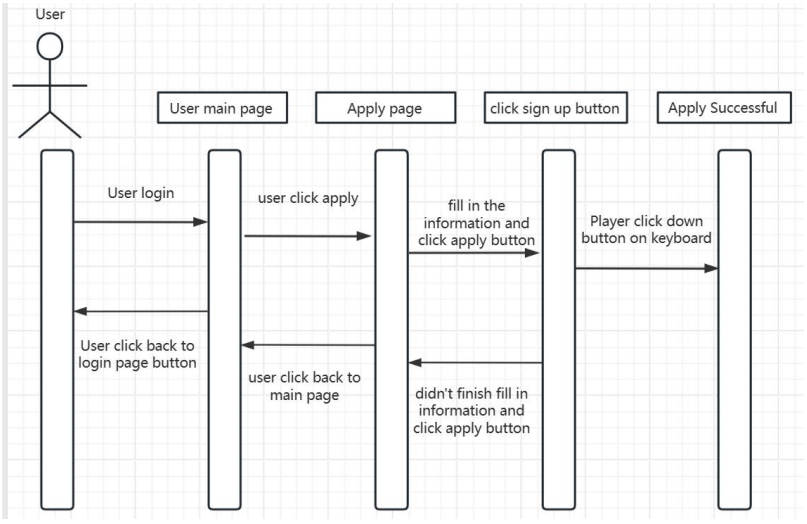
2.4.1. User register



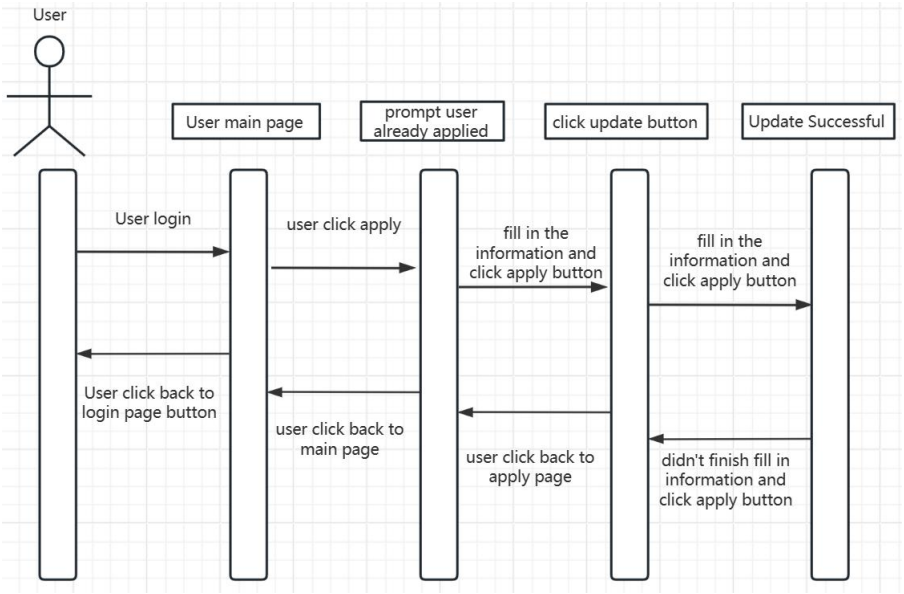
2.4.2. Login



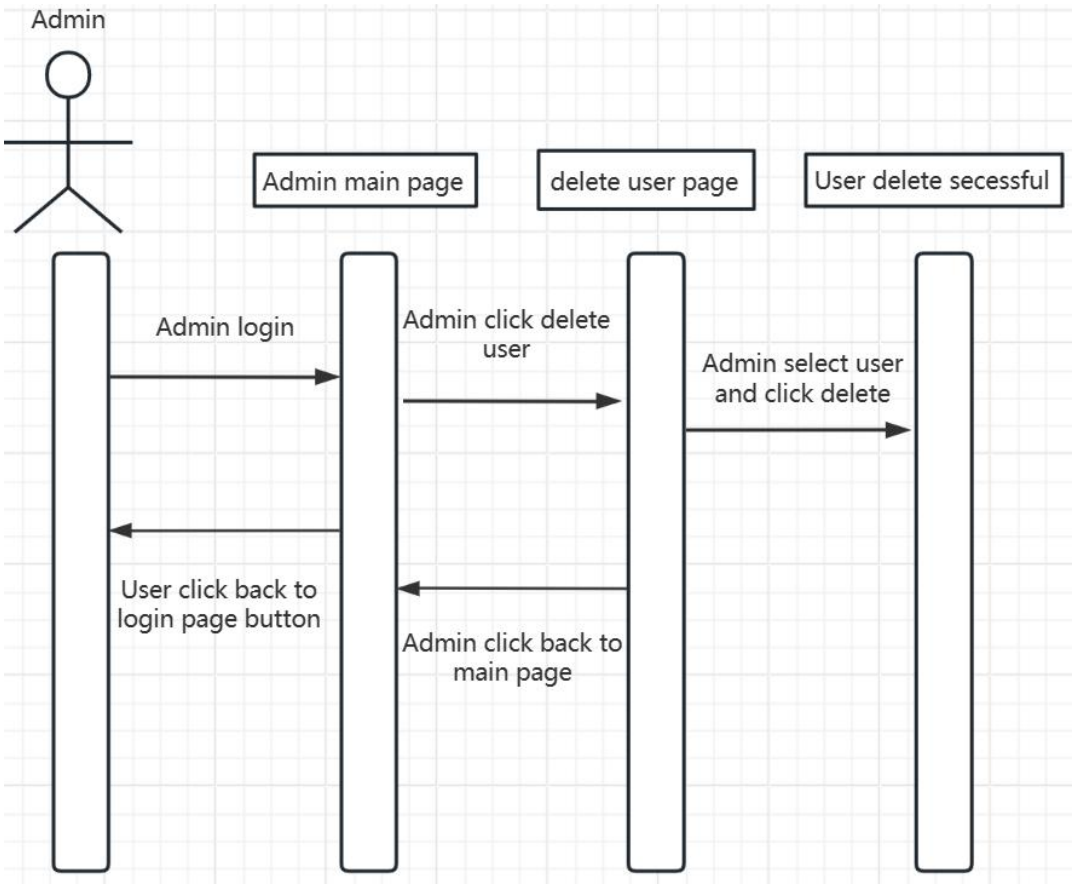
2.4.3. User apply



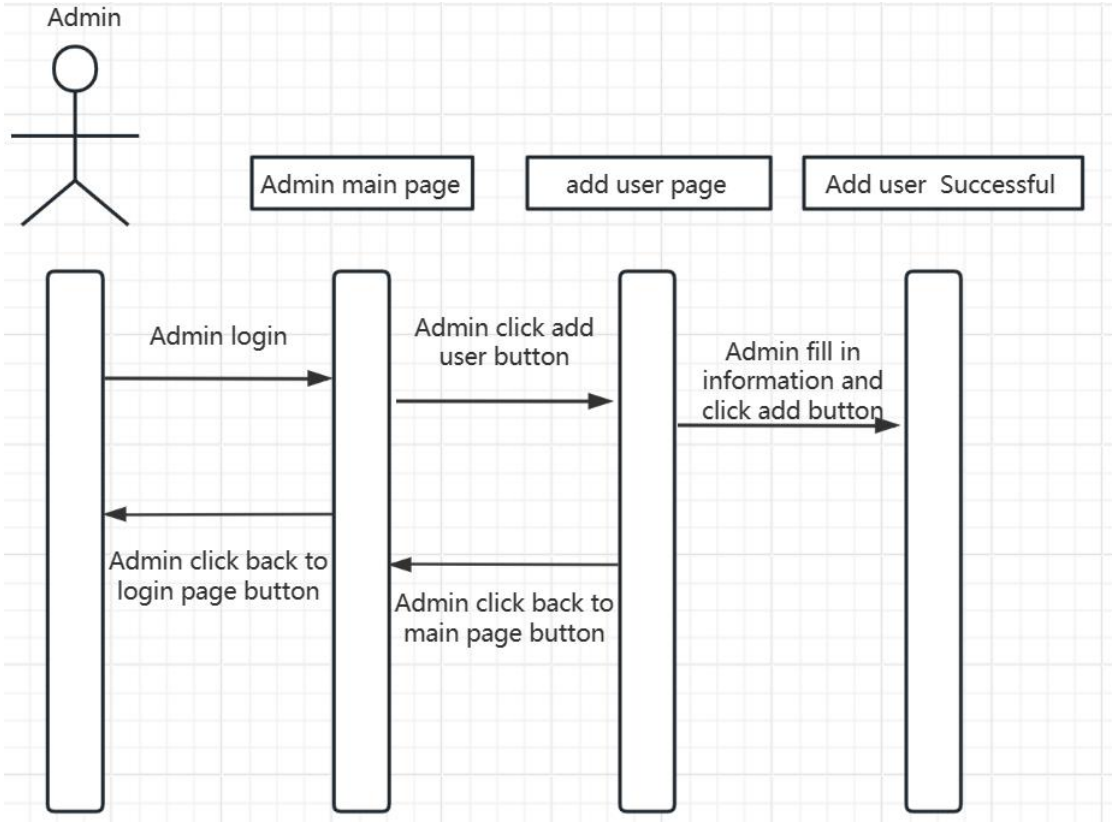
2.4.4. User update application information



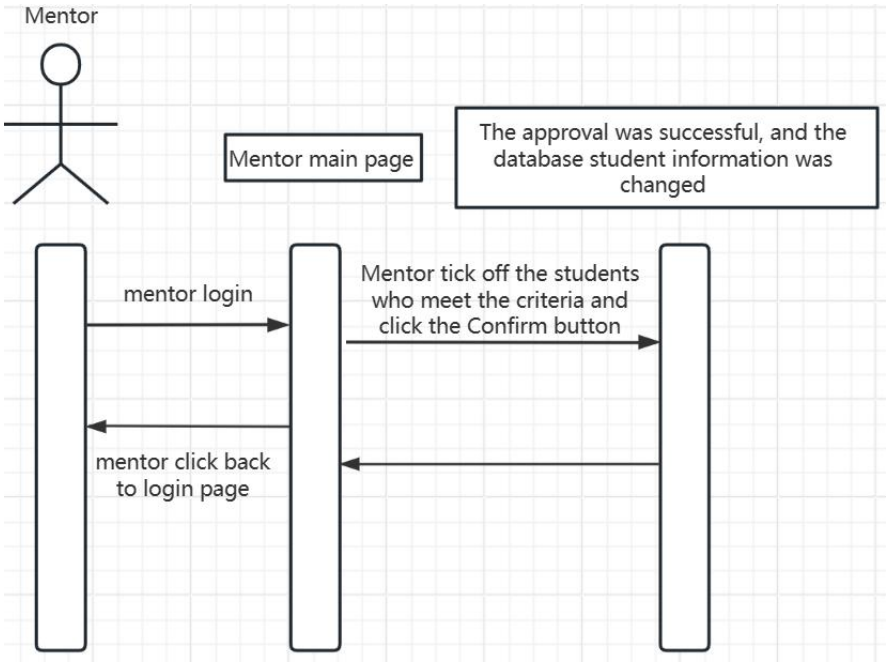
2.4.5. Admin delete user



2.4.6. Admin add user

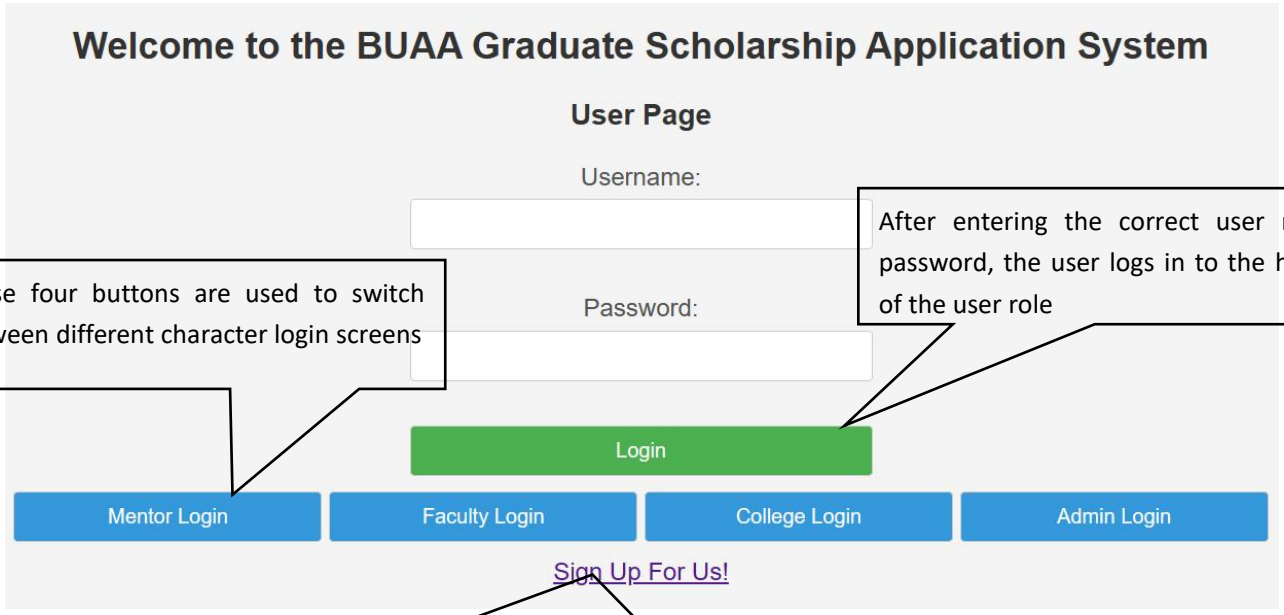


2.4.7. Approve applications



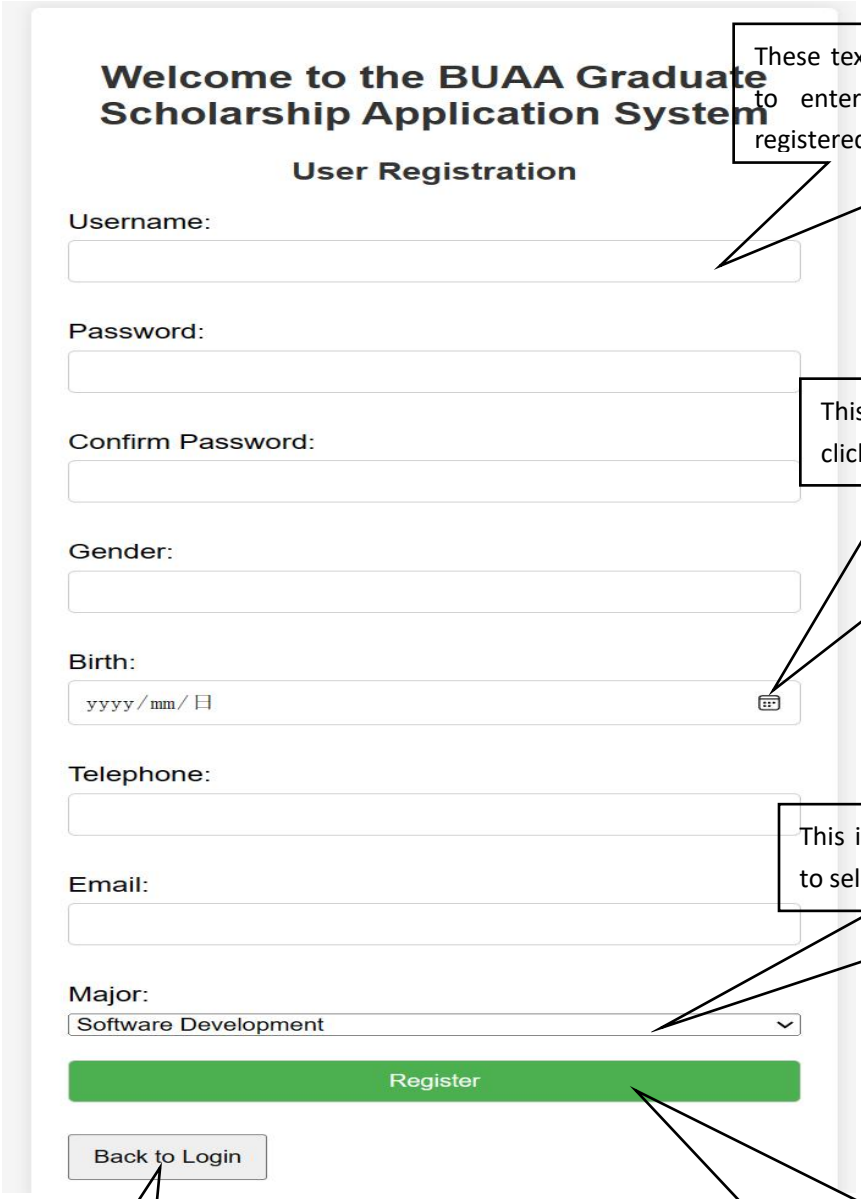
2.5. GUI design and HCI

Login Page



This is a login page for five roles

User Register page



These text boxes are designed to allow users to enter basic information that can be registered into the system

This is a button to select a date, which can be clicked and then selected with the calendar.

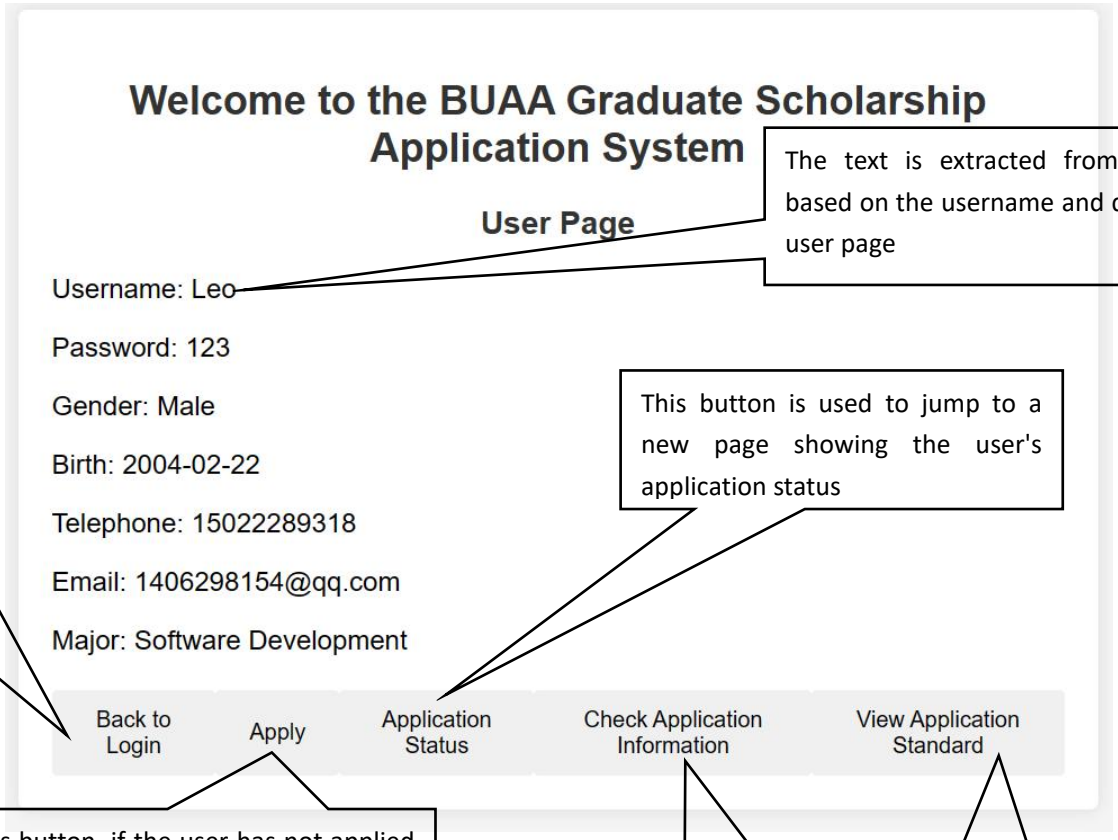
This is a drop down box that allows the user to select his major.

After clicking this button, if the input information is correct and filled in, you can register successfully and jump to the user login interface

The user will be taken back to the login screen after clicking this button

This page is for wechat users to register, requiring the above information to be filled in

User Page



The text is extracted from the database based on the username and displayed to the user page

This button is used to jump to a new page showing the user's application status

The user will be taken back to the login screen after clicking this button

After clicking this button, if the user has not applied for the scholarship, it will jump to the application interface; If the user has applied and there is no result, the user will be prompted whether to update the information; If the user has been rejected or successfully applied for a scholarship, the user will be notified that they cannot apply again

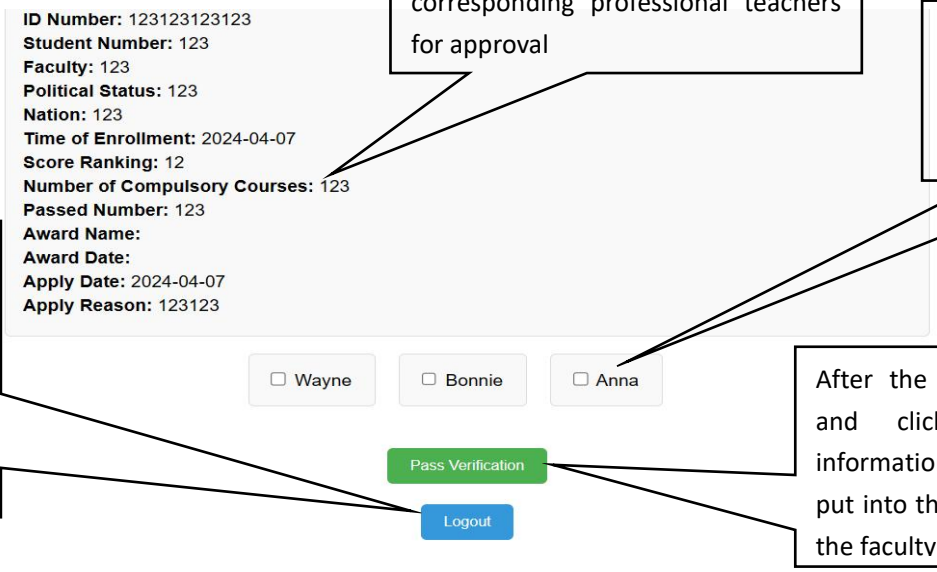
This button will take the user to another screen and display the submitted application information.

This button will take the user to another screen and display the criteria for applying for the scholarship

Mentor Page

These information are retrieved from the database and assigned to the corresponding professional teachers for approval

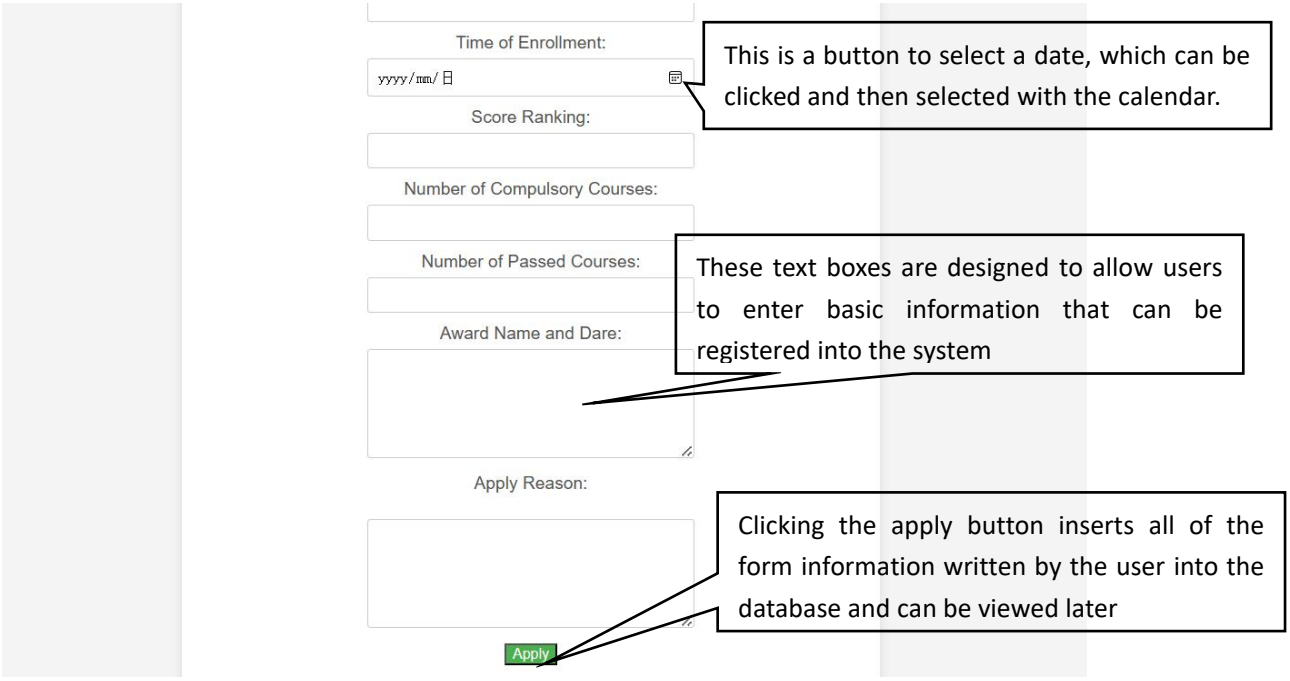
These boxes allow mentor to select which students meet the criteria and can pass the initial approval



The user will be taken back to the login screen after clicking this button

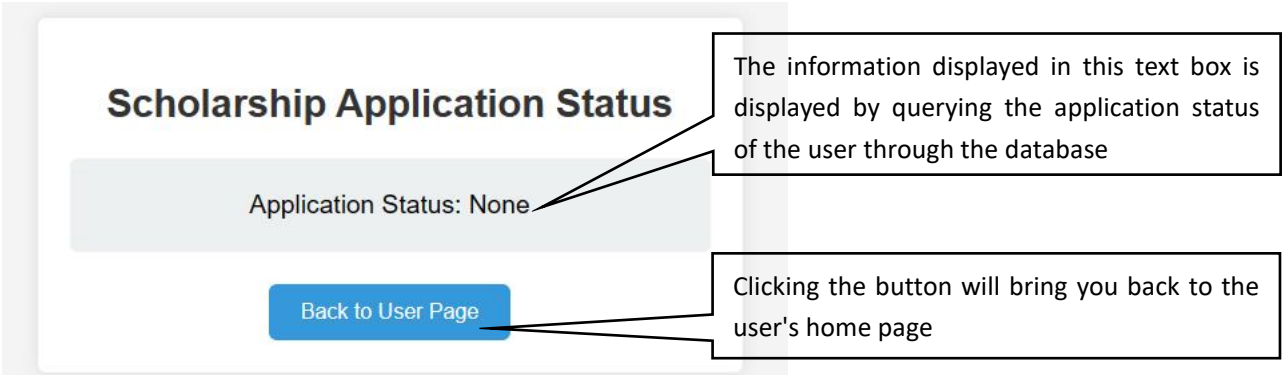
After the mentor selects students and clicks this button, the information of these students will be put into the database and passed to the faculty for second review.

User Apply page



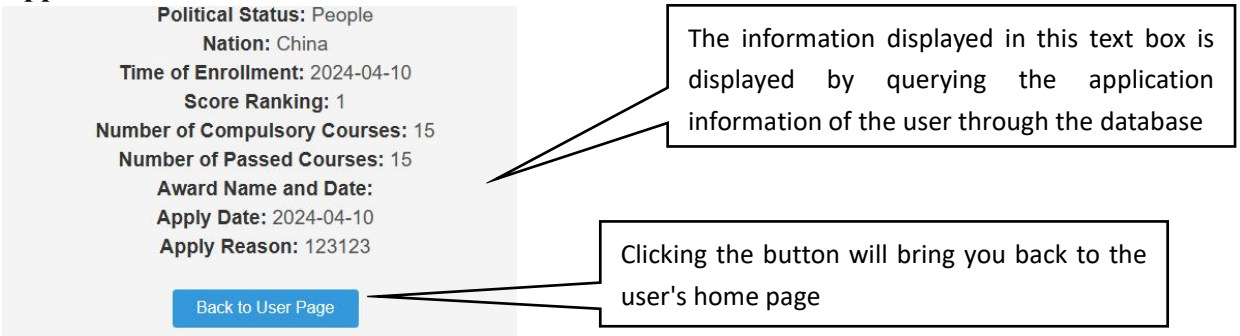
This page is for users to request ice Pride form information

Application status page



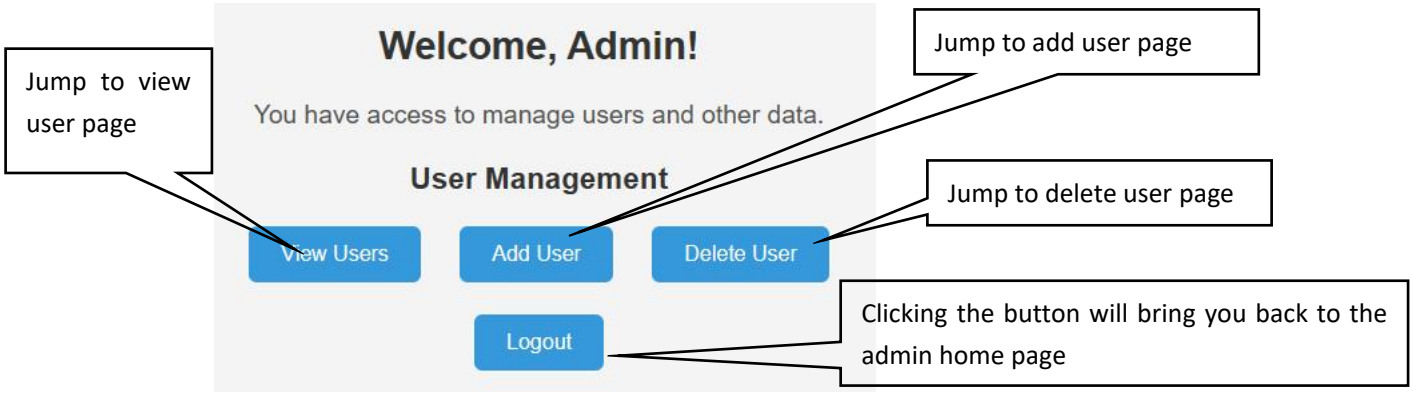
This interface is through the database command query to obtain the user's application status and display

Application Information



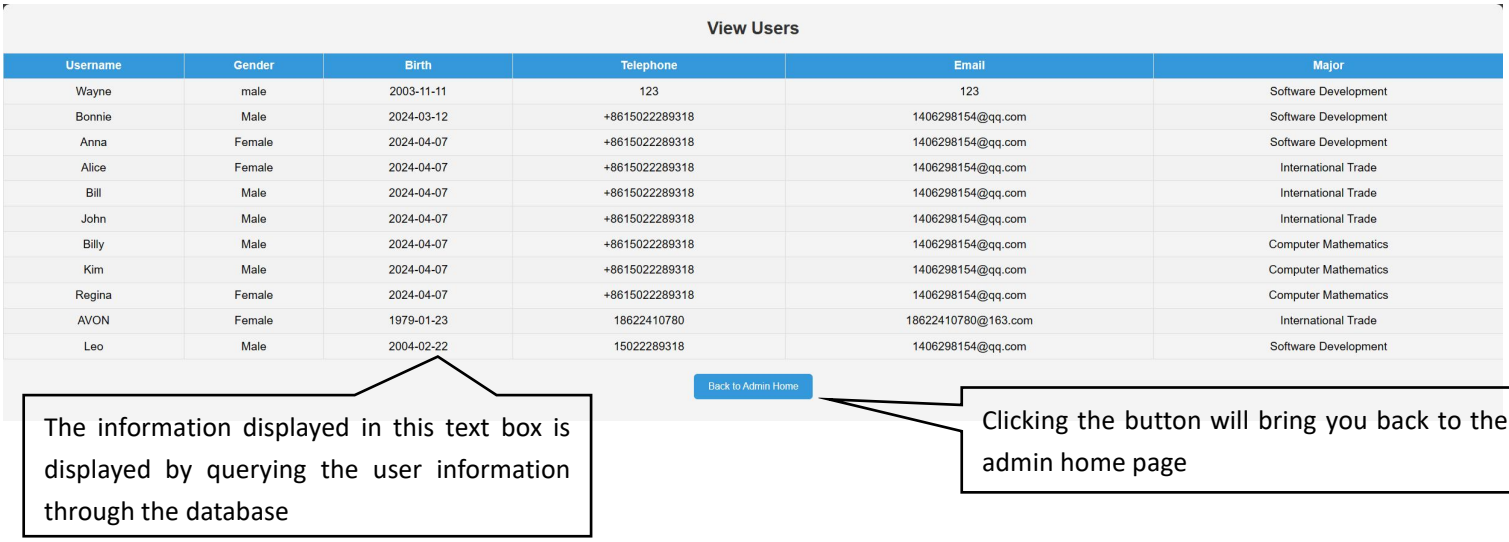
This interface is through the database command query to obtain the user's application information and display

Admin Page



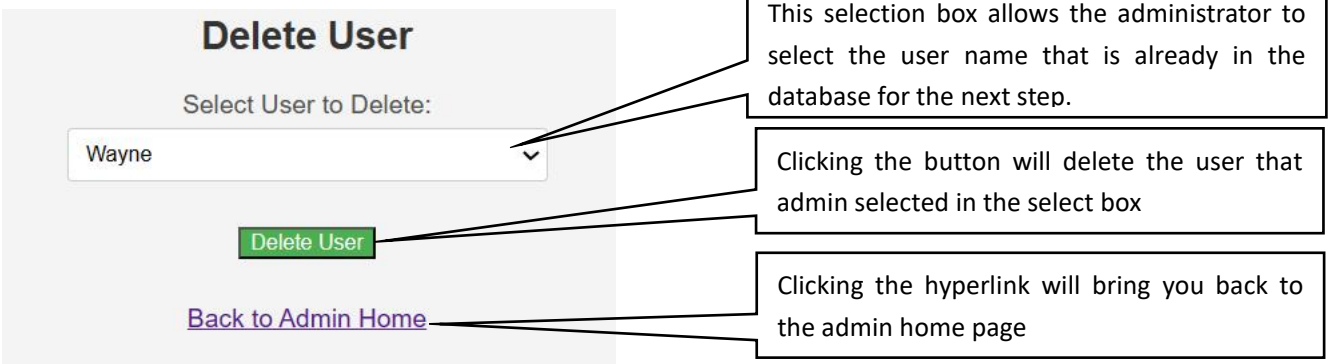
This is an administrator's main page, which allows the administrator to proceed to the next step.

View user page

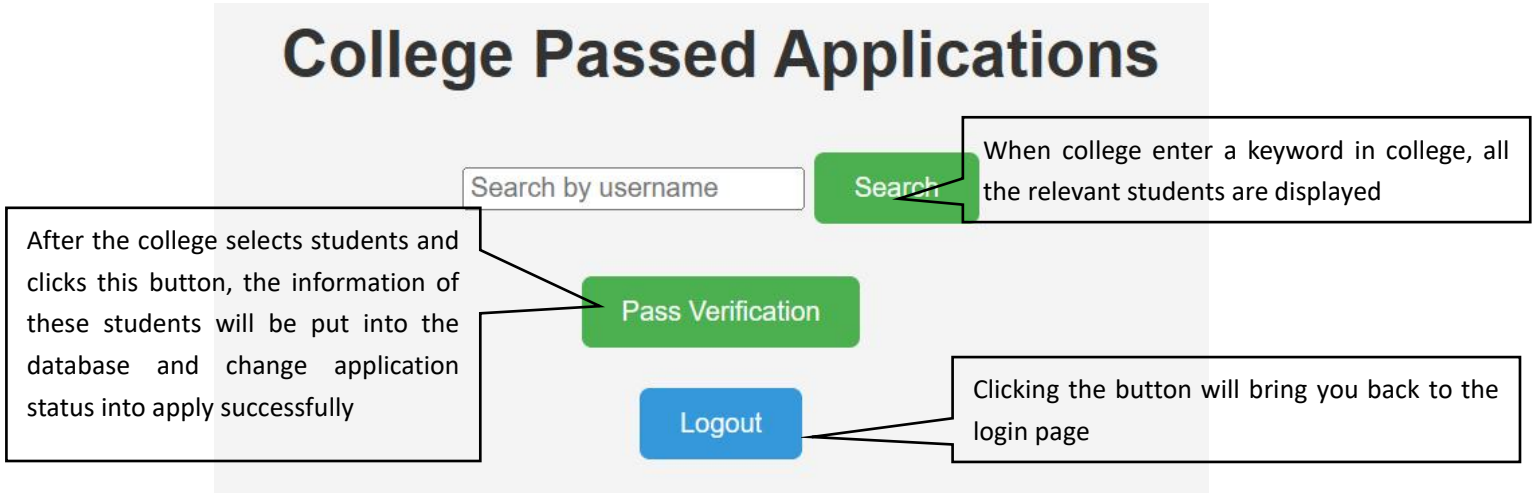


This page displays all user information through the database query command.

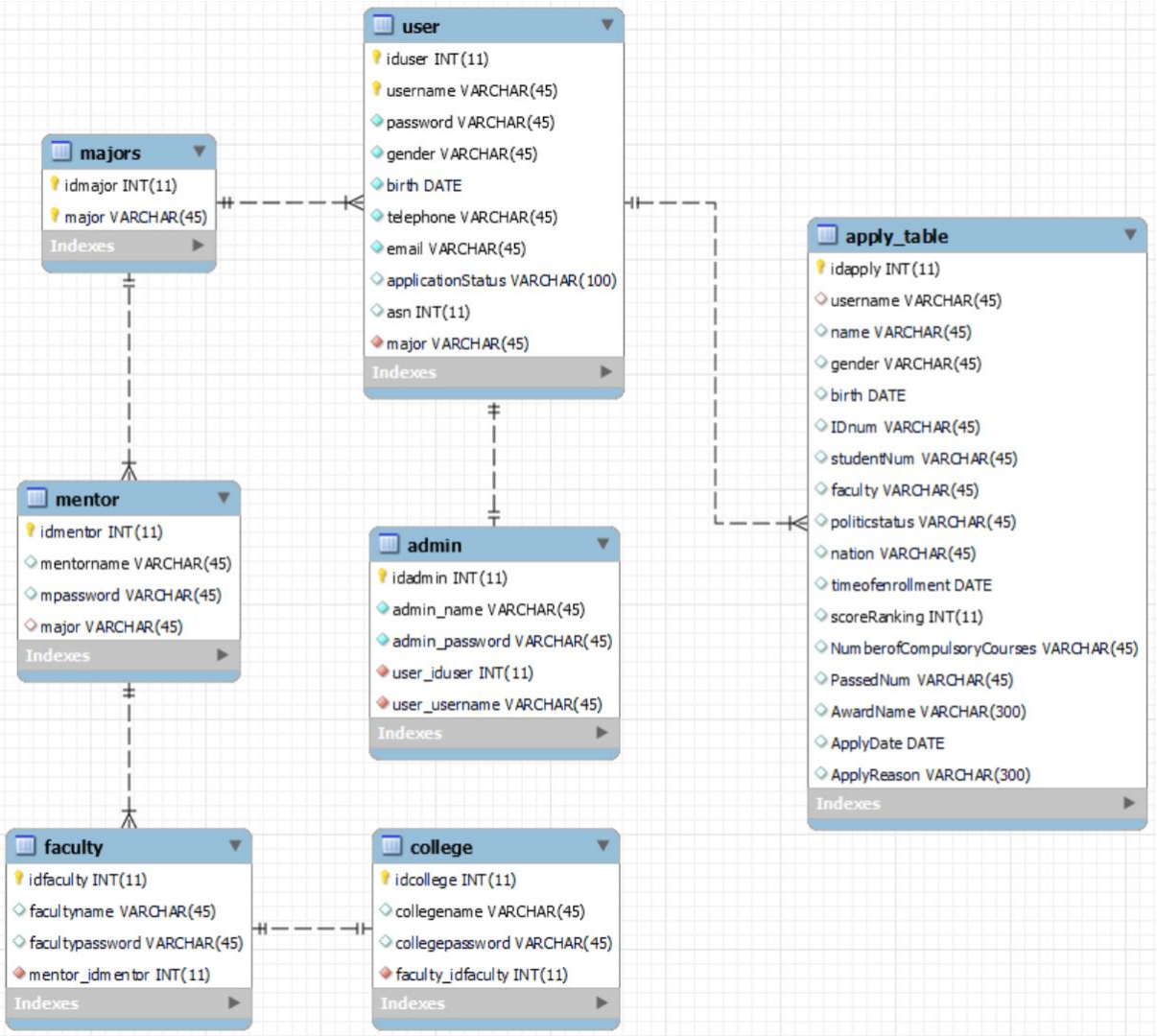
Delete user page



This interface allows the administrator to select the user and delete the user through the database command.



2.6. E-R Diagram



2.7. Data Dictionary

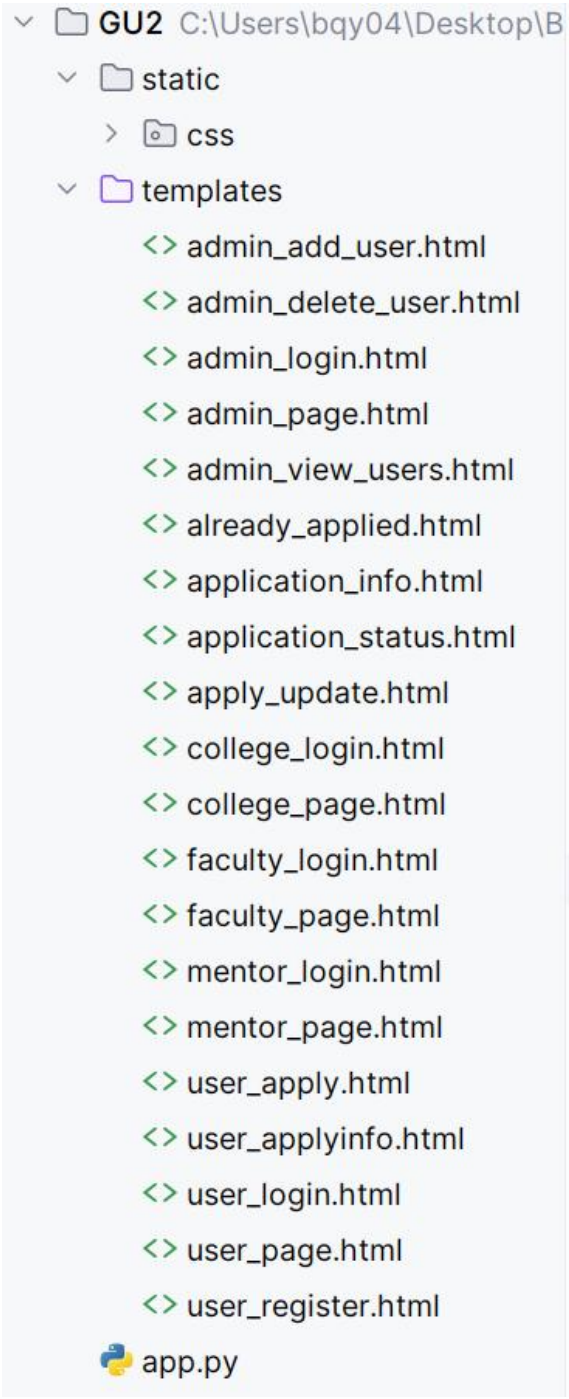
Name	Type	Description
app	Flask	The Flask application instance

db	SQLAlchemy	The SQLAlchemy database instance
User	SQLAlchemy Model	Model representing users in the database
Admin	SQLAlchemy Model	Model representing administrators in the database
Majors	SQLAlchemy Model	Model representing majors in the database
ApplyTable	SQLAlchemy Model	Model representing application details in the database
Mentor	SQLAlchemy Model	Model representing mentors in the database
Faculty	SQLAlchemy Model	Model representing faculty members in the database
College	SQLAlchemy Model	Model representing colleges in the database
cmbMusic	JComboBox	Store the list of music
username	str	Username input from various login and registration forms
password	str	Password input from various login and registration forms
confirm_password	str	Confirm password input from user registration form
gender	str	Gender input from user registration form
birth	str	Birth date input from user registration form
telephone	str	Telephone input from user registration form
email	str	Email input from user registration form
major	str	Major input from user registration form
collegename	str	College name input from college login form
collegepassword	str	College password input from college login form
mentorname	str	Mentor name input from mentor login form
mpassword	str	Mentor password input from mentor login form
facultyname	str	Faculty name input from faculty login form
facultypassword	str	Faculty password input from faculty login form
selected_students	list of str	List of selected student usernames from pass verification forms
student_name	str	Username of individual student during pass verification
application	ApplyTable	Application details of a student during pass verification
user	User	User instance retrieved during login

college	College	College instance retrieved during college login
mentor	Mentor	Mentor instance retrieved during mentor login
faculty	Faculty	Faculty instance retrieved during faculty login
error_message	str	Error message displayed during login if credentials are invalid
mentorname	str	Mentor name retrieved from URL parameter during mentor page display
students	list of User	List of students associated with a mentor
applications	str	List of applications associated with students
selected_students_set	list of ApplyTable	Set of selected student usernames during pass verification
all_students	set of str	List of all student usernames in the database
new_user	list of str	New user instance created during user registration
message	User	Message displayed during user registration
admin_name	str	Admin username input from admin login form
admin_password	str	Admin password input from admin login form
admin	Admin	Admin instance retrieved during admin login
session	Flask session	Flask session object used for admin authentication
admin_id	int	ID of the logged-in admin stored in session
user_list	list of User	List of users retrieved for admin operations
name	str	Name input from user apply/update form
gender	str	Gender input from user apply/update form
birth	str	Birth date input from user apply/update form
IDnum	str	ID number input from user apply/update form
studentNum	str	Student number input from user apply/update form
faculty	str	Faculty input from user apply/update form
politicstatus	str	Political status input from user apply/update form
nation	str	Nationality input from user apply/update form
timeofenrollment	datetime.date	Time of enrollment input from user apply/update form

3. Coding & Test Part

3.1. Business Model Code Listing



css folder: used to store all the web style files, make the site more beautiful.

admin_add_user: The page for the administrator to add users

admin_delete_user: The page for the administrator to delete the user

admin_login: administrator login page

admin_page: Administrator's home page, which allows the administrator to select the next action

admin_view_user: The administrator views the user page

already_applied: When the user has already applied for a scholarship, ask the user what to do next

application_info: Criteria for applying for the scholarship

application_status: Scholarship application status

apply_update: A page for users to update scholarship application information

college_login: allows you to login to the college role

college_page: college role page

faculty_login: allows you to login to the faculty role

faculty_page: faculty role page

mentor_login: allows you to login to the mentor role

mentor_page: mentor role page

user_apply: Users can fill in the scholarship application information on this page.

user_applyinfo: Users can view submitted applications on this page

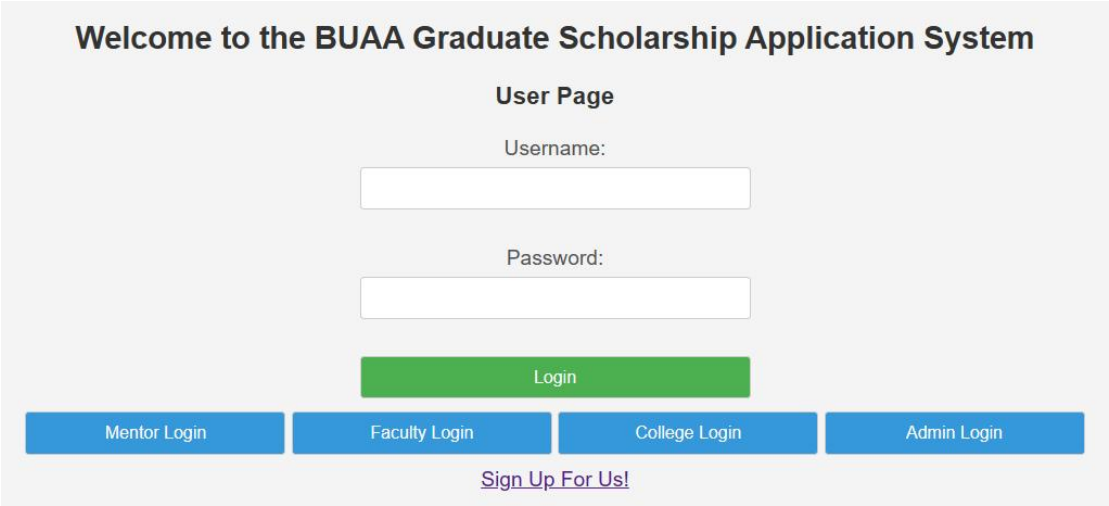
user_login: indicates the user login page

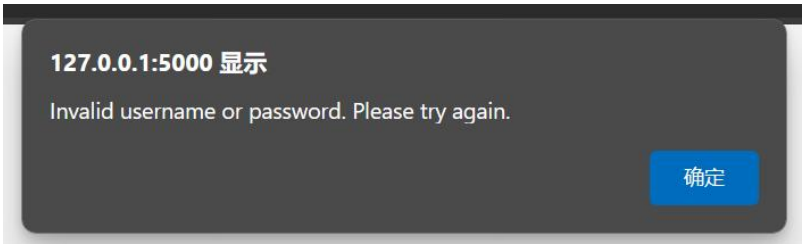
user_page: indicates the home page of a user. You can perform the next operation based on your requirements.

user_register: Allows users to create a new student account to apply for scholarships

3.2. Viewing Model Code Listing

3.2.1. Login Page






```
@app.route('/user_login', methods=['GET', 'POST'])
def user_login():
    error_message = None

    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        user = User.query.filter_by(username=username, password=password).first()

        if user:
            return redirect(url_for('user_page', username=username))
        else:
            error_message = 'Invalid username or password. Please try again.'

    return render_template('user_login.html', error_message=error_message)
```

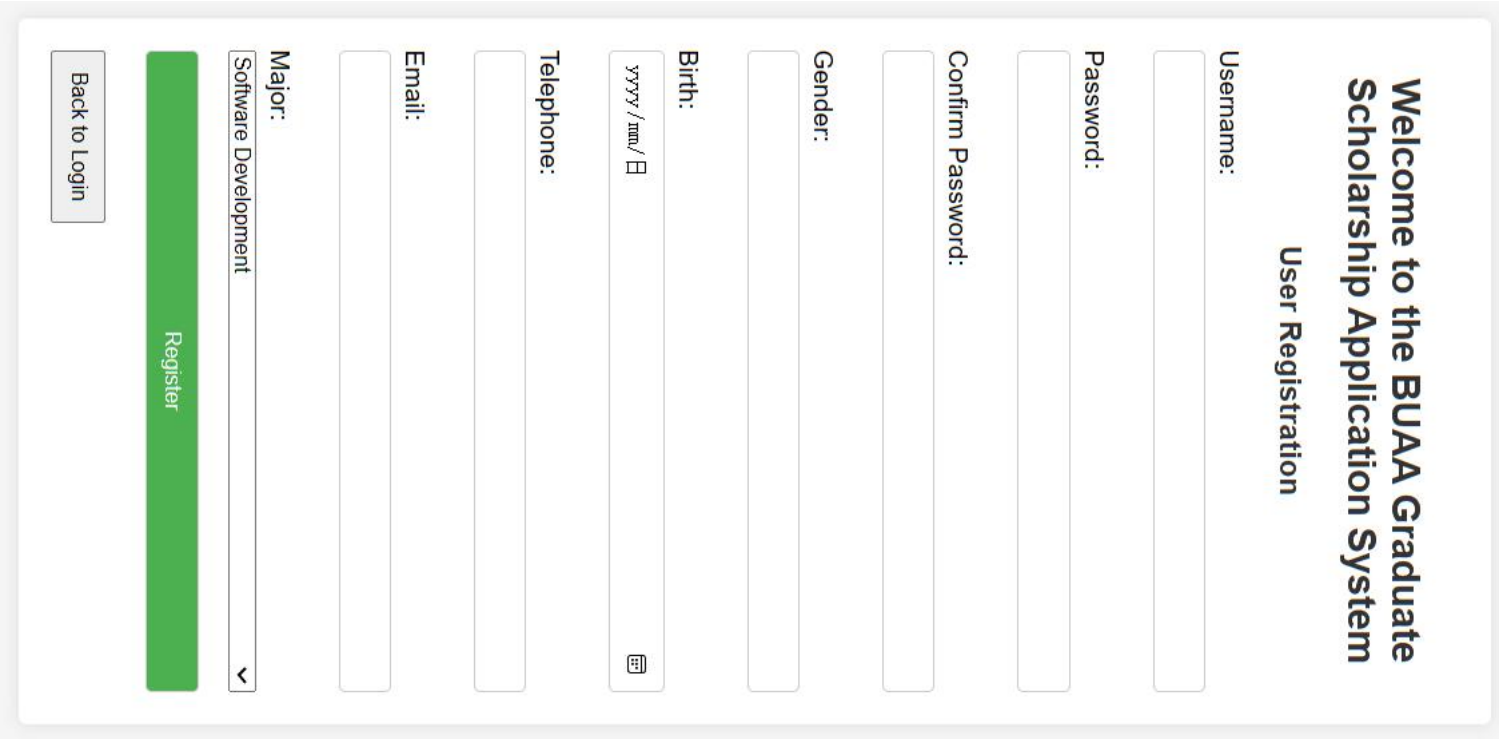
Front-end core code:

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/loginStyle.css') }}">
    <script>
        {% if error_message %}
            window.onload = function() {
                alert("{{ error_message }}");
            };
        {% endif %}
    </script>
</head>
<body>
    <h2>Welcome to the BUAA Graduate Scholarship Application System</h2>
```

The user enters a username and password into the site. When the user clicks the login button, the route receives the information entered by the user and checks whether the corresponding information is available through the database query command. If yes, login to the main page; If the user name or password is incorrect, the user is prompted to enter the correct password; If no information is filled in, the user is prompted to fill in the information first.

3.2.2. User role

User_register




```
@app.route('/user_register', methods=['GET', 'POST'])
def user_register():
    message = None
```

```
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        gender = request.form['gender']
        birth = request.form['birth']
        telephone = request.form['telephone']
        email = request.form['email']
        major = request.form['major']

        if password != confirm_password:
            message = 'Passwords do not match. Please try again.'
        else:
            if User.query.filter_by(username=username).first():
                message = 'Username already exists. Please choose a different
username.'
            else:
                new_user = User(username=username, password=password,
gender=gender, birth=birth,
                                telephone=telephone, email=email,
major=major)

                db.session.add(new_user)
                db.session.commit()

                message = 'Registration successful! You can now log in.'
    return render_template('user_register.html', message=message)
```

Front-end core code:

```
<script>
    // Function to display the message in an alert and redirect to user_login
    function displayMessage() {
        var message = "{{ message }}";
        if (message && message !== "None") {
            if (message === 'Registration successful! You can now log in.') {
                alert(message);
                window.location.href = '{{ url_for('user_login') }}';
            } else {
                alert(message);
            }
        }
    }

    // Function to validate password
```

```
        function validatePassword() {
            var password = document.getElementById("password").value;
            var confirmPassword =
document.getElementById("confirm_password").value;

            if (password !== confirmPassword) {
                alert("Passwords do not match!");
                return false;
            }
            displayMessage();
            return true;
        }
</script>
```

Users enter personal information such as a username and password into the website. When the user clicks the register button, the route receives the information entered by the user and inserts the user's data into the database through the database insert command. If the password and confirmation password do not match, the user is prompted to enter the correct password. If no information is filled in, the user is prompted to fill in the information first. At the same time, the username entered on this page is passed to the user's home page through the url, so that the user can operate according to their username

User_page




```
@app.route('/user_page/<username>', methods=['GET', 'POST'])
def user_page(username):
```

```
    user = User.query.filter_by(username=username).first()
    if user:
        user_info = {
            'username': user.username,
            'password': user.password,
            'gender': user.gender,
            'birth': user.birth,
            'telephone': user.telephone,
            'email': user.email,
            'major': user.major
        }
        return render_template('user_page.html', **user_info)
    else:
        return 'User not found.'
```

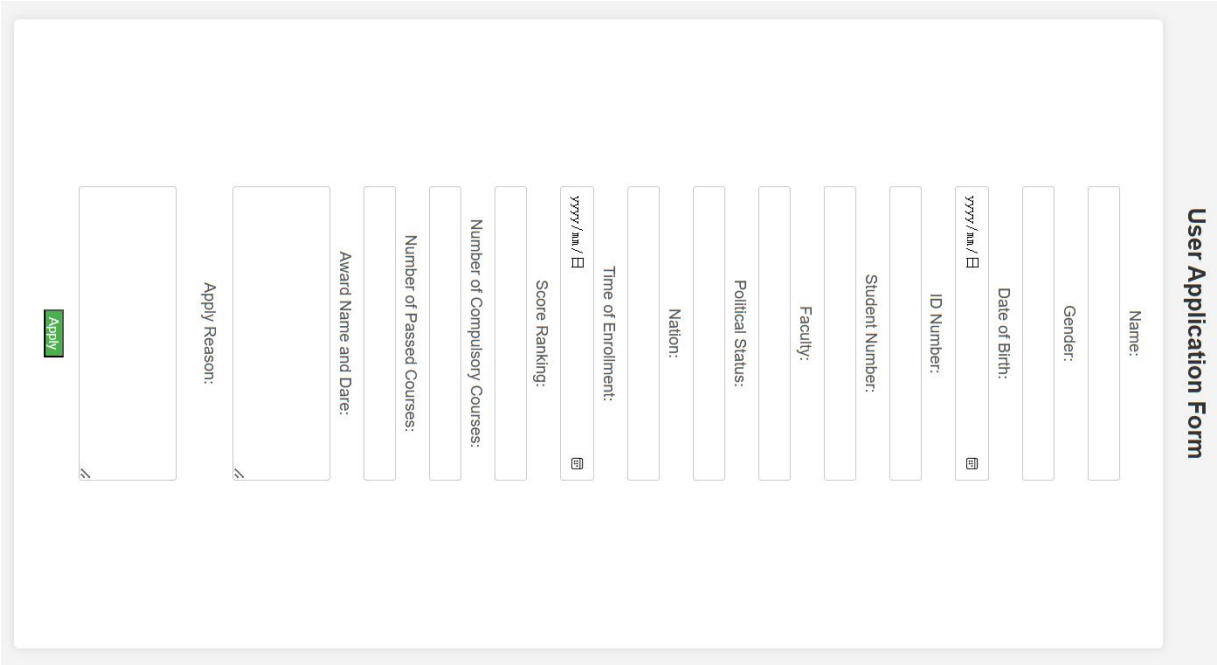
Front-end core code:

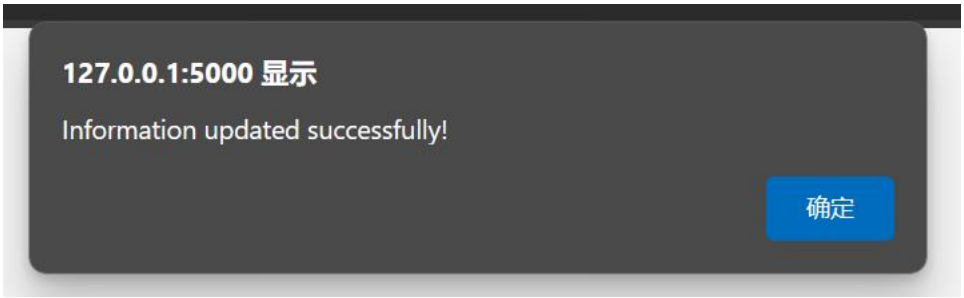
```
<div class="user-info">
    <p>Username: {{ username }}</p>
    <p>Password: {{ password }}</p>
    <p>Gender: {{ gender }}</p>
    <p>Birth: {{ birth }}</p>
    <p>Telephone: {{ telephone }}</p>
    <p>Email: {{ email }}</p>
    <p>Major: {{ major }}</p>
</div>

<div class="buttons">
    <button onclick="window.location.href='{{ url_for('user_login') }}'">Back to
Login</button>
    <button onclick="window.location.href='{{ url_for('user_apply',
username=username) }}'">Apply</button>
    <button
onclick="window.location.href='{{ url_for('application_status') }}?username={{ user
name }}'">Application Status</button>
    <button onclick="window.location.href='{{ url_for('user_applyinfo',
username=username) }}'">Check Application Information</button>
    <button onclick="window.location.href='{{ url_for('application_info',
username=username) }}'">View Application Standard</button>
</div>
```

This page is the user's main interface, and will use the user name passed through the url to query all the information in the database that matches the user name, and put it in the user's main interface to display the information. There are also multiple buttons to jump to different screens.

User_apply page






```
@app.route('/user_apply/<username>', methods=['GET', 'POST'])
def user_apply(username):
    if request.method == 'POST':
        name = request.form['name']
        gender = request.form['gender']
        birth = request.form['birth']
        IDnum = request.form['IDnum']
        studentNum = request.form['studentNum']
        faculty = request.form['faculty']
        politicstatus = request.form['politicstatus']
        nation = request.form['nation']
        timeofenrollment = request.form['timeofenrollment']
        scoreRanking = request.form['scoreRanking']
        NumberofCompulsoryCourses =
request.form['NumberofCompulsoryCourses']
        PassedNum = request.form['PassedNum']
        AwardName = request.form['AwardName']
        ApplyReason = request.form['ApplyReason']
```

```
            current_date = datetime.now().date()

            new_application = ApplyTable(
                username=username,
                name=name,
                gender=gender,
                birth=birth,
                IDnum=IDnum,
                studentNum=studentNum,
                faculty=faculty,
                politicstatus=politicstatus,
                nation=nation,
                timeofenrollment=timeofenrollment,
                scoreRanking=scoreRanking,
                NumberofCompulsoryCourses=NumberofCompulsoryCourses,
                PassedNum=PassedNum,
                AwardName=AwardName,
                ApplyDate=current_date,
                ApplyReason=ApplyReason,
            )
            db.session.add(new_application)
            db.session.commit()

            user = User.query.filter_by(username=username).first()
            if user:
                user.asn = 1
                user.applicationStatus = "applying"
                db.session.commit()

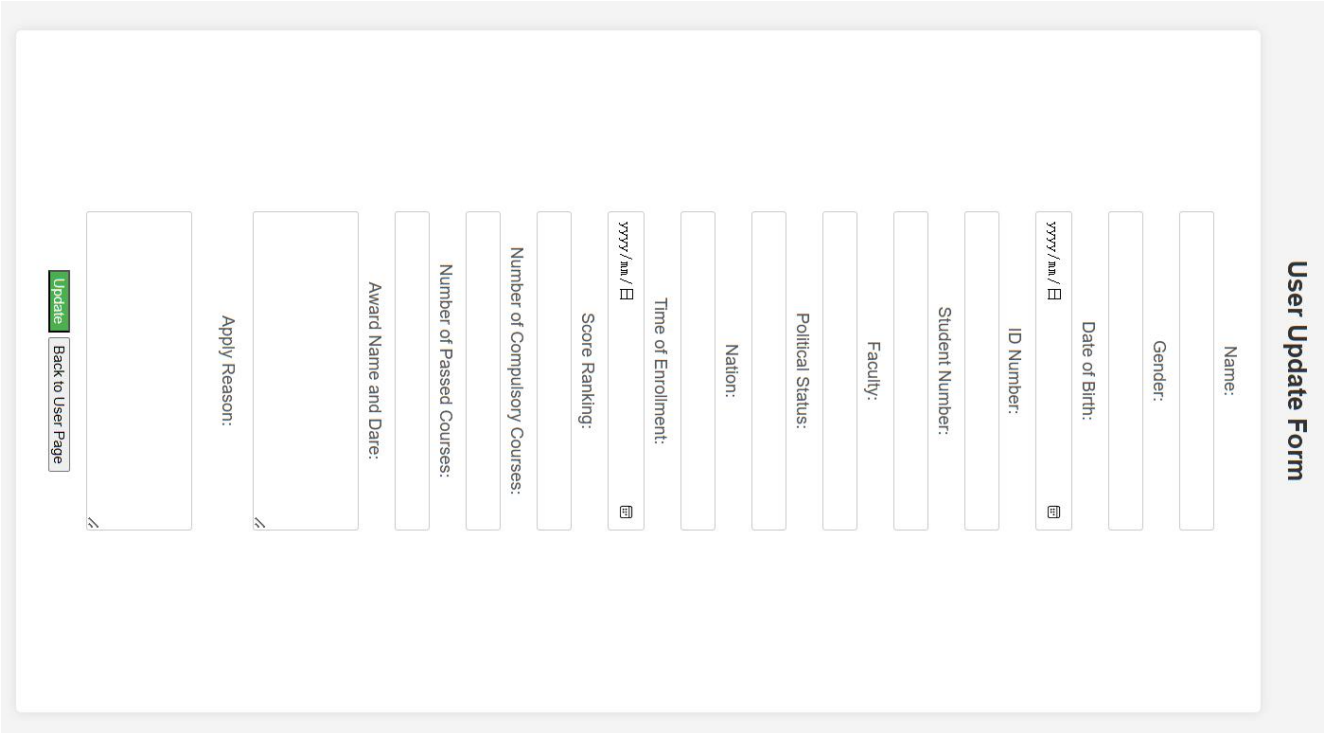
            return redirect(url_for('user_page', username=username))

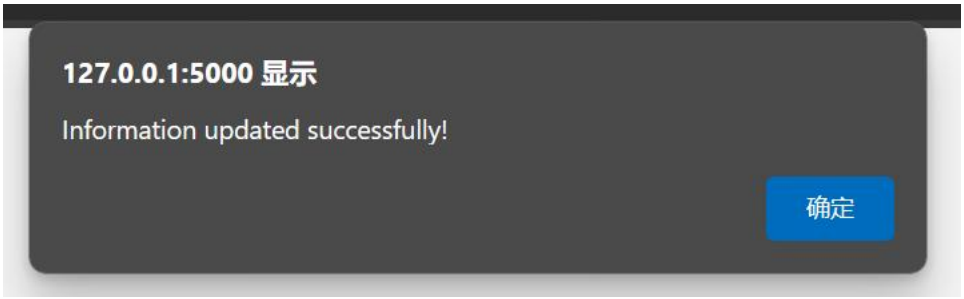
        else:
            user = User.query.filter_by(username=username).first()
            if user and user.asn == 1:
                return render_template('already_applied.html', username=username)
            elif user and user.asn == 0:
                return render_template('user_apply.html', username=username)
            elif user and user.asn == 2 or 3 or 4:
                return '''
                                    <script>
                                        alert("The application is already under review
and you cannot change the information");
                                        window.location.href = "{0}";
```

```
        </script>
    '''.format(url_for('user_page', username=username))
```

This is a page for a user to apply for a scholarship, asking the user to fill in the following information on the page. When clicking from the userpage to the application page, the user's current status is also determined. When the user has never applied, the user will be allowed to apply normally, and when the user has applied but has not yet started the approval, the user can update the submitted application information. If the user has submitted the application information and the faculty department has already started the approval process, the user is alerted by an alert that the approval is in progress and the submission cannot be modified.

Apply_update page






```
@app.route('/apply_update/<username>', methods=['GET', 'POST'])
def apply_update(username):
    if request.method == 'POST':
        name = request.form['name']
        gender = request.form['gender']
```

```
            birth = request.form['birth']
            IDnum = request.form['IDnum']
            studentNum = request.form['studentNum']
            faculty = request.form['faculty']
            politicstatus = request.form['politicstatus']
            nation = request.form['nation']
            timeofenrollment = request.form['timeofenrollment']
            scoreRanking = request.form['scoreRanking']
            NumberofCompulsoryCourses =
request.form['NumberofCompulsoryCourses']
            PassedNum = request.form['PassedNum']
            AwardName = request.form['AwardName']
            ApplyReason = request.form['ApplyReason']

            current_date = datetime.now().date()

            application = ApplyTable.query.filter_by(username=username).first()
            if application:
                application.name = name
                application.gender = gender
                application.birth = birth
                application.IDnum = IDnum
                application.studentNum = studentNum
                application.faculty = faculty
                application.politicstatus = politicstatus
                application.nation = nation
                application.timeofenrollment = timeofenrollment
                application.scoreRanking = scoreRanking
                application.NumberofCompulsoryCourses =
NumberofCompulsoryCourses
                application.PassedNum = PassedNum
                application.AwardName = AwardName
                application.ApllyDate = current_date
                application.ApplyReason = ApplyReason

                db.session.commit()

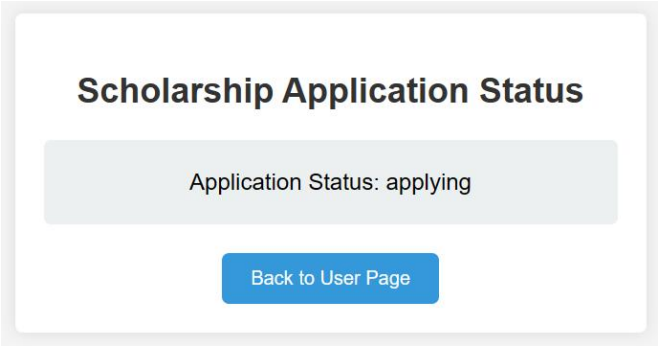
                return """
                                <script>
                                    alert('Information updated successfully!');
                                    window.location.href = '{}';
                                </script>
                            """.format(url_for('user_page', username=username))
            else:
```

```
            return 'Application not found.'

    return render_template('apply_update.html', username=username)
```

The back-end code of Apply_update page is similar to that of User_apply page, which operates on a row of a table in the database. However, the update changes the information after the corresponding row is found through username. After the update, the user is alerted by alert that the update was successful, and then returned to the user's home page.

Application_status




```
@app.route('/application_status')
def application_status():
    username = request.args.get('username')
    user = User.query.filter_by(username=username).first()

    if user:
        application_status = user.applicationStatus
        return render_template('application_status.html', username=username,
application_status=application_status)
    else:
        return 'User not found.'
```

Front-end core code:

```
<div class="container">
    <h2>Scholarship Application Status</h2>

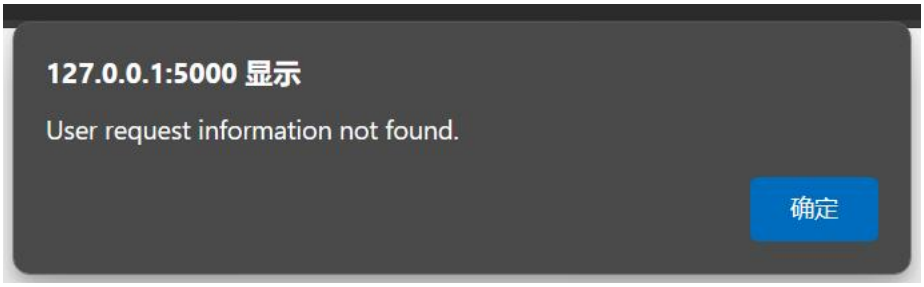
    <div class="central-box">
        <p>Application Status: {{ application_status }}</p>
    </div>

    <div class="buttons">
        <button onclick="window.location.href='{{ url_for('user_page', username=username) }}'">Back to User Page</button>
```

This is a page to query the current application status of the user. It will check the status of the current student's scholarship application by checking the approval status of the tutor and other roles in the database

User_applyinfo page






```
@app.route('/user_applyinfo/<username>')
def user_applyinfo(username):
    application = ApplyTable.query.filter_by(username=username).first()
    if application:
        return render_template('user_applyinfo.html', username=username,
application=application)
    else:
        return """
                    <script>
                        alert('User request information not found.');
                        window.location.href = '{}';
                    </script>
                """.format(url_for('user_page', username=username))
```

Front-end core code:

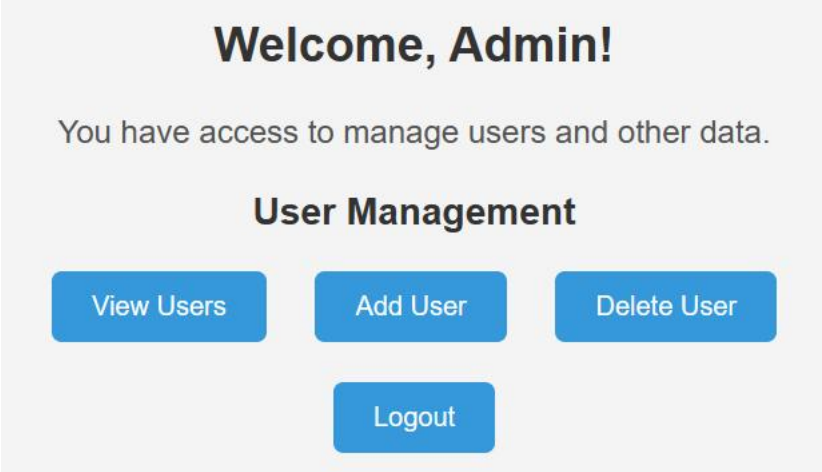
```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/user_applyinfoStyle.css') }}">
    <title>User Application Information</title>
</head>
<body>
    <h2>User Application Information</h2>
    <p><strong>Username:</strong> {{ username }}</p>
    <p><strong>Name:</strong> {{ application.name }}</p>
    <p><strong>Gender:</strong> {{ application.gender }}</p>
    <p><strong>Date of Birth:</strong> {{ application.birth }}</p>
    <p><strong>ID Number:</strong> {{ application.IDnum }}</p>
    <p><strong>Student Number:</strong> {{ application.studentNum }}</p>
    <p><strong>Faculty:</strong> {{ application.faculty }}</p>
    <p><strong>Political Status:</strong> {{ application.politicstatus }}</p>
    <p><strong>Nation:</strong> {{ application.nation }}</p>
    <p><strong>Time of Enrollment:</strong>
{{ application.timeofenrollment }}</p>
    <p><strong>Score Ranking:</strong> {{ application.scoreRanking }}</p>
    <p><strong>Number of Compulsory Courses:</strong>
{{ application.NumberofCompulsoryCourses }}</p>
    <p><strong>Number of Passed Courses:</strong>
{{ application.PassedNum }}</p>
    <p><strong>Award Name and Date:</strong>
{{ application.AwardName }}</p>
    <p><strong>Apply Date:</strong> {{ application.ApplyDate }}</p>
    <p><strong>Apply Reason:</strong> {{ application.ApplyReason }}</p>
    <button onclick="window.location.href='{{ url_for('user_page',
username=username) }}'">Back to User Page</button>
</body>
</html>
```

This page is used to view the application information submitted by the user, and query the corresponding information in the apply_table by using the username passed by the url. If there is no current request, the User request information is not found via alert.

3.2.3. Admin role

Admin page




```
@app.route('/admin_page')
def admin_page():
    if 'admin_id' in session:
        return render_template('admin_page.html')
    else:
        return redirect(url_for('admin_login'))
```

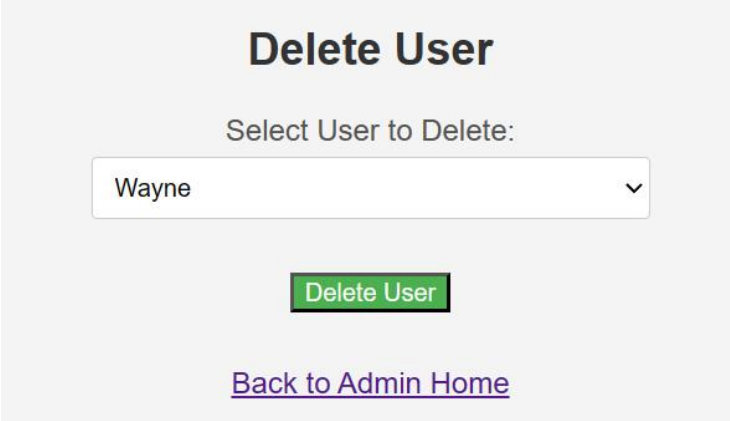
Front-end core code:

```
<h3>User Management</h3>
<button onclick="window.location.href='{{ url_for('admin_view_users') }}'">View
Users</button>
<button onclick="window.location.href='{{ url_for('admin_add_user') }}'">Add
User</button>
<button onclick="window.location.href='{{ url_for('admin_delete_user') }}'">Delete
User</button>

<div class="buttons">
    <button
onclick="window.location.href='{{ url_for('home') }}'">Logout</button>
</div>
```

This is admin's home page, which allows admin to choose the next action. In the admin role, admin can add, delete, and query users

Admin_delete_user page






```
@app.route('/admin_delete_user', methods=['GET', 'POST'])
def admin_delete_user():
    if request.method == 'POST':
        username = request.form['username']

        # Retrieve the user from the database based on the provided username
        user = User.query.filter_by(username=username).first()

        if user:
            # Attempt to delete related data in the apply_table before deleting the user
            try:
                # Delete entries related to the user in the apply_table
                ApplyTable.query.filter_by(username=username).delete()
                db.session.commit()
            except IntegrityError:
                # If a foreign key constraint error occurs, rollback the session and display an error message
                db.session.rollback()
                return 'Error: Cannot delete user. Related data in apply_table exists.'

            db.session.delete(user)
            db.session.commit()
```

```
    user_list = User.query.all()

    return render_template('admin_delete_user.html', user_list=user_list)
```

Front-end core code:

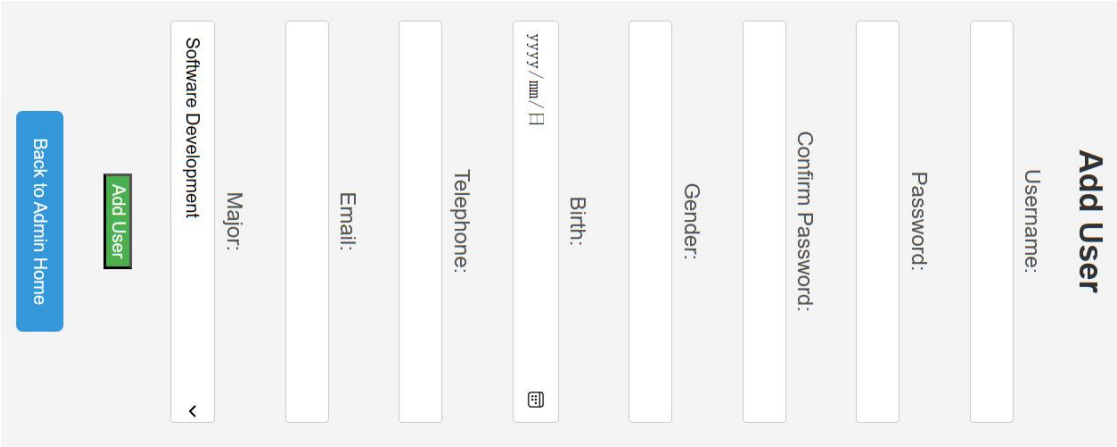
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/admin_add_user.css') }}">
    <title>Delete User</title>
    <script>
        function showMessage() {
            alert('User deleted successfully!');
            location.reload();
            return true;
        }
    </script>

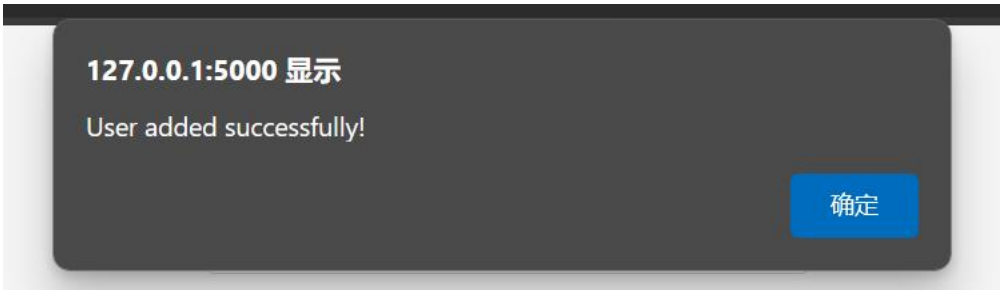
</head>
<body>
<h2>Delete User</h2>
<form method="post" action="{{ url_for('admin_delete_user') }}" onsubmit="return
showMessage()">
    <label for="username">Select User to Delete:</label>
    <select id="username" name="username" required>
        {% for user in user_list %}
            <option value="{{ user.username }}">{{ user.username }}</option>
        {% endfor %}
    </select>
    <br>

    <input type="submit" value="Delete User">
</form>
<br>
<a href="{{ url_for('admin_page') }}">Back to Admin Home</a>
</body>
</html>
```

In the admin Delete user page, admin is given a selection box to select all existing user names. admin will select the user by these user names and then click the delete user button to delete the user. admin can also click on the hyperlink to return to the admin page

Admin_add_user page






```
@app.route('/admin_add_user', methods=['GET', 'POST'])
def admin_add_user():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        gender = request.form['gender']
        birth = request.form['birth']
        telephone = request.form['telephone']
        email = request.form['email']
        major = request.form['major']

        if password != confirm_password:
            message = 'Passwords do not match. Please try again.'
        else:
            if User.query.filter_by(username=username).first():
                message = 'Username already exists. Please choose a different
username.'
            else:
                new_user = User(username=username, password=password,
gender=gender, birth=birth,
                                telephone=telephone, email=email,
major=major)

                db.session.add(new_user)
```

```
                db.session.commit()
                message = 'Registration successful! You can now log in.'
            return render_template('admin_add_user.html', message=message))
```

Front-end core code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin_add_user.css') }}">
    <title>Add User</title>
    <script>
        // Function to display the message in an alert and redirect to user_login
        function displayMessage() {
            var message = "{{ message }}";
            if (message && message !== "None") {
                if (message === 'Registration successful! You can now log in.') {
                    alert(message);
                    window.location.href = '{{ url_for('admin_add_user') }}';
                } else {
                    alert(message);
                }
            }
        }

        // Function to validate password
        function validatePassword() {
            var password = document.getElementById("password").value;
            var confirmPassword = document.getElementById("confirm_password").value;

            if (password !== confirmPassword) {
                alert("Passwords do not match!");
                return false;
            }
            displayMessage();
            return true;
        }
    </script>
```

```
  <label for="gender">Gender:</label>
  <input type="text" id="gender" name="gender" required><br>

  <label for="birth">Birth:</label>
  <input type="date" id="birth" name="birth" required><br>

  <label for="telephone">Telephone:</label>
  <input type="text" id="telephone" name="telephone" required><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required><br>

  <label for="major">Major:</label>
  <select id="major" name="major" required>
      <option value="Software Development">Software Development</option>
      <option value="International Trade">International Trade</option>
      <option value="Computer Mathematics">Computer Mathematics</option>
  </select><br>

  <input type="submit" value="Add User">
form>

iv class="buttons">
  <button onclick="window.location.href='{{ url_for('admin_page') }}'">Back to Admin Home</button>
```

The underlying logic of adding a user by admin is basically the same as that of registering a new user by filling in basic information and then using the database insert command to erase the data

Admin_view_user page



View Users

Username	Gender	Birth	Telephone	Email	Major
Wayne	male	2003-11-11	123	123	Software Development
Bonnie	Male	2024-03-12	+8615022289318	1406298154@qq.com	Software Development
Anna	Female	2024-04-07	+8615022289318	1406298154@qq.com	Software Development
Alice	Female	2024-04-07	+8615022289318	1406298154@qq.com	International Trade
Bill	Male	2024-04-07	+8615022289318	1406298154@qq.com	International Trade
John	Male	2024-04-07	+8615022289318	1406298154@qq.com	International Trade
Billy	Male	2024-04-07	+8615022289318	1406298154@qq.com	Computer Mathematics
Kim	Male	2024-04-07	+8615022289318	1406298154@qq.com	Computer Mathematics
Regina	Female	2024-04-07	+8615022289318	1406298154@qq.com	Computer Mathematics
AVON	Female	1979-01-23	18622410780	18622410780@163.com	International Trade
Leo	Male	2004-02-22	15022289318	1406298154@qq.com	Software Development
Leo12333	Male	2024-04-27	15022289318	1406298154@qq.com	International Trade

Back to Admin Home

```
@app.route('/admin_view_users')
def admin_view_users():
    users = User.query.all()

    return
render_template('admin_view_users.html',
users=users)
```

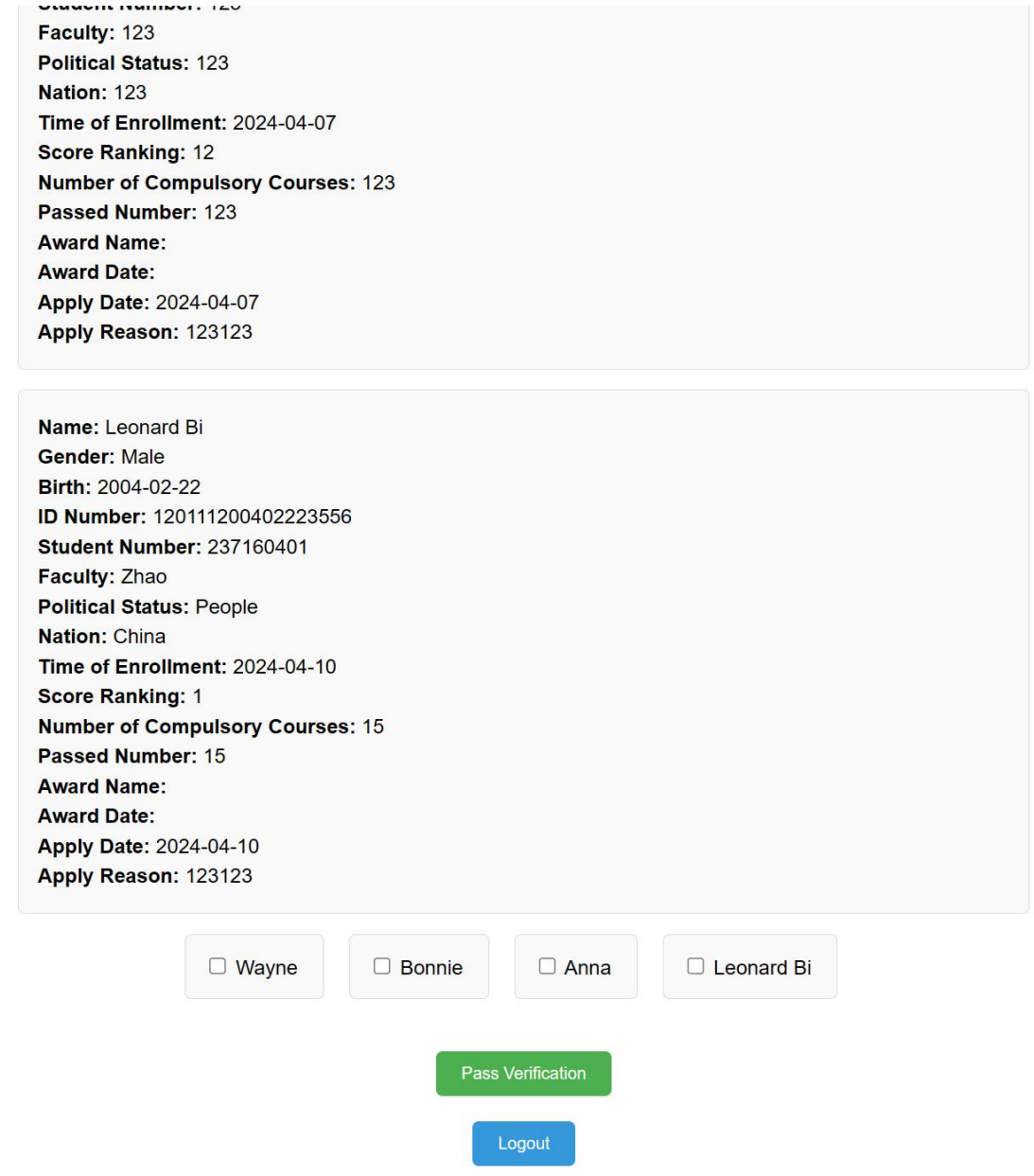
Front-end core code:

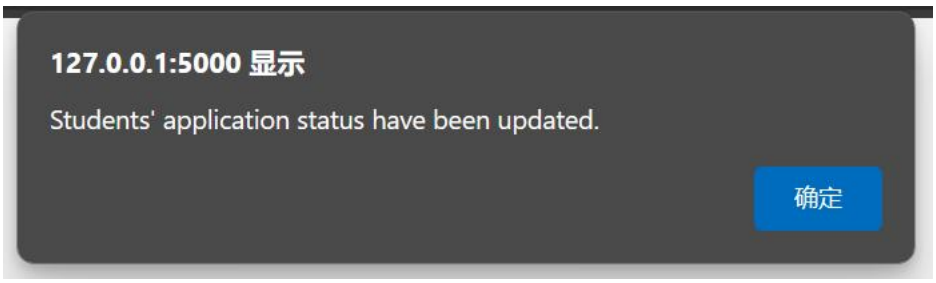
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>View Users</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/admin_view_users.css') }}">
</head>
<body>
    <h2>View Users</h2>
    <table>
        <thead>
            <tr>
                <th>Username</th>
                <th>Gender</th>
                <th>Birth</th>
                <th>Telephone</th>
                <th>Email</th>
                <th>Major</th>
            </tr>
        </thead>
        <tbody>
            {% for user in users %}
                <tr>
                    <td>{{ user.username }}</td>
                    <td>{{ user.gender }}</td>
                    <td>{{ user.birth }}</td>
                    <td>{{ user.telephone }}</td>
                    <td>{{ user.email }}</td>
                    <td>{{ user.major }}</td>
                </tr>
            {% endfor %}
        </tbody>
    </table>
    <div class="buttons">
        <button onclick="window.location.href='{{ url_for('admin_page') }}'">Back to Admin Home</button>
    </div>
</body>
</html>
```

In the admin view user page, although the route only displays all the information of the user table, in html, all the information needs to be displayed according to the <tr> and <th> tags through the for loop, so that all the user information can be displayed beautifully

3.2.4. Mentor role

Mentor Page






```
@app.route('/mentor_pass_verification', methods=['POST'])
def mentor_pass_verification():
    selected_students = request.form.getlist('selected_students')
```

```
        selected_students_set = set()

        selected_students_set.update(selected_students)

        for student_name in selected_students:
            student = User.query.filter_by(username=student_name).first()
            if student:
                student.applicationStatus = 'mentor passed'
                student.asn = 2
                db.session.add(student)

        all_students = [student.username for student in User.query.all()]

        for student_name in all_students:
            if student_name not in selected_students_set:
                student = User.query.filter_by(username=student_name).first()
                if student and student.applicationStatus == 'applying':
                    student.applicationStatus = 'mentor did not pass'
                    student.asn = 2
                    db.session.add(student)

        db.session.commit()

        return '<script>alert("Students\' application status have been updated.");
window.history.back();</script>'

@app.route('/mentor_page')
def mentor_page():
        mentorname = request.args.get('mentorname')
        mentor = Mentor.query.filter_by(mentorname=mentorname).first()

        if mentor:
            students = User.query.filter_by(major=mentor.major

).all()
            applications = []
            for student in students:
                application =
ApplyTable.query.filter_by(username=student.username).first()
                if application:
                    applications.append(application)
            return render_template('mentor_page.html', applications=applications)
        else:
            return "Mentor not found."
```

Front-end core code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mentor Page</title>
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/mentor_page.css') }}">
</head>
<body>
<div class="container">
    <h2>Mentor Page</h2>
    <div class="student-applications">
        <h3>Student Applications</h3>
        <ul>
            {% for application in applications %}
                <li>
                    <p><strong>Name:</strong> {{ application.name }}</p>
                    <p><strong>Gender:</strong> {{ application.gender }}</p>
                    <p><strong>Birth:</strong> {{ application.birth }}</p>
                    <p><strong>ID Number:</strong>
{{ application.IDnum }}</p>
                    <p><strong>Student Number:</strong>
{{ application.studentNum }}</p>
                    <p><strong>Faculty:</strong> {{ application.faculty }}</p>
                    <p><strong>Political Status:</strong>
{{ application.politicstatus }}</p>
                    <p><strong>Nation:</strong> {{ application.nation }}</p>
                    <p><strong>Time of Enrollment:</strong>
{{ application.timeofenrollment }}</p>
                    <p><strong>Score Ranking:</strong>
{{ application.scoreRanking }}</p>
                    <p><strong>Number of Compulsory Courses:</strong>
{{ application.NumberofCompulsoryCourses }}</p>
                    <p><strong>Passed Number:</strong>
{{ application.PassedNum }}</p>
                    <p><strong>Award Name:</strong>
{{ application.AwardName }}</p>
                    <p><strong>Award Date:</strong>
{{ application.AwardDate }}</p>
                    <p><strong>Apply Date:</strong>
{{ application.ApplyDate }}</p>
```

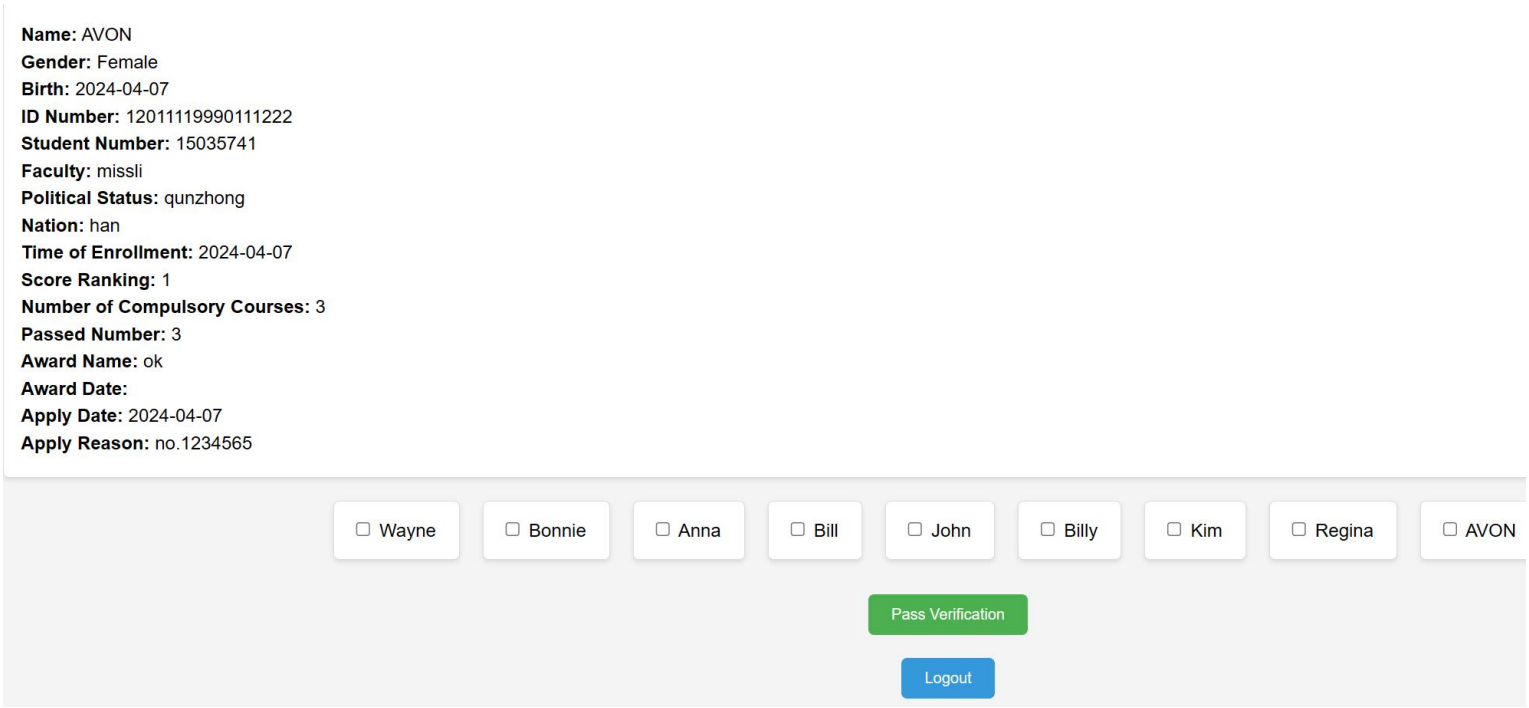
```
                        <p><strong>Apply Reason:</strong>
{{ application.ApplyReason }}</p>
                    </li>
                {% endfor %}
            </ul>
            <form action="{{ url_for('mentor_pass_verification') }}" method="post">
                <ul>
                    {% for application in applications %}
                        <li>
                            <input type="checkbox" name="selected_students"
value="{{ application.name }}">
                            <label>{{ application.name }}</label>
                        </li>
                    {% endfor %}
                </ul>
                <button type="submit">Pass Verification</button>
            </form>
        </div>

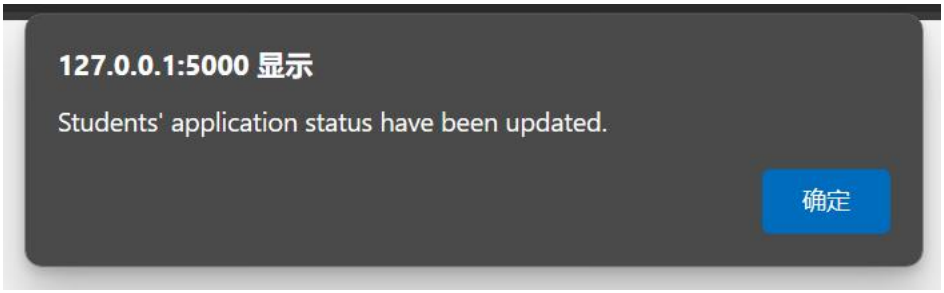
        <div class="buttons">
            <button
onclick="window.location.href='{{ url_for('mentor_login') }}'">Logout</button>
        </div>
</div>
</body>
</html>
```

In the role of mentor, the main function of a mentor is to approve students with the same major as that mentor. So in the database, we need to find all the students with the same major as mentor and display their data in the apply_table. At the bottom of the page, the mentor will check the information of all the students who passed the examination initially, and click the Pass Vertification button to change the data information of these students to mentor passed. And the information of these students will be passed to the next role, faculty, for the next step of approval. When the button is clicked, the mentor approval message is also displayed.

3.2.5. Faculty role

Faculty Page






```
@app.route('/faculty_page', methods=['GET', 'POST'])
def faculty_page():
    if request.method == 'GET':
        passed_students = User.query.filter_by(applicationStatus='mentor
passed').all()

        applications = []

        for student in passed_students:
            application =
ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)
                print(application)

        if not applications:
            return 'No applications found.'
```

```
            return render_template('faculty_page.html', applications=applications)
        else:
            pass


@app.route('/faculty_pass_verification', methods=['POST'])
def faculty_pass_verification():
        selected_students = request.form.getlist('selected_students')

        selected_students_set = set()

        selected_students_set.update(selected_students)

        for student_name in selected_students:
            student = User.query.filter_by(username=student_name).first()
            if student:
                student.applicationStatus = 'faculty passed'
                student.asn = 2
                db.session.add(student)

        all_students = [student.username for student in User.query.all()]

        for student_name in all_students:
            if student_name not in selected_students_set:
                student = User.query.filter_by(username=student_name).first()
                if student and student.applicationStatus == 'mentor did not pass':
                    student.applicationStatus = 'faculty did not pass'
                    student.asn = 3
                    db.session.add(student)

        db.session.commit()

        return '<script>alert("Students\' application status have been updated.");
window.history.back();</script>'
```

Front-end core code:

```
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Faculty Passed Applications</title>
        <link rel="stylesheet" href="{{ url_for('static',
filename='css/faculty_page.css') }}">
```

```
</head>
<body>
<h1>Faculty Passed Applications</h1>
<ul>
    {% for application in applications %}
        <li>
            <p><strong>Name:</strong> {{ application.name }}</p>
            <p><strong>Gender:</strong> {{ application.gender }}</p>
            <p><strong>Birth:</strong> {{ application.birth }}</p>
            <p><strong>ID Number:</strong> {{ application.IDnum }}</p>
            <p><strong>Student Number:</strong>
{{ application.studentNum }}</p>
            <p><strong>Faculty:</strong> {{ application.faculty }}</p>
            <p><strong>Political Status:</strong>
{{ application.politicstatus }}</p>
            <p><strong>Nation:</strong> {{ application.nation }}</p>
            <p><strong>Time of Enrollment:</strong>
{{ application.timeofenrollment }}</p>
            <p><strong>Score Ranking:</strong>
{{ application.scoreRanking }}</p>
            <p><strong>Number of Compulsory Courses:</strong>
{{ application.NumberofCompulsoryCourses }}</p>
            <p><strong>Passed Number:</strong>
{{ application.PassedNum }}</p>
            <p><strong>Award Name:</strong>
{{ application.AwardName }}</p>
            <p><strong>Award Date:</strong> {{ application.AwardDate }}</p>
            <p><strong>Apply Date:</strong> {{ application.ApplyDate }}</p>
            <p><strong>Apply Reason:</strong>
{{ application.ApplyReason }}</p>
        </li>
    {% endfor %}
    <form action="{{ url_for('faculty_pass_verification') }}" method="post">
        <ul>
            {% for application in applications %}
                <li>
                    <input type="checkbox" name="selected_students"
value="{{ application.name }}">
                    <label>{{ application.name }}</label>
                </li>
            {% endfor %}
        </ul>
        <button type="submit">Pass Verification</button>
    </form>
```

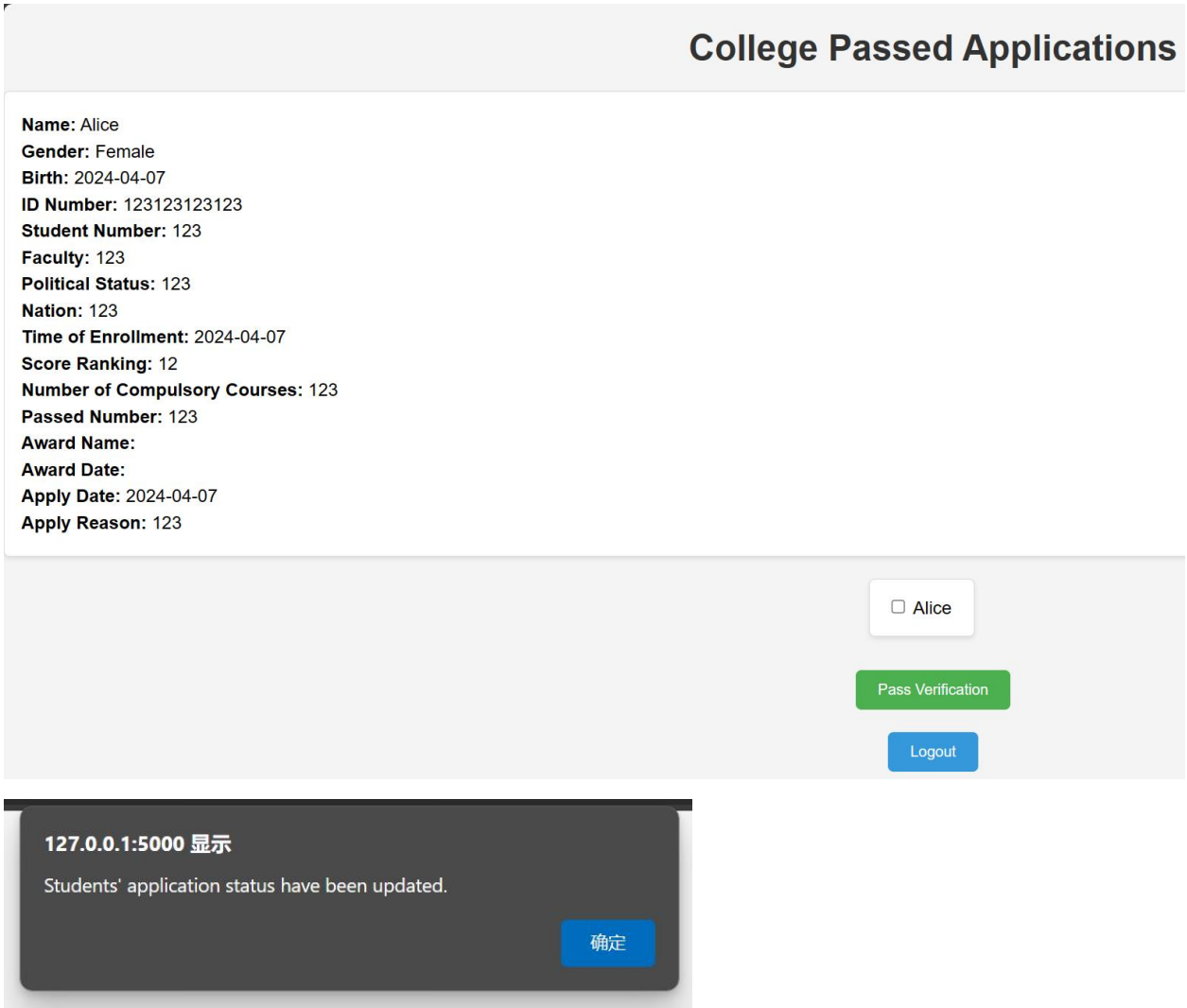
```
        </div>

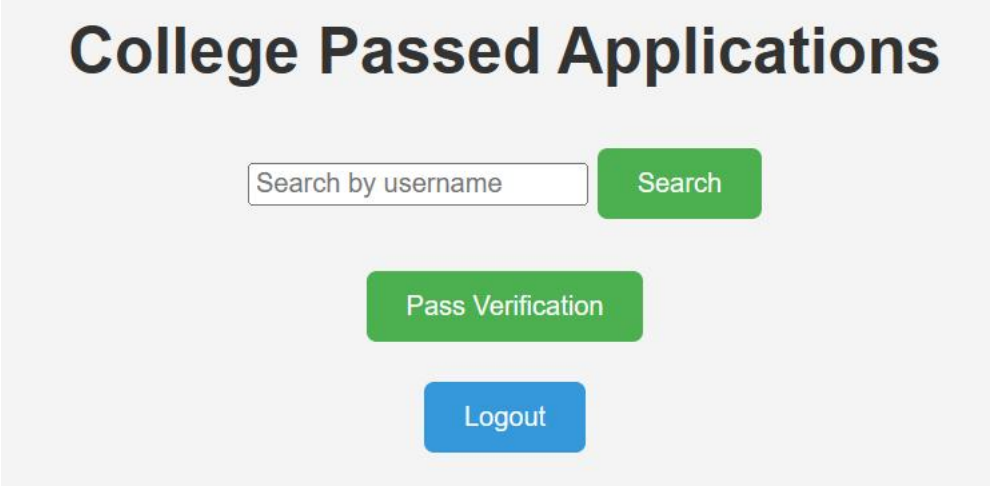
        <div class="buttons">
            <button
onclick="window.location.href='{{ url_for('faculty_login') }}'">Logout</button>
        </div>
</ul>
</body>
</html>
```

The underlying logic of the faculty role is similar to that of mentor, but the workload is much higher than that of mentor. Because mentor only preliminarily approves the information of students with the same major role, while faculty combines all the information of students preliminarily approved by the three mentors for approval. These students will be approved and then transferred to the college for approval.

3.2.6. College role

College Page






```
@app.route('/college_page', methods=['GET', 'POST'])
def college_page():
    if request.method == 'POST':
        search_query = request.form.get('search_query')
        if search_query:
            passed_students =
User.query.filter(User.username.like(f'%{search_query}%')).all()
        else:
            passed_students = User.query.filter_by(applicationStatus='faculty
passed').all()

        applications = []

        for student in passed_students:
            application =
ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)

        return render_template('college_page.html', applications=applications)

    else:
        passed_students = User.query.filter_by(applicationStatus='faculty
passed').all()

        applications = []

        for student in passed_students:
            application =
ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)
```

```
        return render_template('college_page.html', applications=applications)


@app.route('/college_pass_verification', methods=['POST'])
def college_pass_verification():
    selected_students = request.form.getlist('selected_students')

    selected_students_set = set()

    selected_students_set.update(selected_students)

    for student_name in selected_students:
        student = User.query.filter_by(username=student_name).first()
        if student:
            student.applicationStatus = 'Congratulation! You have successfully
applied for the scholarship!'
            student.asn = 4
            db.session.add(student)

    all_students = [student.username for student in User.query.all()]

    for student_name in all_students:
        if student_name not in selected_students_set:
            student = User.query.filter_by(username=student_name).first()
            if student and student.applicationStatus == 'mentor did not pass':
                student.applicationStatus = 'college did not pass'
                student.asn = 4
                db.session.add(student)

    db.session.commit()

    return '<script>alert("Students\' application status have been updated.");
window.history.back();</script>'
```

Front-end core code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>College Passed Applications</title>
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/faculty_page.css') }}">
```

```
</head>
<body>
<h1>College Passed Applications</h1>
<form action="{{ url_for('college_page') }}" method="POST">
    <input type="text" name="search_query" placeholder="Search by username">
    <button type="submit">Search</button>
</form>
<ul>
    {% if applications %}
        {% for application in applications %}
            <li>
            <p><strong>Name:</strong> {{ application.name }}</p>
            <p><strong>Gender:</strong> {{ application.gender }}</p>
            <p><strong>Birth:</strong> {{ application.birth }}</p>
            <p><strong>ID Number:</strong> {{ application.IDnum }}</p>
            <p><strong>Student Number:</strong>
{{ application.studentNum }}</p>
            <p><strong>Faculty:</strong> {{ application.faculty }}</p>
            <p><strong>Political Status:</strong>
{{ application.politicstatus }}</p>
            <p><strong>Nation:</strong> {{ application.nation }}</p>
            <p><strong>Time of Enrollment:</strong>
{{ application.timeofenrollment }}</p>
            <p><strong>Score Ranking:</strong>
{{ application.scoreRanking }}</p>
            <p><strong>Number of Compulsory Courses:</strong>
{{ application.NumberofCompulsoryCourses }}</p>
            <p><strong>Passed Number:</strong>
{{ application.PassedNum }}</p>
            <p><strong>Award Name:</strong>
{{ application.AwardName }}</p>
            <p><strong>Award Date:</strong> {{ application.AwardDate }}</p>
            <p><strong>Apply Date:</strong> {{ application.ApplyDate }}</p>
            <p><strong>Apply Reason:</strong>
{{ application.ApplyReason }}</p>
            </li>
        {% endfor %}
        <form action="{{ url_for('college_pass_verification') }}" method="post">
            <ul>
                {% for application in applications %}
                    <li>
                        <input type="checkbox" name="selected_students"
value="{{ application.name }}">
                        <label>{{ application.name }}</label>
```

```
                    </li>
                {% endfor %}
            </ul>
            <button type="submit">Pass Verification</button>
        </form>
    {% else %}
        <p>No applications found.</p>
    {% endif %}
</ul>

<div class="buttons">
    <button
onclick="window.location.href='{{ url_for('college_login') }}'">Logout</button>
</div>
</body>
</html>
```

The college role is to conduct the final approval, and the approval results are reflected in the user page. Information on all user applications approved by faculty will be collected before final approval. There will also be a message after approval, and there will also be a button to go back and forth to the login screen. At the same time, a fuzzy query can be performed in the text box, and all the relevant students are displayed to determine whether the scholarship can be passed

3.3. Use of unfamiliar construct

For invoking the database in flask routing, this project uses the creation of a data model to store the data, for example:

```
class User(db.Model):
    iduser = db.Column(db.Integer, primary_key=True, autoincrement=True)
    username = db.Column(db.String(45), unique=True, nullable=False)
    password = db.Column(db.String(45), nullable=False)
    gender = db.Column(db.String(45))
    birth = db.Column(db.Date)
    telephone = db.Column(db.String(45))
    email = db.Column(db.String(45), unique=True, nullable=False)
    applicationStatus = db.Column(db.String(45))
    asn = db.Column(db.Integer, default='0')
    major = db.Column(db.String(45), nullable=False)
```

3.4. Error handling in coding

This project also has targeted exception handling, for example, in the admin delete user, if a foreign key constraint error occurs during the deletion process, the session is rolled back so as not to affect the consistency of the database, and an error message is returned indicating that the user cannot be deleted. Because there is data associated with the user in apply_table. Then, regardless of whether the user was successfully deleted, the user instance in the user table is deleted and the session is committed to save the changes. Finally, re-fetch all users from the database and pass them into the rendering template 'admin_delete_user.html' so that the administrator can view the updated list of users.

```
@app.route( rule: '/admin_delete_user', methods=['GET', 'POST'])
def admin_delete_user():
    if request.method == 'POST':
        username = request.form['username']

        # Retrieve the user from the database based on the provided username
        user = User.query.filter_by(username=username).first()

        if user:
            # Attempt to delete related data in the apply_table before deleting the user
            try:
                # Delete entries related to the user in the apply_table
                ApplyTable.query.filter_by(username=username).delete()
                db.session.commit()
            except IntegrityError:
                # If a foreign key constraint error occurs, rollback the session and display an error message
                db.session.rollback()
                return 'Error: Cannot delete user. Related data in apply_table exists.'

            # Delete the user from the user table
            db.session.delete(user)
            db.session.commit()

    # Retrieve the updated list of users from the database
    user_list = User.query.all()

    return render_template( template_name_or_list: 'admin_delete_user.html', user_list=user_list)
```

3.5. Internal Documentation

When naming each file, it will start with the role name, or with the main function, so that you can quickly find the file name when you look it up.

At the same time, after each modification of the code, the shortcut key "ctrl+alt+l" will be used to indent the code, which can effectively avoid errors in circular statements, and can also make the code look more beautiful, convenient for later debugging and testing

3.6. Testing

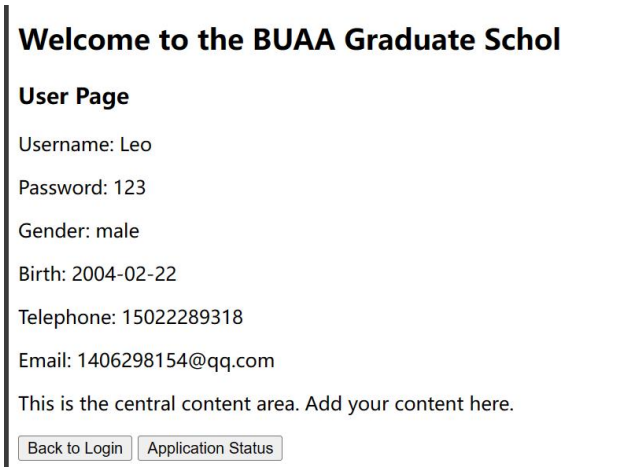
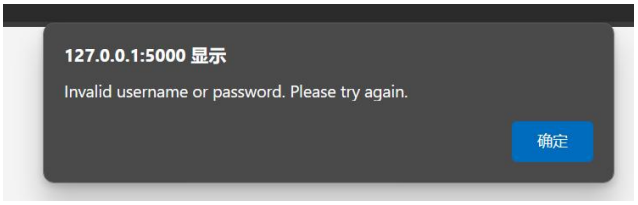
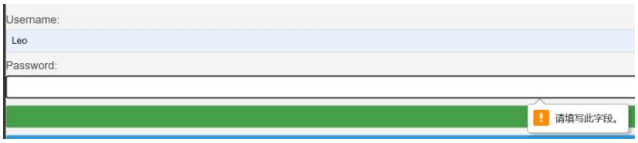
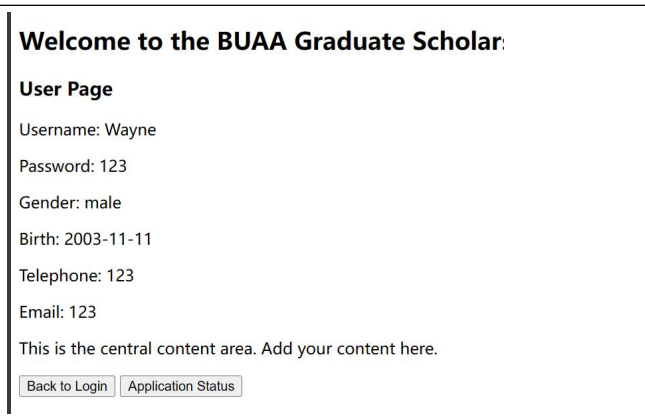
Test plan:

This test will be based on flask's web function to conduct a comprehensive test for this project.

First, all the functions of all the roles are tested to see if there are any anomalies,

Secondly, it will focus on testing each form information. Finally, it will check whether the connection between the program and the database, query, add, delete and other functions are complete and available.

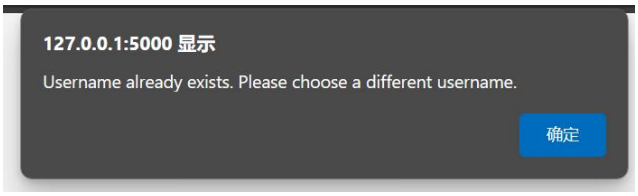
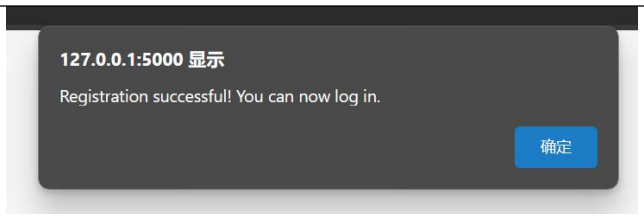
3.6.1. Login

No	Action	Expected result	Actual result	Pass/Fail
1	Login with username "Leo", password "123"	Login and jump to the Leo's user page	Login and jump to the user page 	Pass
2	Login with username "Leo", password "1"	Prompt out error message with alert		Pass
3	Login with username "Leo"	Prompt user to enter password		Pass
4	Login with username "Wayne", password "123"	Login and jump to the Wayne's user page		Pass

Name of tester: Li Yiwei Date of Test: 2024/2/26 Test P/F: P

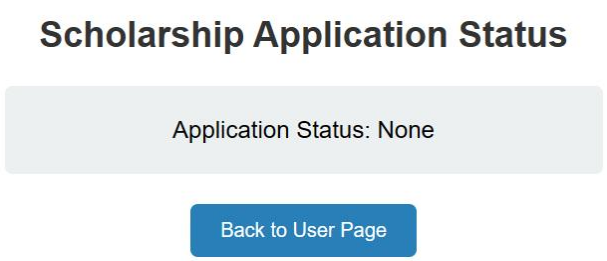
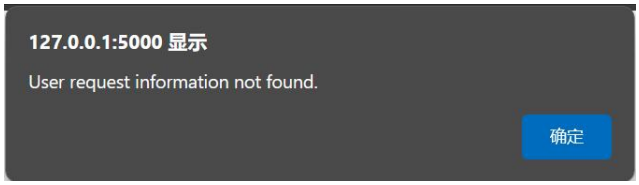
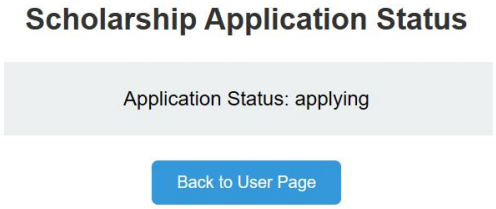
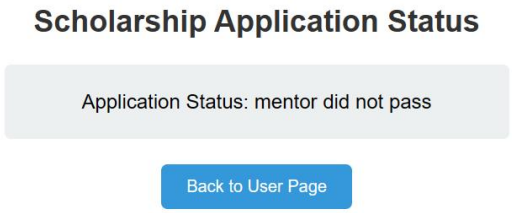
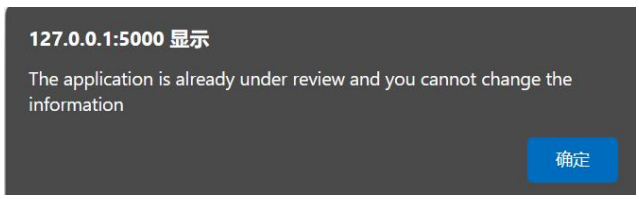
3.6.2. User Role

User register

No	Action	Expected result	Actual result	Pass/Fail
5	Sign up with username “Leo”, password “123”, confirm password “123”, gender male, birth 2004/2/22, telephone “15022289318”, email “1406298154@qq.com”	Prompt user the username already used		Pass
6	Sign up with username “Leo123”, password “123”, confirm password “123”, gender male, birth 2004/2/22, telephone “15022289318”, email “1406298154@qq.com”	Sign up successfully and prompt message, then jump to the user login page		Pass
7	No password was entered during registration	Prompt user to enter password		Pass
8	Enter password 123, and confirm password 12	Prompt user the passwords do not match		Pass

Name of tester: Lu Pengyu Date of Test: 2024/2/26 Test P/F: P

User Page

No	Action	Expected result	Actual result	Pass/Fail
9	Click back to login button	Jump to user login page	As expected	Pass
10	Click apply button(haven't applied before)	Jump to apply page	As expected	Pass
11	Click Application status button	Jump to Application status page and display "Application status: None"		Pass
12	Click check Application information button	Alert message to prompt user haven't applied		Pass
13	Click view application standard	Jump to application_info page	As expected	Pass
14	Fill in information and submit the application	Submit successfully and back to user page	As expected	Pass
15	Click check Application information button(after submit application)	Application status display: applying		Pass
16	Click check Application information button(after mentor rejected)	Application Status: mentor did not pass		Pass
17	Click apply button after mentor started approval	Prompt user can't change the application information		Pass

Name of tester: Zheng Xinyu Date of Test: 2024/2/26 Test P/F: P

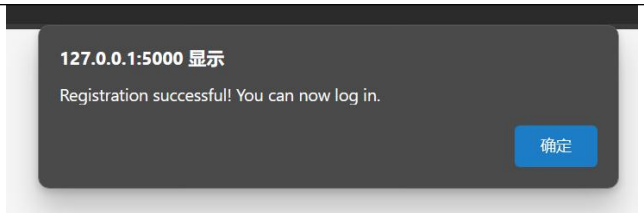
3.6.3. Admin Role

Admin page

No	Action	Expected result	Actual result	Pass/Fail
18	Click view users button	Jump to view user page and display all registered users	As expected	Pass
19	Click Add user button	Jump to add user page	As expected	Pass
20	Click Delete user button	Jump to delete user page	As expected	Pass
21	Click log out button	Back to login page	As expected	Pass

Name of tester: Zhao Dake Date of Test: 2024/2/27 Test P/F: P

Admin add user page

No	Action	Expected result	Actual result	Pass/Fail
22	Sign up with username "Leo", password "123", confirm password "123", gender male, birth 2004/2/22, telephone "15022289318", email "1406298154@qq.com"	Prompt user the username already used	Back to a blank admin add user page and nothing changed	**Fail**
23	Sign up with username "Leo123", password "123", confirm password "123", gender male, birth 2004/2/22, telephone "15022289318", email "1406298154@qq.com"	Sign up successfully and prompt message, then jump to the user login page		Pass

24	No password was entered during registration	Prompt user to enter password		Pass
25	Enter password 123, and confirm password 12	Prompt user the passwords do not match		Pass

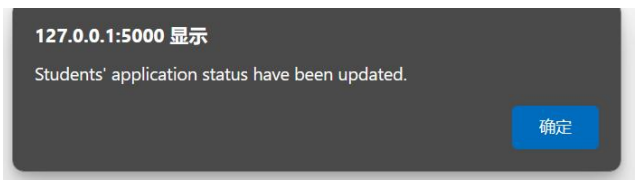
Name of tester: Zhao Hangliang Date of Test: 2024/2/27 Test P/F: F on NO.22

Admin delete user page

No	Action	Expected result	Actual result	Pass/Fail
26	Select user “123123” and delete	Prompt admin delete successfully and remove all data from database	The web page displays an error message indicating that the database cannot execute the command because there are constraints on executing the command	**Fail**
27	Click back to admin home	Back to main page	As expected	Pass

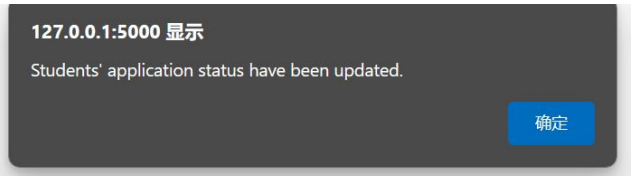
Name of tester: Lu Yixuan Date of Test: 2024/2/27 Test P/F: F on NO.26

3.6.4. Mentor role

No	Action	Expected result	Actual result	Pass/Fail
28	Mentor login	Display all students in same major with mentor	All student information is displayed	**Fail**
29	Select the student below and click the Pass Vertification button	A message is displayed indicating that the mentor information is updated successfully and the data is inserted into the database	As expected 	Pass
30	Click log out button	Back to login page	As expected	Pass

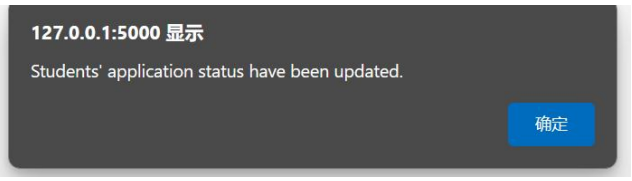
Name of tester: Wu Sijin Date of Test: 2024/2/27 Test P/F: F on NO.28

3.6.5. Faculty role

No	Action	Expected result	Actual result	Pass/Fail
31	Faculty login	Display all students that approved by mentor	Haven't display any students	**Fail**
32	Select the student below and click the Pass Vertification button	A message is displayed indicating that the faculty information is updated successfully and the data is inserted into the database	As expected 	Pass
33	Click log out button	Back to login page	As expected	Pass

Name of tester: Yu Qinnan Date of Test: 2024/2/28 Test P/F: F on NO.31

3.6.6. College role

No	Action	Expected result	Actual result	Pass/Fail
34	College login	Display all students that approved by Faculty, and display a search box on the top of website body	Display "No applications found."	**Fail**
35	Select the student below and click the Pass Vertification button	A message is displayed indicating that the college information is updated successfully and the data is inserted into the database	As expected 	Pass
36	Click log out button	Back to login page	As expected	Pass

Name of tester: Xu Shengju Date of Test: 2024/2/28 Test P/F: F on NO.34

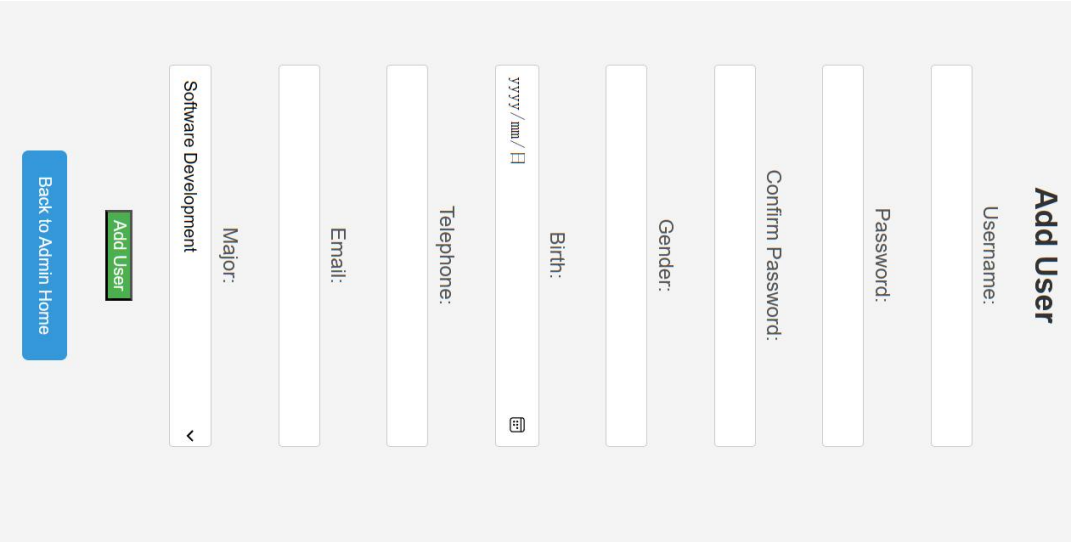
3.7. Debugging

3.7.1. Test No.22

Error:

No	Action	Expected result	Actual result	Pass/Fail
22	Sign up with username "Leo", password "123", confirm password "123", gender male, birth 2004/2/22, telephone "15022289318", email "1406298154@qq.com"	Prompt user the username already used	Back to a blank admin add user page and nothing changed	**Fail**

Error screenshot:



Analyze the cause of the error: Since this script uses Jinja2 language rendering, and at the end of the code is reload on the page. So the page is only refreshed, rather than refreshed after the form is submitted.

```
    <script>
        function showMessage() {
            alert('User added successfully!');
            location.reload();
            return true;
        }
    </script>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css') }}">
</head>
<body>
<h2>Add User</h2>
```

How to fix

The way to fix this bug is to pass all the information from the html to the route through the form action, and then do all the if else judgments in the route. Now create the str variable message at the top of the route, change the message information in different cases, pass it to html, and display it using alert

```python
@app.route( rule: '/admin_add_user', methods=['GET', 'POST'])
def admin_add_user():
    message = None
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        confirm_password = request.form['confirm_password']
        gender = request.form['gender']
        birth = request.form['birth']
        telephone = request.form['telephone']
        email = request.form['email']
        major = request.form['major']

        if password != confirm_password:
            message = 'Passwords do not match. Please try again.'
        else:
            if User.query.filter_by(username=username).first():
                message = 'Username already exists. Please choose a different username.'
            else:
                new_user = User(username=username, password=password, gender=gender, birth=birth,
                                telephone=telephone, email=email, major=major)

                db.session.add(new_user)
                db.session.commit()

                message = 'Registration successful! You can now log in.'

    return render_template( template_name_or_list: 'admin_add_user.html', message=message)
```

Test again

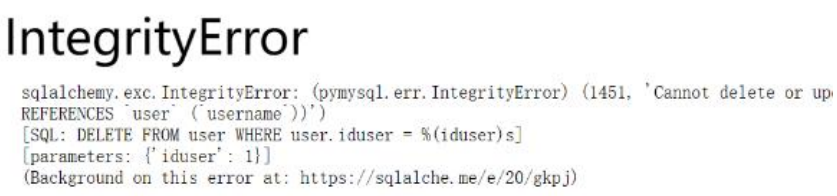
No	Action	Expected result	Actual result	Pass/Fail
22	Sign up with username “Leo”, password “123”, confirm password “123”, gender male, birth 2004/2/22, telephone “15022289318”, email “1406298154@qq.com”	Prompt user the username already used	Prompt user the username already used 	**Pass**

3.7.2. Test No.26

Error:

No	Action	Expected result	Actual result	Pass/Fail
26	Select user “123123” and delete	Prompt admin delete successfully and remove all data from database	The web page displays an error message indicating that the database cannot execute the command because there are constraints on executing the command	**Fail**

Error screenshot:



Analyze the cause of the error: When the database is created, username in the apply_table is restricted by the foreign key of username in the user table. Therefore, an apply_table data error may occur when data in the user table is deleted

```
@app.route(rule='/admin_delete_user', methods=['GET', 'POST'])
def admin_delete_user():
    if request.method == 'POST':
        username = request.form['username']
        user = User.query.filter_by(username=username).first()
        if user:
            db.session.delete(user)
            db.session.commit()
    user_list = User.query.all()
    return render_template(template_name_or_list='admin_delete_user.html', user_list=user_list)
```

How to fix

To fix this problem, add a step to the deletion process, that is, delete the corresponding data in the apply_table first, so that there is no foreign key constraint data. After deleting the corresponding user, the command can be successfully executed

```
@app.route( rule: '/admin_delete_user', methods=['GET', 'POST'])
def admin_delete_user():
    if request.method == 'POST':
        username = request.form['username']

        # Retrieve the user from the database based on the provided username
        user = User.query.filter_by(username=username).first()

        if user:
            # Attempt to delete related data in the apply_table before deleting the user
            try:
                # Delete entries related to the user in the apply_table
                ApplyTable.query.filter_by(username=username).delete()
                db.session.commit()
            except IntegrityError:
                # If a foreign key constraint error occurs, rollback the session and display an error message
                db.session.rollback()
                return 'Error: Cannot delete user. Related data in apply_table exists.'

            # Delete the user from the user table
            db.session.delete(user)
            db.session.commit()

    # Retrieve the updated list of users from the database
    user_list = User.query.all()

    return render_template( template_name_or_list: 'admin_delete_user.html', user_list=user_list)
```

Test again:

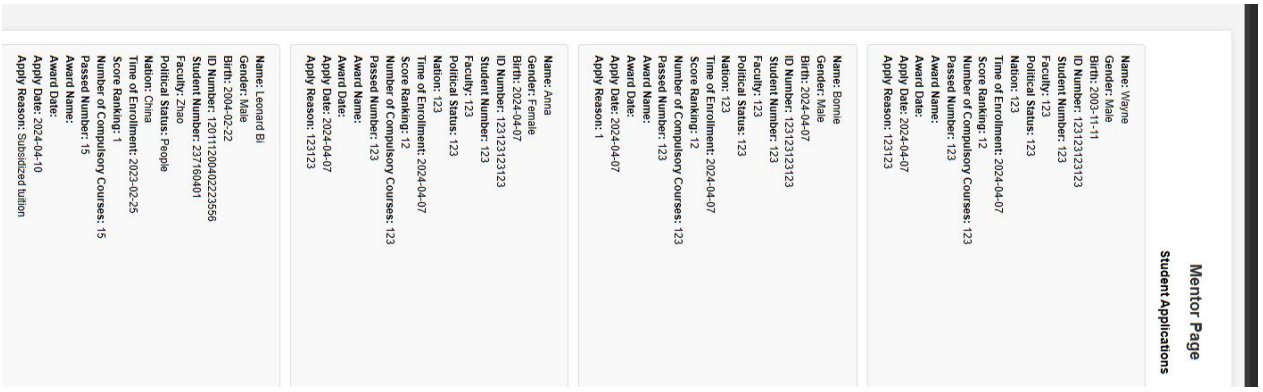
No	Action	Expected result	Actual result	Pass/Fail
26	Select user “123123” and delete	Prompt admin delete successfully and remove all data from database		**Pass**

3.7.3. Test No.28

Error

No	Action	Expected result	Actual result	Pass/Fail
28	Mentor login	Display all students in same major with mentor	All student information is displayed	**Fail**

Error screenshot:



Analyze the cause of the error: The preliminary determination of the problem of displaying all students' information is that the query command of the database has an error, there is no constraint or the wrong constraint condition is selected, thus displaying all students incorrectly

```
@app.route('/mentor_page')
def mentor_page():
    mentorname = request.args.get('mentorname')
    mentor = Mentor.query.filter_by(mentorname=mentorname).first()

    if mentor:
        applications = []
        for student in students:
            application = ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)
        return render_template( template_name_or_list: 'mentor_page.html', applications=applications)
    else:
        return "Mentor not found."
```

How to fix

When you change the query command, you look at the following three tables: major, user, apply_table, the common major, and the common username. In this way, all students can avoid being exported.

```python
@app.route('/mentor_page')
def mentor_page():
    mentorname = request.args.get('mentorname')
    mentor = Mentor.query.filter_by(mentorname=mentorname).first()

    if mentor:
        students = User.query.filter_by(major=mentor.major).all()
        applications = []
        for student in students:
            application = ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)
        return render_template( template_name_or_list: 'mentor_page.html', applications=applications)
    else:
        return "Mentor not found."
```

Test again:

No	Action	Expected result	Actual result	Pass/Fail
28	Mentor login	Display all students in same major with mentor	As expected	**Pass**

3.7.4. Test No.31

Error:

No	Action	Expected result	Actual result	Pass/Fail
31	Faculty login	Display all students that approved by mentor	Haven't display any students	**Fail**

Error screenshot:



Analyze the cause of the error: In contrast to the other routes, this route does not specify whether the code is about the POST method or the GET method, so the corresponding method is not used when getting the information, so it is not obtained

```python
@app.route('/faculty_page')
def faculty_page():
    passed_students = User.query.filter_by(applicationStatus='mentor passed').all()

    applications = []

    for student in passed_students:
        application = ApplyTable.query.filter_by(username=student.username).first()
        if application:
            applications.append(application)
            print(application)

    if not applications:
        return 'No applications found.'

    return render_template( template_name_or_list: 'faculty_page.html', applications=applications)
```

How to fix

Specify the corresponding if statement for the GET and POST methods in the route, and perform the operation.

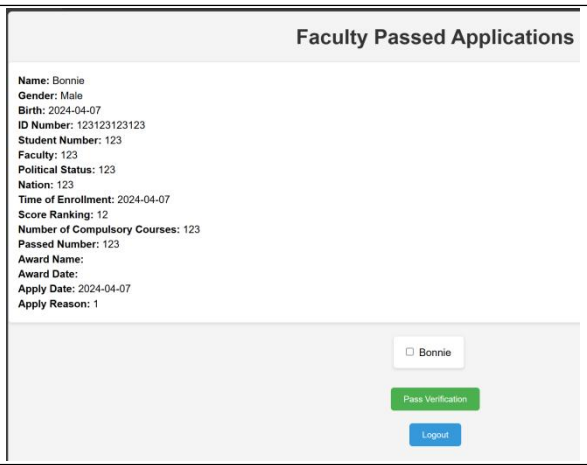
```python
@app.route( rule: '/faculty_page', methods=['GET', 'POST'])
def faculty_page():
    if request.method == 'GET':
        passed_students = User.query.filter_by(applicationStatus='mentor passed').all()

        applications = []

        for student in passed_students:
            application = ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)
                print(application)

        if not applications:
            return 'No applications found.'

        return render_template( template_name_or_list: 'faculty_page.html', applications=applications)
    else:
        pass
```

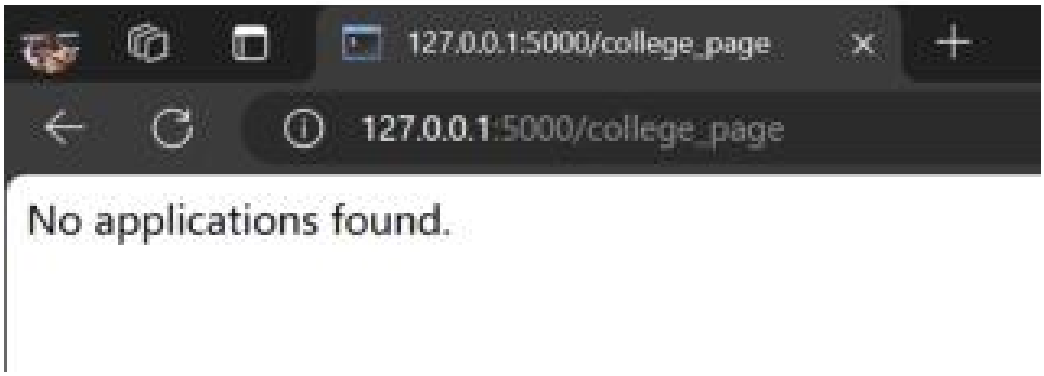
No	Action	Expected result	Actual result	Pass/Fail
31	Faculty login	Display all students that approved by mentor		**Pass**

3.7.5. Test No.34

Error:

No	Action	Expected result	Actual result	Pass/Fail
34	College login	Display all students that approved by Faculty, and display a search box on the top of website body	Display “No applications found.”	**Fail**

Error screenshot:



Analyze the cause of the error: When the college role does not receive any student application information in the database, the route directly adjusts the if statement to "No applications found" instead of the original search function.

```
@app.route( rule: '/college_page', methods=['GET', 'POST'])
def college_page():
    if request.method == 'POST':
        search_query = request.form.get('search_query')
        if search_query:
            passed_students = User.query.filter(User.username.like(f'%{search_query}%')).all()
        else:
            passed_students = User.query.filter_by(applicationStatus='faculty passed').all()

        applications = []

        for student in passed_students:
            application = ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)

        if not applications:
            return 'No applications found.'

        return render_template( template_name_or_list: 'college_page.html', applications=applications)
```

How to fix

Remove the return value from the if statement, and no student information is displayed in the if statement in the html

```
            <p><strong>Apply Reason:</strong> {{ application.ApplyReason }}</p>
            </li>
        {% endfor %}
        <form action="{{ url_for('college_pass_verification') }}" method="post">
            <ul>
                {% for application in applications %}
                    <li>
                        <input type="checkbox" name="selected_students" value="{{ application.name }}">
                        <label>{{ application.name }}</label>
                    </li>
                {% endfor %}
            </ul>
            <button type="submit">Pass Verification</button>
        </form>
    {% else %}
        <p>No applications found.</p>
    {% endif %}
</ul>
```

```
@app.route( rule: '/college_page', methods=['GET', 'POST'])
def college_page():
    if request.method == 'POST':
        search_query = request.form.get('search_query')
        if search_query:
            passed_students = User.query.filter(User.username.like(f'%{search_query}%')).all()
        else:
            passed_students = User.query.filter_by(applicationStatus='faculty passed').all()

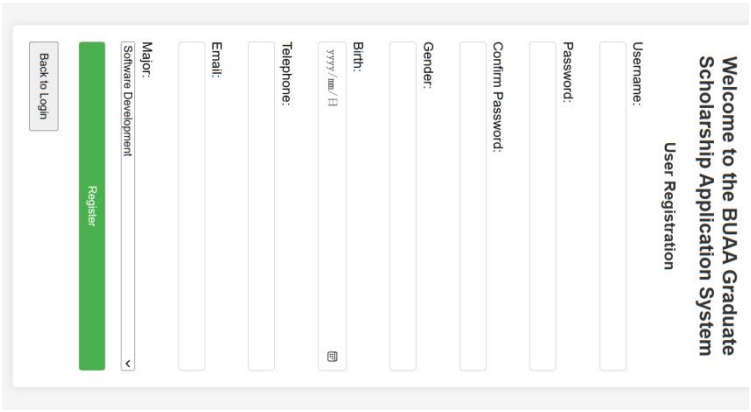
        applications = []

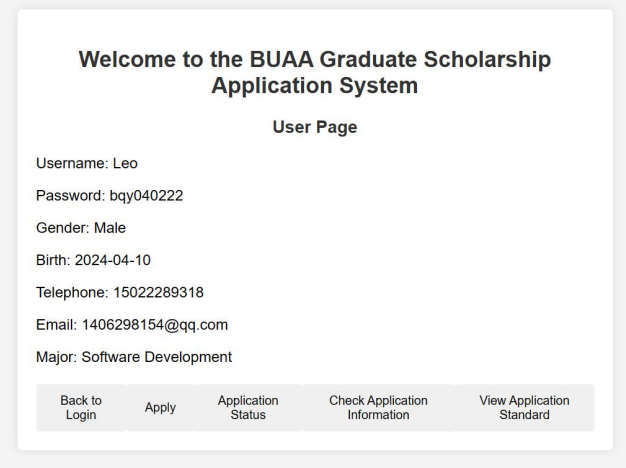
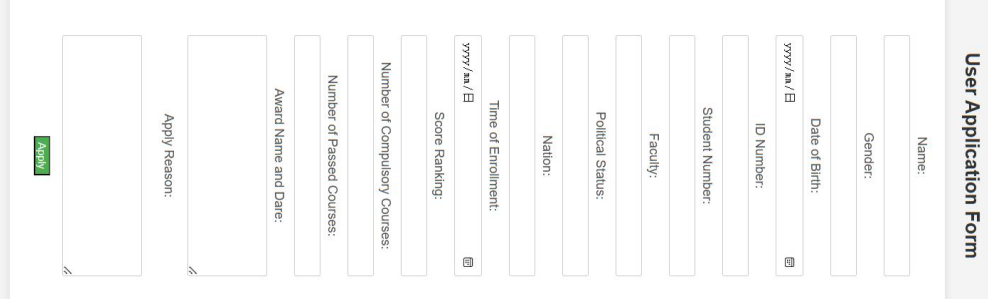
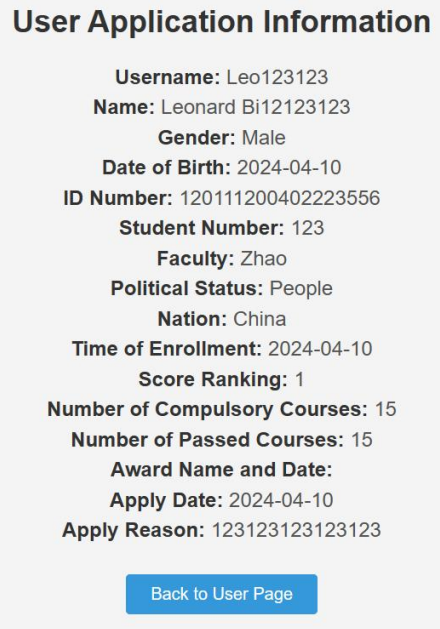
        for student in passed_students:
            application = ApplyTable.query.filter_by(username=student.username).first()
            if application:
                applications.append(application)

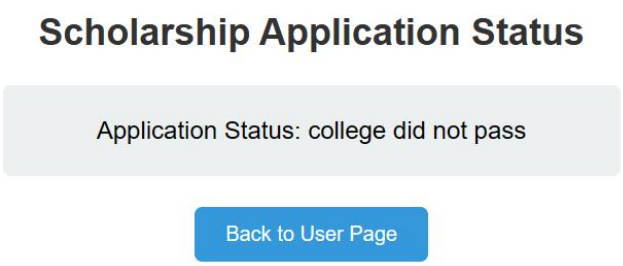
        return render_template( template_name_or_list: 'college_page.html', applications=applications)
```

3.8. User documentation

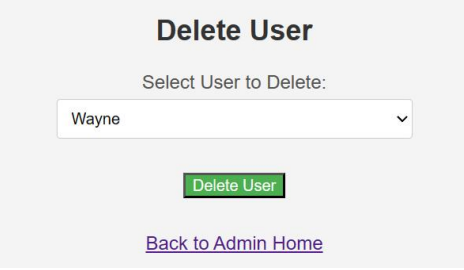
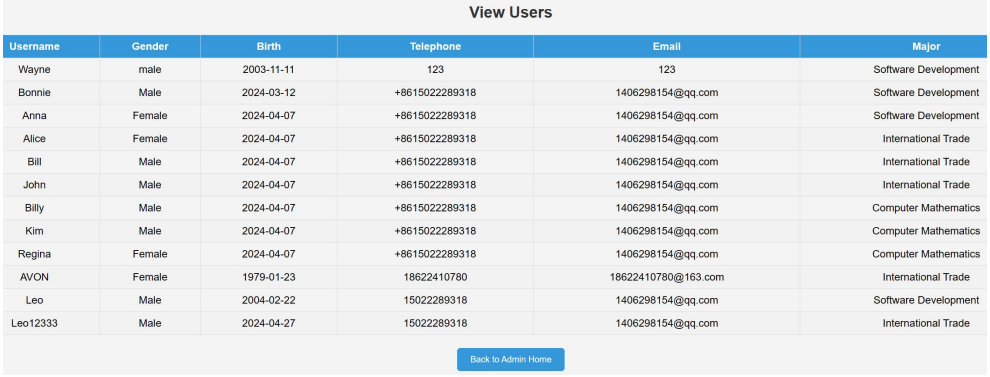
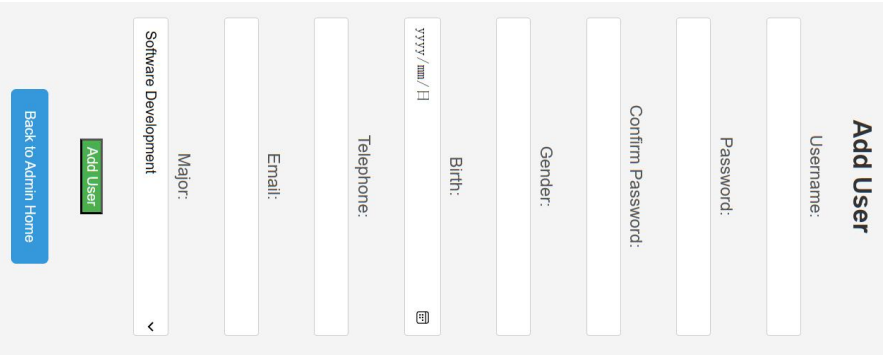
User role manual:

Step	Action
1.Click hyperlink and Sign up for us	

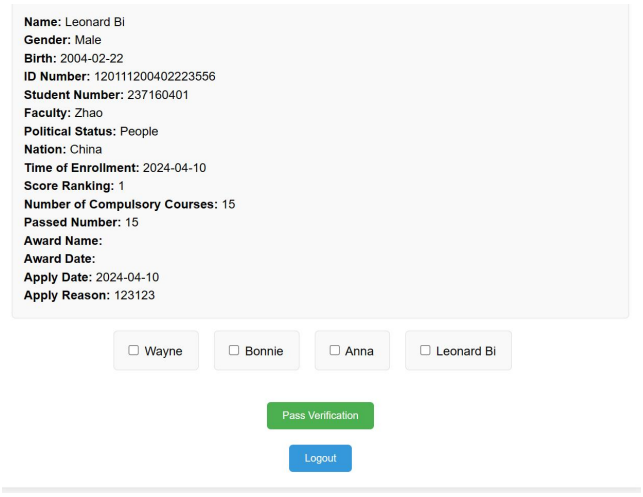
2.Login into user page	
3.Fill in information and apply for scholarship	
4.Could update information any time	
5.Check submitted application any time	

6.Waiting for update application status	

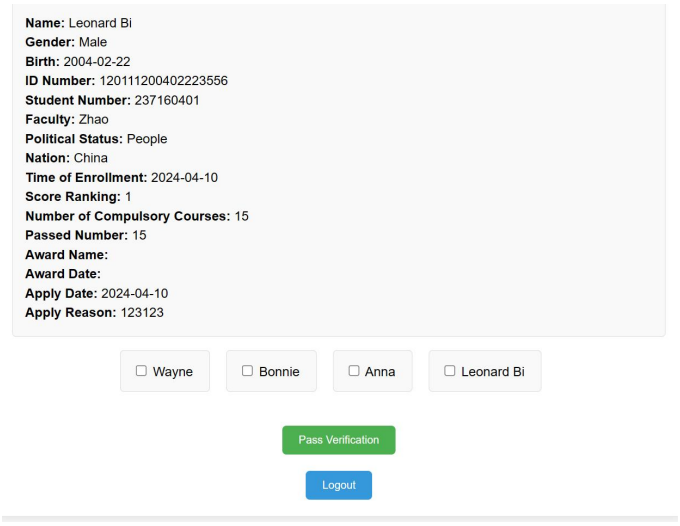
Admin role manual:

Step	Action
1.Login as administrator	
2.Click button to delete, view or add user	  

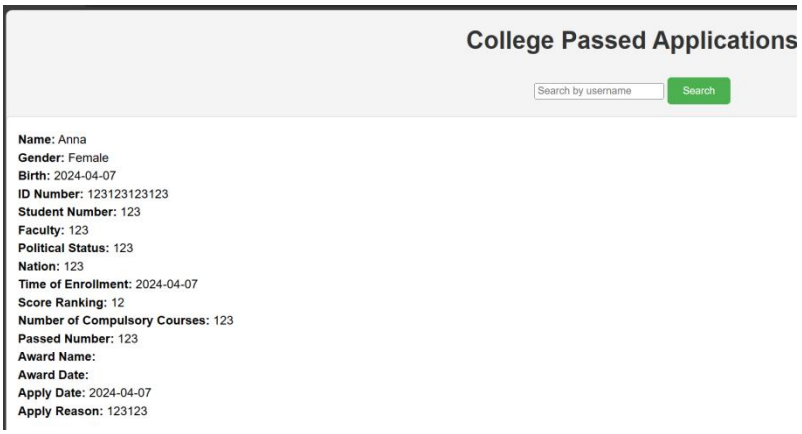
Mentor role manual:

Step	Action
1.Login as mentor	
2.Choose students who meet the criteria and click pass verification	

Faculty role manual:

Step	Action
1.Login as faculty	
2.Choose students who meet the criteria and click pass verification	

College role manual:

Step	Action
1.Login as college	
2.Choose students who meet the criteria and click pass verification	
3.Select students by search box	

4. Evaluation

4.1. An outline of the assignment and to what extent the solution met the original requirements of the assignment brief

The purpose of this project is to develop a set of scholarship application management system for undergraduates in Beihanguniversity, providing a convenient application and management platform for students, tutors, departments and faculties. Flask will be used as programming language and MySQL as database for the development of this system.

This project involves four main roles: user, mentor, faculty, and college. Users will be able to check personal information, complete and submit scholarship application forms, upload relevant information, and check application results. mentor will manage student information, review application forms and materials, provide advice, and perform initial screening. The faculty will collect and re-review student applications and submit them to the college for decision making. The college will set the application criteria, summarize the results of each department's selection and make a final decision, as well as provide feedback to the user. This process will ensure the efficient functioning and impartiality of the scholarship application management system.

The project will begin by researching the scholarship application requirements of the school and studying the scholarship application system of other schools. In the development process, it will be divided into planning, development, testing, evaluation and other stages to ensure the stability and reliability of the system.

While the project fulfilled most of the design requirements, the unfinished parts included college roles providing scholarship application requirements and roles explaining to users why applications were accepted or rejected. During the background investigation, I found that the scholarship application criteria were extremely strict and had a lot of words. If these are stored in the database and can be changed by the college role, first of all, there is no guarantee that the role will have a

good change experience, and second, there is no way to make these standards well displayed on the page (there is no suitable css file to adapt). Therefore, this feature was abandoned in this project

4.2. An assessment of the strengths and weaknesses of the outputs of the practical assignment

For this program, the interconnection between each role can make the whole system more complete. This is also reflected in the database, where each table has a foreign key corresponding to the other tables. When each role logs in, the username is passed between pages to ensure the continuity of the role when operating the system. And can also bring good operating experience for users. Each successful operation or failed operation will have an alert message, which also makes the human-computer interaction more efficient.

But the system's shortcomings are also obvious. Most intuitively speaking, first of all, this system does not have a good css file, it is not very beautiful, and it is not very intuitive to read when viewing information. Second, in addition to the user and admin roles, the functions of the remaining three roles are somewhat repetitive, although they are divided into three different levels of approval, but each level should have some features. However, this is also reflected in the role of college, which can query students and then approve them.

4.3. Recommendations for any future development of the solution and reasons for these recommendations

For the future development of the project, the following solutions are proposed to further enhance the function and efficiency of the system. First, the function of the College role in providing scholarship application conditions should be improved so that the College role can publish clear application conditions in the system to improve the quality and accuracy of applications.

Secondly, the role of each role in explaining the application results to students is strengthened to enhance the transparency and fairness of the application process. In

addition, the development of data analysis and reporting capabilities for regular analysis and reporting of application processes and results helps management optimize decisions and processes.

Finally, strengthen the security and stability of the system to ensure the security of user data and the reliability of the system. The implementation of these proposed solutions can further enhance the user experience, enhance the efficiency and transparency of the system, and provide strong support for the continuous development and optimization of the university's scholarship application management system.

4.4. A summary of any modifications to the project plan, solution design and/or implementation, that were made during the project. Including reference to any unforeseen events and how they were handled

During the background investigation, I found that the scholarship application criteria were managed by the top administrator of the school, and it required multiple administrators to discuss how to set the criteria, which also made the criteria very rigorous and varied. This also makes college unable to reflect this function in the database, and the direct output text can not have a good css file to beautify, so that the user role can better read the application criteria. This leaves college with fewer operational features, and it has to add some other features to enrich the system.

But it did encounter some unpredictable problems. In February 2024, due to excessive dust, the computer suddenly suffered a blue screen of death during operation, and the computer that opened the database at the same time to insert data and python had no chance to save all the data, which caused me to lose part of my work, and this is irreversible. I operated the database through MySQL workbench, and I had to click apply every time to execute the database command. However, python has an automatic save function, so that I can effectively retrieve some query statements in the route to solve computer hardware problems, so as to achieve the database.

4.5. Identification of any knowledge and skills which have been gained or

developed while carrying out the project assignment and how the actions/process of carrying out the project could have been improved

During the development of this project, I not only accumulated a lot of technical knowledge, but also cultivated many skills in practical operation. By developing this project independently, I delved into the Flask framework and gained a better understanding of how it works and best practices. I learned how to build Web applications with Flask, handling critical tasks like routing, rendering templates, and interacting with databases.

In terms of database, through practical operation of MySQL, I not only learned how to design database schema efficiently, but also mastered basic skills such as adding, deleting, correcting and checking data, and had a deeper understanding of database performance optimization. In addition, in order to ensure the security of the system, I studied the user authentication and authorization mechanism, strengthened the verification of user input data, and prevented potential security vulnerabilities.

In addition to the technical improvement, this project also trained my problem-solving ability. In the process of development, I encountered many unexpected problems, such as data consistency assurance, system performance optimization and so on. These problems forced me to conduct in-depth research and practice, and finally found an effective solution.

As a one-man development team, I adopted a variety of strategies to improve the project. First of all, I made a detailed project plan and carried it out strictly. This helps me manage my time and ensure that all phases of the project are completed on time.

Second, I made full use of online resources such as tutorials, forums, and blogs to solve problems encountered during the development process. These resources not only provide technical support, but also inspire me to think creatively about the functionality of the project.

I also conducted many tests and debugging to ensure the stability and reliability of the system. After each test, I tweak the code based on the test results, fix potential

problems, and optimize system performance.

Finally, I focus on code readability and maintainability. I followed good programming practices and used clear naming and comments so that I could quickly understand the logic and functionality of the code in future maintenance

5. References

[1] https://tutorial.helloflask.com/ [Accessed 2023/12/28]
[2] https://gjps.xszz.moe.edu.cn/#/index [Accessed 2024/1/21]
[3] https://www.w3school.com.cn [Accessed 2024/1/21]
[4] https://www.csdn.net/ [Accessed 2024/2/9]
[5] https://www.jetbrains.com/pycharm/[Accessed 2024/2/9]
[6] https://dev.mysql.com/downloads/workbench/[Accessed 2024/2/9]
[7] https://www.python.org/[Accessed 2024/2/9]
[8] https://flask-sqlalchemy.palletsprojects.com/[Accessed 2024/2/9]
[9] https://stackoverflow.com/[Accessed 2024/2/9]
[10] https://www.sqlalchemy.org/[Accessed 2024/2/9]
[11] https://dev.mysql.com/doc/[Accessed 2024/3/5]
[12] https://dormousehole.readthedocs.io/en/latest/[Accessed 2024/3/5]
[13] https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world[Accessed 2024/3/5]
[14] https://flask.palletsprojects.com/en/2.0.x/[Accessed 2024/3/5]

6. History of changes to Report

Version	Description of change	Date
1-1-BiQiaoyang GU2 version1.docx	Analyze and Design Part of the report	2024/1/21
1-1-BiQiaoyang GU2 version2.docx	Coding and Development Part of the report	2024/3/5
1-1-BiQiaoyang GU2 version3.docx	Evaluation part of report	2024/3/28
1-1-BiQiaoyang GU2 version4.docx	Refine class descriptions and data dictionaries	2024/4/10

History of changes to Program

Version	Description of change	Date
1-1-BiQiaoyang GU2 version1	Program basic framework	2024/1/21
1-1-BiQiaoyang GU2 version2	The functions of user roles and administrator roles are complete	2024/2/9
1-1-BiQiaoyang GU2 version3	Coding of mentor, faculty, college role function	2024/3/5
1-1-BiQiaoyang GU2 version4	Debugging, final version of program	2024/4/1